TECHNISCHE UNIVERSITÄT WIEN

NEUTRONS FOR SOCIETY

BACHELORARBEIT

*q*B*ounce*: Autonomous Detection and Characterization of Particles within the CMOS Sensor

im Rahmen des Studiums

Technische Physik

vorgelegt von

Loïc Posta

Matrikelnummer: 12505712

Durchgeführt am
Atominstitut der Fakultät für Physik der Technischen Universität Wien
in Zusammenarbeit mit
Institut Laue-Langevin (ILL), Grenoble

Betreuung:
Begutachter: Privatdoz. Dipl.-Ing. Dr. Johann Marton
TU Wien / Atominstitut
Betreuer: Univ.Ass. Dipl.-Ing. Dr.techn. Joachim Bosina (TU Wien / Atominstitut)
Mitwirkung: Univ.Prof. Dipl.-Phys. Dr.rer.nat. Dr.h.c. Hartmut Abele (TU Wien / Atominstitut)

Wien, 3. September 2026

Abstract

The *q*B*ounce* experiment studies gravitational quantum states of ultracold neutrons, which appear when a neutron bounces above a polished mirror in Earth's gravitational field. It measures a counting rate, so what it can say about gravity at the micrometre scale depends directly on how reliably single particles are detected. The detector is a CMOS image sensor with a ^{10}B converter layer: a captured neutron sends a lithium ion or an α particle into the silicon, and the charge collected along its path is read out with the frame as a cluster of ten to twenty pixels. Unlike the CR39 sheets previously used for this measurement, which record position without time and must be etched before they can be read, a CMOS sensor records both coordinates and can be read during the run. It also produces about 108 frames a minute at 20.1 Mpixel, some 215 clusters in each, which is what makes an automatic evaluation necessary rather than convenient.

This thesis presents the detection and classification pipeline for that sensor. A `C++` core learns the sensor's own noise one pixel at a time from frames taken in the dark, flags the pixels that rise significantly above it, groups them into clusters and characterises each by fifteen measured quantities. A Python layer labels a sample of those clusters by hand, trains a random-forest classifier on them and applies it to the rest. A graphical interface drives the whole chain, and a self-contained kit installs it on the offline machine at the beamline.

The pipeline processes a 20.1 Mpixel frame in 169 ms on Apple Silicon, nearly six frames a second against the two the camera delivers, and 561 ms on the low-power Windows laptop the laboratory uses, which is exactly the interval between frames and leaves no margin. Booting that same laptop into Linux brings it to 372 ms, a third of the period to spare, so the margin the laboratory lacks is available on the hardware it already owns — for the analysis, the camera never having been driven under Linux. Three compilers on two architectures return the same 39 071 clusters, so the result does not depend on the machine that produced it. Over 1 447 466 clusters found in beam-off frames with the hot-pixel filter switched off, none was classified as a neutron, and the classifier's accuracy on the neutron category is 95.75 ± 2.20 % over a thousand retrainings. The counting statistics were tested against Poisson instead of assumed to follow it, and they pass: the Fano factor is 0.977 ± 0.037. The counted rate falls by 3.1 ± 1.4 % over the run, which bounds any monotonic drift at 6 %. Losing 34 850 pixels during a single 1500-frame run does not measurably bias that rate. Three-fifths of them fall where the beam does; the rest are independent of it.

Two limits remain, and both need beam time rather than analysis. At the gain used, essentially every heavy track saturates the sensor, so the deposited-energy spectrum that would separate lithium from α is not yet readable. That separation was one of the stated motivations for a pixel detector, and this work does not achieve it. And the instrumental part of the false-positive floor requires a shielded acquisition that has not been made. What the pipeline delivers today is a neutron count with an uncertainty attached to it, produced fast enough to watch a run while it happens — on the two platforms with the margin to do so, neither of which has yet been connected to the camera.

Contents

1. Introduction

Gravity is the oldest and most familiar force in physics, from Newton's *Principia* (1687) to its description as a smooth curvature of spacetime with General Relativity (1915), yet it remains the least understood at the quantum level. At this scale quantum mechanics governs the discrete, probabilistic behaviour of particles, and the two frameworks have resisted unification for nearly a century. Most tests of gravity operate at scales where quantum effects are completely negligible: planets, satellites, and even atom interferometers probe length scales far above the regime where quantum discretisation of gravitational energy becomes observable.

Gravitational quantum states of neutrons were first observed by Nesvizhevsky *et al.* [1]. The goal of the *q*B*ounce* experiment [2, 3] is to provide a unique study niche: it observes gravitational quantum states of neutrons at the scale of tens of micrometres. The motion there is quantum mechanical, and gravity is what confines it: the two enter the same Schrödinger equation, the gravitational potential mgz setting a level spacing the experiment can resolve. That regime is what lets the experiment set bounds — on the Yukawa coefficient, for instance — on theories beyond the Standard Model, and sharpen our understanding of how gravity acts at the quantum scale.

A neutron bouncing above a polished mirror in Earth's gravitational field is the simplest possible quantum bouncer. Its *vertical* motion is bound, like an electron in a hydrogen atom, but the binding potential is not electromagnetic; it is purely gravitational. Horizontally it is not bound at all: neutrons are shot through the apparatus and leave it at the far end, and the whole measurement happens during that flight. Trapping ultracold neutrons in a closed volume is a different experiment, one this group is also pursuing, and it relies on a material potential step too high for the neutron to climb rather than on gravity.

What makes this remarkable is the energy scale. Gravitationally bound states of a neutron have energies of order 1–5 peV — twelve orders of magnitude smaller than the energies of atomic electrons. Detecting and manipulating such states requires techniques borrowed from atomic physics: resonance spectroscopy. And because neutrons are electrically neutral, carry no electric dipole moment, and are only weakly polarisable, they are essentially immune to the electromagnetic noise that plagues precision experiments with charged particles or atoms. This makes them uniquely clean probes of short-range gravitational physics. A non-technical presentation of the experiment and of the free-falling neutron is given in [4].

To reduce the statistical noise and be able to probe new physics, one needs to have a sufficient flux of neutrons entering the chamber. To improve those rates, the experiment is regularly conducted in Grenoble at the Institut Laue-Langevin (ILL) where I had the opportunity to participate in a beam test for 10 days, mounting and testing the CMOS sensors that the detection software of this thesis processes. That gave first-hand experimental context for the software work that follows, and many of the problems encountered there motivate the software choices of Section 3.

Three kinds of detector can count those neutrons, and the choice decides what the experiment can ask. A proportional counter, the instrument the experiment uses at the end of the spectrometer [5], counts them better than anything else. It converts on a ^{10}B layer, the same capture the CMOS relies on, so what separates the two is not the physics but the readout: the counter integrates over its volume and returns no position, and a gravitational state is a structure in height. A CR39 sheet returns position better than any pixel grid, and returns nothing else: every neutron of the run lands on one surface with no arrival time, and the sheet has to be etched

for five hours and then scanned before there is anything to look at [6], so a problem during a beam time is discovered days later. A CMOS sensor returns both. Every frame is a time slice and every cluster within it carries its pixel positions, so a track records where the neutron landed and when. Section 2.4 returns to the three in detail.

What a CMOS sensor also returns is a great deal of data. The camera is asked for two frames a second and sustains 1.8, so a run produces about 108 images a minute at 20.1 Mpixel each, and the detector finds some 215 clusters in every one of them. A fifteen-hundred-frame run is thirty gigapixels and three hundred thousand objects, of which a few tens of thousands are neutrons and the rest is the sensor talking to itself. Nobody sorts that by hand, and nobody sorts it afterwards either if the point is to know during the run whether the beam is still there. The detector makes the automatic evaluation necessary rather than convenient.

That is what this thesis builds, and it runs fast enough to keep up. On Apple Silicon the pipeline processes a 20.1 Mpixel frame in 169 ms, which is nearly six frames a second, three times what the camera delivers; on the low-power Windows laptop the lab actually uses it takes 561 ms against the 557 ms between frames, which is exactly at the limit and is measured in Section 3.6. The margin exists on the other two platforms and neither of them has been connected to the camera, so what is demonstrated today is that an operator *could* watch the particle count rise while the beam is on, and see a dead run in the first minute rather than in the analysis three weeks later. Section 5 says what closing that last gap takes; it is an installation rather than a design.

The goal of this thesis is to design, implement and validate a CMOS-based particle detection and classification pipeline: a `C++` background-modelling and cluster-detection core, written for speed so that the counting rate can be followed live, and a `Python` labelling/ML-classification layer which analyses the data and manages the user interface.

What that pipeline lets someone do, once it is installed, is four things. An operator at the beamline starts a run and watches the neutron count rise frame by frame, with the interface turning red the moment frames are being dropped. An analyst points it at a folder of recorded frames and gets a rate with an uncertainty on it, traceable to the exact command that produced it. Someone studying the sensor itself gets the map of which pixels died and where, which is how Section 4.7 found that the beam kills pixels where it illuminates. And a student arriving on the experiment installs the whole thing on the offline machine at the ILL from a folder, with no network, and has it running the same afternoon.

Automated track classification for this experiment is not new, but it has only been done on a different detector. [6] developed, in the same group, a machine-vision pipeline for CR39 solid-state track detectors: neutrons converted in a ^{10}B layer, the resulting damage etched and scanned under a microscope, and the tracks separated from cracks and dust by a random forest. The conversion physics is identical to the one used here; everything between the capture and the classifier is not. No equivalent exists for a CMOS sensor, which is what this thesis builds, and Section 4.8 sets the two side by side — what is followed, what is reproduced, and what departs.

Two things the pipeline was not built to find came out of using it, and both are reported here because they bear on what the detector can be asked to do next. The first is a limit: at the gain these acquisitions used, almost every track clips at the top of the 8-bit range, so the deposited-charge axis is a lower bound and the lithium and α branches of the capture cannot be told apart. That is not a defect of the analysis and no re-analysis repairs it — the information was lost in the sensor before any software saw it — and what it costs is a class the classifier cannot have. The second is a discovery: the sensor loses pixels while it counts, permanently, and

it loses them where the beam falls. Neither was anticipated when the work started.

The same measurements suggest what the sensor could do that a counter cannot, and Section 5 develops it. The spectrometer analyses the gravitational states by destroying them: a rough surface at a chosen height removes the neutrons that reach it, and what is counted downstream is a transmission — one number per setting, the height information used and thrown away. A pixel sensor at that same plane would record where each neutron was when it arrived, so instead of a transmission it would return the vertical density, and the population of every state could be fitted from it at once, each with its own error bar. The thesis puts numbers on that: what the scales allow, what a scan would cost, and the one systematic that would manufacture the effect being looked for. Because the pipeline classifies a frame faster than the camera delivers the next, it could also be done while the beam is on, which turns a resonance scan into something an operator steers rather than reads afterwards. At the precision the laboratory works to that turns an hour-and-a-quarter scan into eight minutes, a factor of nine, and Section 5 separates the part of it that is the sensor from the part that is the scheduling.

One methodological choice runs through all of it and is worth stating at the start, because it decides what the later sections are allowed to say. A number that appears in this thesis is either measured, or derived in front of the reader from numbers that are. Where a quantity could not be measured with the data available — the spatial resolution of the sensor, the instrumental part of the false-positive floor, the share of the noise excess that is not accounted for — it is named as missing rather than estimated, and Section 5 lists what each one would take. Three compilers on two architectures were run over the same frames to check that the pipeline is not quietly a property of one machine; they return the same clusters, which is a weaker claim than correctness and a necessary one. Where a result rests on an assumption, the assumption is given its own sentence and, where the figure allows it, the figure prints how the conclusion moves if that assumption is wrong.

The thesis is structured as follows. Section 2 motivates the experiment: why the neutron is the probe, how its states above a mirror are quantised, how the Ramsey method drives transitions between them, what the five-region spectrometer looks like and where the CMOS detector developed in this thesis sits within it (Section 2.4), the precision the method reaches, and why reaching it leaves no choice but to analyse the detector's frames automatically. Section 3 presents the detection software on its own, so that it can be read without the physics: the data flow, the architecture, what the detector decides and on what grounds, three commented excerpts, the charge it measures, what a frame costs to process, and the two ways the chain is run. It quotes its own measured performance from Section 4 as it goes, so the two are best read in that order. Section 4 reports what the pipeline measures on the July 2026 datasets, and Section 5 draws the conclusions, the measured performance and an outlook on what remains.

2. How to Probe the Physics of Gravity?

Quantum field theory describes the strong, weak and electromagnetic interactions to great precision, and general relativity describes gravity; what neither does is describe gravity at the quantum scale. *q*B*ounce* probes that regime by using a quantum particle as the probe. This section motivates the experiment, explains what it measures, and closes with the motivation for the work of this thesis.

2.1. Why Neutrons?

What kind of physical system allows us to probe gravity while being described by quantum mechanics? The four fundamental interactions have very different ranges and orders of magnitude, as summarised in Table 2.1.

Interaction	**Relative strength**	**Range [m]**	**Mediator / acts on**
Strong	1	$\sim 10^{-15}$	gluons; quarks and hadrons
Electromagnetic	$\sim 10^{-2}$	∞	photon; electric charge
Weak	$\sim 10^{-6}$	$\sim 10^{-18}$	$W^{\pm}, Z^{0}$; all fermions
Gravitation	$\sim 10^{-38}$	∞	(graviton, hypothetical); mass–energy

Table 2.1: The four fundamental interactions: relative coupling strength and range. Strengths are quoted relative to the strong interaction taken as 1, for two protons separated by roughly one femtometre — a convention, not an absolute property, since the ratios depend on the particles and the distance chosen. Ranges follow from the mediator mass: a massless mediator gives an infinite range, whereas the massive W and Z bosons confine the weak interaction to a distance of order $\hbar/(m_{\mathrm{W}}\, c)$. Values from [7].

The table makes the experimental strategy visible. The weak interaction is confined to $\sim 10^{-18}\,\mathrm{m}$ and the strong one to $\sim 10^{-15}\,\mathrm{m}$, so beyond a femtometre only electromagnetism and gravitation are left — and of those two, electromagnetism is some 36 orders of magnitude stronger. Isolating gravitation therefore means suppressing the electromagnetic coupling as far as possible, which is a statement about the choice of probe particle rather than about the apparatus.

An electrically neutral probe is the obvious first move, and an atom is the obvious candidate. It is not good enough. An atom such as rubidium is highly polarisable: its static electric polarisability is $\alpha_{\mathrm{Rb}} \approx 5.3 \times 10^{-39}\,\mathrm{C\,m^2/V}$, so its electron cloud distorts easily in an external field and the electromagnetic coupling survives its neutrality. That has a very concrete consequence a few micrometres above a surface, which is exactly where these experiments happen. A polarisable atom feels the van der Waals attraction of the mirror, and instead of bouncing above it, it sticks to it. The neutron's polarisability is $\alpha_{\mathrm{n}} \approx 1.3 \times 10^{-58}\,\mathrm{C\,m^2/V}$, smaller by nearly twenty orders of magnitude, and it carries no electric dipole moment either. It bounces where an atom would adsorb, and the gravitational interaction is left as the dominant one at these scales. Figure 2.1 contrasts the two.

The neutron does carry a magnetic moment, because it carries spin 1/2 and a charged internal structure, so a magnetic field splits its two spin states. That is not a problem here, because the whole assembly sits inside a μ-metal shield in the vacuum chamber (Section 2.4.1), which suppresses the residual field to the point where the magnetic coupling is negligible against the

gravitational one. That shield is what makes the measurement possible, and it also marks what is unusual about it. The states themselves are gravitational either way — they come from the quantised vertical motion of a neutron confined by gravity above a mirror — and both ways of driving a transition between them are called gravity resonance spectroscopy, GRS, in the literature. What differs is the handle. *q*B*ounce* vibrates the mirror: in the frame of the mirror that vibration is a modulation of g itself, so the transition is driven *mechanically* and never couples to a charge or a moment at all, and the absence of any electromagnetic handle is the novelty [2, 8]. The alternative couples to the neutron's magnetic moment with an oscillating field *gradient* ∇B, proposed for the GRANIT flow-through arrangement [9]. Our group calls that one BRS, after the field B, though the published name for it is still GRS. Both measure the same gravitational level splitting. The μ-metal shield is what keeps the mechanical drive clean, by removing the magnetic handle rather than by making it impossible.

Secondly, to trap the neutrons in a chamber and do experiments on them, one needs low-energy neutrons — equivalent to low speeds, $\sim$8 m/s, against the $\sim$2200 m/s of thermal neutrons — so that they do not pass through the walls of the chamber and mechanical instruments have more time to interact with them. The free neutron's lifetime of about 15 minutes leaves ample room for this. This is why the experiment is conducted at the Institut Laue-Langevin (ILL) in Grenoble, France, where the world's most intense steady-mode source of ultracold neutrons (UCNs) is available, with mean horizontal velocities in the range $v_x \approx 6$–9 m/s [10].

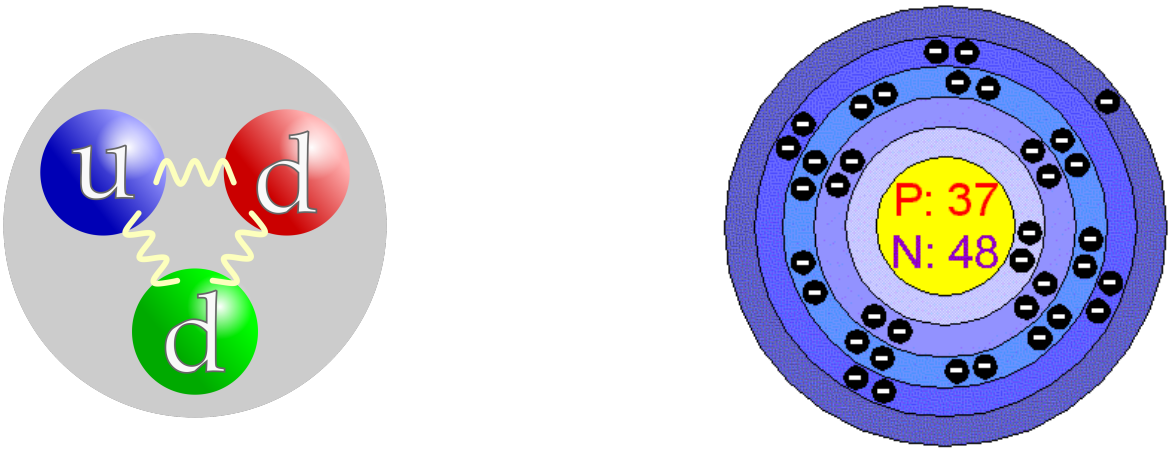


Figure 2.1: Both are electrically neutral, and only one of them is also barely polarisable. Right: a rubidium atom, $Z = 37$ with 48 neutrons, carries as many electrons as protons, so its net charge is zero — but the negative charge sits in a cloud on the outside while the positive charge sits in the nucleus, and an external field displaces one against the other. That separation is what polarisability is, and it is why neutrality alone does not make an atom a clean probe. Left: the neutron is neutral in the same sense, two down quarks and one up quark bound by gluons, but its charge structure is confined to about a femtometre, so the same field barely distorts it.

Source: left, *Neutron quark structure (ring-shape)*, by Harp, 2006, via Wikimedia Commons. Right, atomic-structure schematic, via ResearchGate, where it is captioned as iron although the nucleus drawn is $Z = 37$, rubidium.

2.2. Gravitational States

Once the gravitational interaction has been isolated, the neutron above the mirror is described by the Schrödinger equation with the gravitational potential alone:

$$-\frac{\hbar^2}{2m_\mathrm{n}}\frac{\partial^2}{\partial z^2}\psi_n(z) + m_\mathrm{n} g z \psi_n(z) = E_n \psi_n(z), \tag{2.1}$$

where m_n is the neutron mass, g the gravitational acceleration, z the vertical position and E_n the energy of the state $\psi_n(z)$.

The shape of the potential decides the shape of the solutions. In an infinite square well the potential is flat between two walls and the eigenfunctions are the familiar sinusoids. Here the potential is not flat but *linear*, $V(z) = m_\mathrm{n} g z$ above the mirror, and Eq. (2.1) is no longer solved by sines: substituting a linear potential turns it into Airy's equation, whose solutions are the Airy functions. Of the two independent solutions, Bi diverges as $z \to \infty$ and is discarded on physical grounds, leaving

$$\psi_n(z) \;\propto\; \mathrm{Ai}\!\left(\frac{z}{z_0} - \frac{E_n}{E_0}\right), \qquad z_0 = \left(\frac{\hbar^2}{2m_\mathrm{n}^2 g}\right)^{1/3}, \tag{2.2}$$

with $z_0 \approx 5.9\,\mu\mathrm{m}$ setting the natural length scale of the problem — which is why the states extend over tens of micrometres rather than the ångstrøms of atomic physics.

The mirror supplies the second condition. Its potential is enormous compared with the neutron's kinetic energy, so to a very good approximation the neutron does not penetrate it at all and the wavefunction is required to vanish at the surface, $\psi_n(0) = 0$. This is a Dirichlet boundary condition, and imposing it quantises the spectrum: the allowed energies are fixed by the zeros of the Airy function, and Figure 2.2 shows where those zeros fall. The ground state is of order $1.4\,\mathrm{peV}$, and the states built on the higher zeros are those of Figure 2.3.

The hard wall is an idealisation. A neutron does penetrate the mirror slightly, so the true condition is not exactly $\psi(0) = 0$ but a Robin condition relating the wavefunction and its derivative at the surface through a penetration length — conceptually the same device as the London penetration depth in superconductivity. Generalised boundary conditions of this kind, and the shifts they induce in the transition frequencies, are treated in [11].

Those turning points are also what makes the states selectable, and that is how the experiment prepares them. A rough surface clamped at a fixed height above the mirror absorbs any neutron whose wavefunction reaches up to it, so the height of that surface cuts the spectrum at a chosen state: a plate at $25\,\mu\mathrm{m}$ sits above the turning points of $|1\rangle$ and $|2\rangle$, $13.7\,\mu\mathrm{m}$ and $24.0\,\mu\mathrm{m}$, and below the $32.4\,\mu\mathrm{m}$ of $|3\rangle$ and everything above it, so the two lowest states pass and the rest are scattered out within milliseconds. That is the job of region I of the spectrometer, described with the rest of the mirror lane in Section 2.4.2, and it is why the level scheme of Figure 2.4 shows the whole ladder E_1 to E_N arriving at the first oscillating region while only the lowest one or two are still there afterwards.

2.3. One and Three Zones: Rabi to Ramsey

The transfer of Ramsey's method of separated oscillating fields to gravitationally bound states is described in [12], the resonance-spectroscopy methods themselves in [8], and the original realisation of the quantum bouncer in [13]. The transitions between these states are driven by

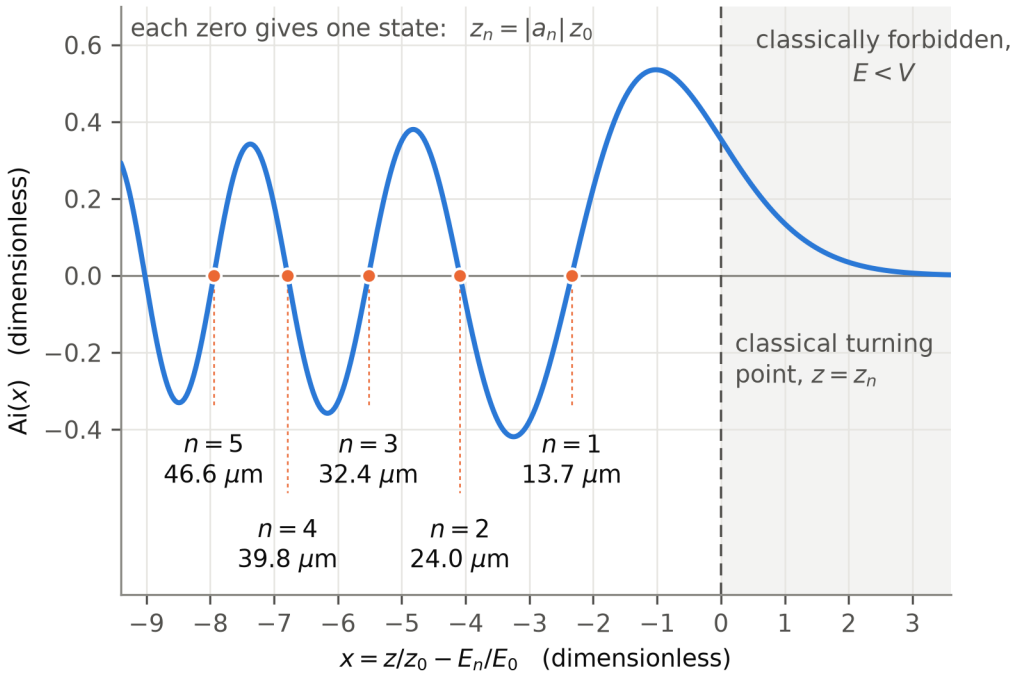


Figure 2.2: The Airy function Ai and the first five of its zeros. It oscillates for negative argument and falls off exponentially for positive argument, which is what makes it the physical solution here: the neutron's wavefunction decays into the classically forbidden region above its turning point instead of diverging, and Bi is discarded for failing exactly that test. The argument plotted is the one of Eq. (2.2), so $x = 0$ is the classical turning point and the mirror, where $\psi(0) = 0$ has to hold, can only sit on a zero. That is what quantises the spectrum: the n-th zero a_n fixes the n-th energy, $E_n = |a_n|\, m_\mathrm{n} g z_0$, and with it the classical turning point $z_n = |a_n|\, z_0$. The five marked here give $13.7\,\mu\mathrm{m}$, $24.0\,\mu\mathrm{m}$, $32.4\,\mu\mathrm{m}$, $39.8\,\mu\mathrm{m}$ and $46.6\,\mu\mathrm{m}$, the same five heights at which the states of Figure 2.3 turn over.

Source: computed from the Airy zeros for this thesis by `make_airy_zeros.py`, after the presentation used in H. Abele's lecture course; it replaces a slide export.

vibrating the mirror. A drive at angular frequency ω couples two states when it matches their energy splitting,

$$\Delta E_{pq} \;=\; E_q - E_p \;=\; \hbar\,\omega_{pq}, \tag{2.3}$$

which is what turns the experiment into a spectroscopy: the quantity actually measured is a mechanical frequency, in hertz, and Eq. (2.3) converts it into an energy difference. For the lowest pair, $\Delta E_{12} = 1.0528\,\mathrm{peV}$ corresponds to $\omega_{12}/2\pi = 254.6\,\mathrm{Hz}$. The number of vibrating regions the neutron crosses determines how sharply that frequency can be measured.

With a single oscillating region the experiment is a Rabi measurement: the neutron interacts with the drive once, and the width of the resonance is set by how long it spends in that one region. Going to three regions — drive, free propagation, drive again — turns it into a Ramsey measurement, and the resolution is no longer limited by the interaction time but by the much longer *free* time between the two drives.

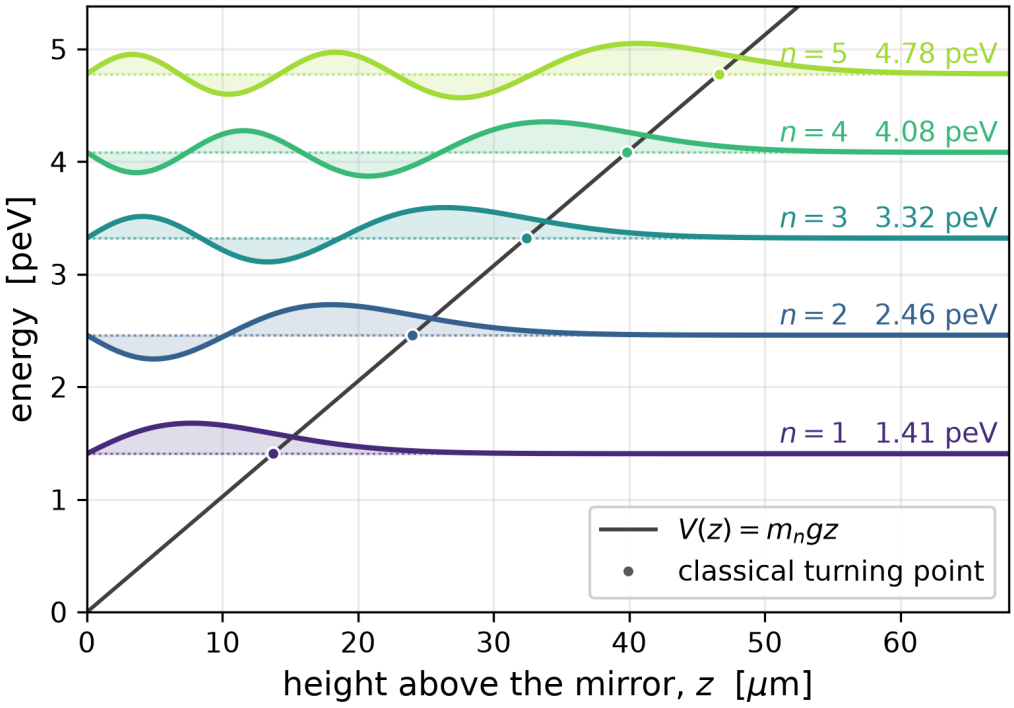


Figure 2.3: The first five gravitational states, computed from Eq. (2.2) and drawn at their own energies against the linear potential $V(z) = m_{\mathrm{n}} g z$. Each state is Ai shifted along its argument until one of its zeros falls on the mirror at $z = 0$, which is what imposes $\psi(0) = 0$; the n-th zero gives the n-th state, so the whole spectrum is fixed by where the zeros of a single function lie, and each state carries one more node than the last. The marker is the classical turning point, where the diagonal crosses that energy and $E_n = m_{\mathrm{n}} g z$. A classical ball of that energy would stop there and fall back. The neutron does not: beyond the marker it is in the classically forbidden region, and the wavefunction stops oscillating and decays exponentially instead of ending. Each state reaches higher before its turning point than the one below it, which is why the states extend over tens of micrometres — $z_0 = 5.87\,\mu\mathrm{m}$ sets that scale, and the ground state sits at $1.41\,\mathrm{peV}$. Amplitudes are rescaled for legibility and carry no physical meaning.

Source: computed from the Airy zeros for this thesis by `make_grav_states.py`. After the presentation used in H. Abele's lecture course.

The clearest way to see why is an analogy with Young's double slit, with one substitution: here the interference is not spatial but **temporal**. In the optical experiment a wave is split between two paths separated in *space* and the pattern records their phase difference. In the Ramsey configuration the neutron's state is split by the first region, propagates freely, and is recombined by the second, so the two contributions are separated in *time*; the pattern records the phase accumulated during that free interval. Lengthening the free propagation sharpens the fringes exactly as widening the slit separation does in the spatial case, which is what buys the resolution. Figures 2.4 and 2.5 show the two configurations, and Figure 2.6 the analogy itself.

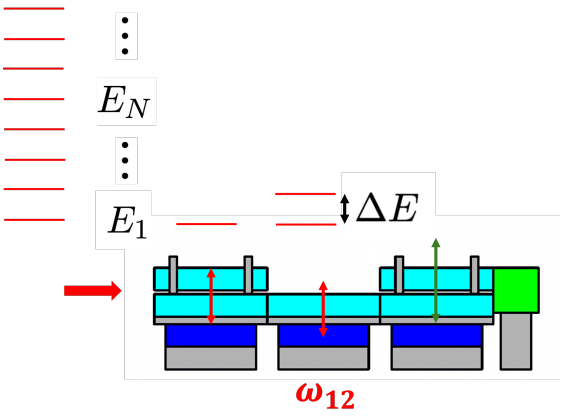


Figure 2.4: The one-zone, Rabi configuration. The neutron crosses a single oscillating region, driven at ω_{12}, so the width of the resonance is set by the time it spends in that one region. The level scheme on the left shows the gravitational states E_1 to E_N and the splitting ΔE the drive has to match.

Source: drawn for this thesis, after the scheme of [10].

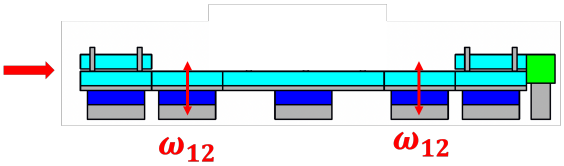


Figure 2.5: The Ramsey configuration. The same mirror lane now carries *two* regions driven at ω_{12}, separated by a stationary one. The resolution is set by the long free time between the two drives rather than by the time spent in either, which is what buys the precision.

Source: drawn for this thesis, after the scheme of [10].

2.4. The Experimental Setup

2.4.1. Overview and philosophy

The *qBounce* experiment is installed at the PF2 ultracold neutron installation of the Institut Laue-Langevin (ILL) in Grenoble, France [2, 10, 15]. PF2 provides the world's most intense steady-mode source of ultra-cold neutrons, with horizontal velocities selectable in the range

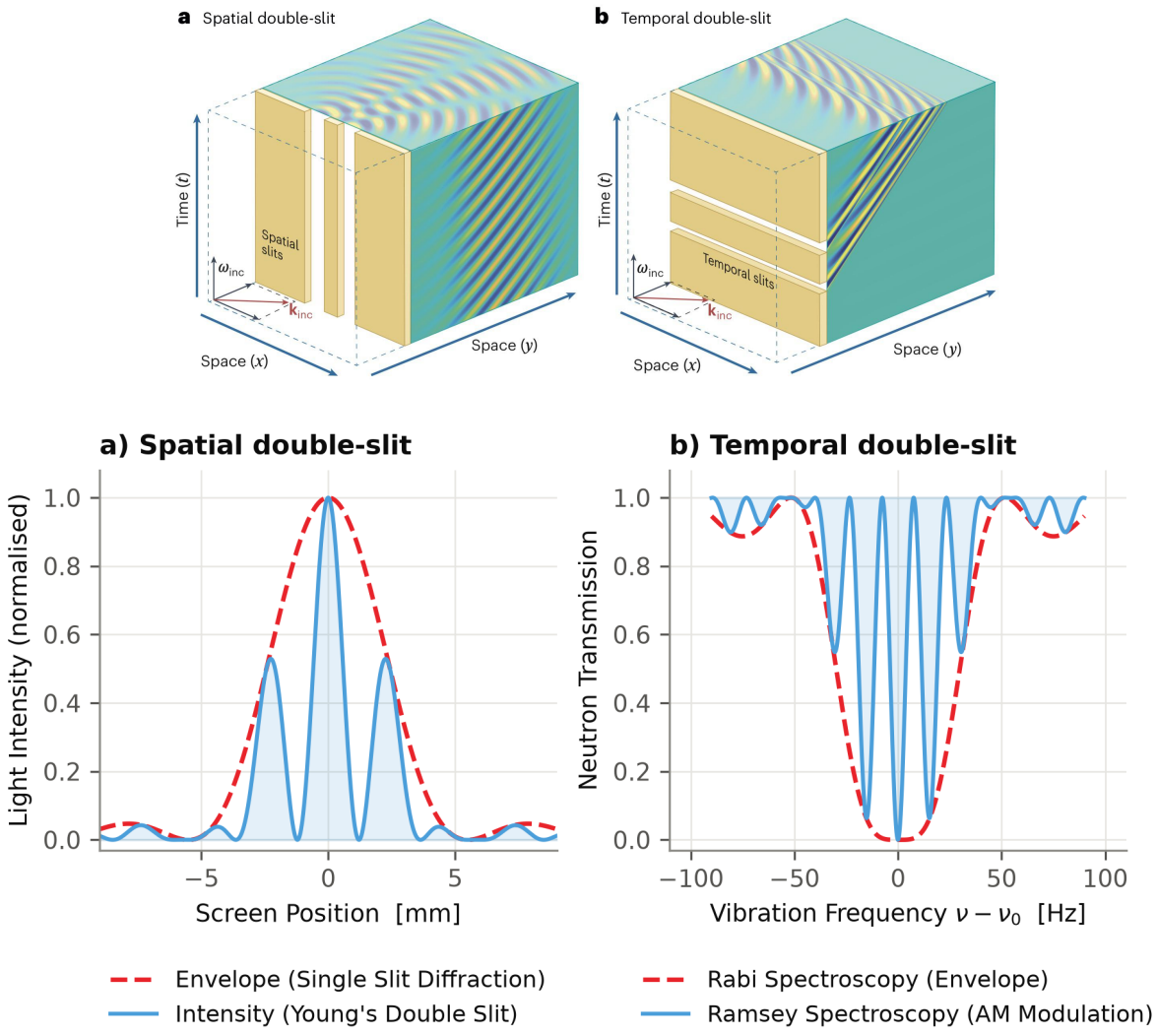


Figure 2.6: The same interference, in space and in time. Above: an incident wave split by two slits separated in *space*, and by two regions separated in *time*. Below, the lineshape each produces, drawn to the same convention — panel (b) plots the transmission the counter actually measures, so a transition shows as a loss. In both, a broad envelope is set by one aperture and fast fringes by the separation between two: in space the slit width and the slit spacing, in time the duration of one oscillating region and the free flight between two, drawn here at the same ratio so that the two panels carry the same structure. The scales are representative and are not measurements of this apparatus. The Ramsey fringes stand to their envelope exactly as the double-slit fringes stand to the single-slit envelope, and that is why two separated regions resolve a transition one region only locates.

Source: above, Figure 1 of [14]; below, `make_ramsey_analogy.py`, from the exact two-level separated-oscillatory-fields expression, the envelope obtained by maximising it over the free-flight phase.

5–13 m/s; the window used for gravity-resonance spectroscopy is the lower part of it, $v_x \approx$ 6–9 m/s [10].

The central idea is to create a situation where a neutron travels horizontally through a corridor of precisely aligned mirrors, spending enough time in each region to undergo well-defined quantum mechanical manipulations, and then to count the neutrons that survive the sequence. With the counter at the end, the signal is a *transmission* rate rather than a direct measurement of the wavefunction. That is a property of the detector, not of the method: a position-sensitive detector in one of these regions would return the vertical density itself, and The outlook of Chapter 5 takes that up, under *a sensor where the absorber is*.

The entire apparatus is mounted on a polished granite block (surface flatness $< 2\,\mu$m over 1900 mm $\times$ 700 mm) placed on an active and passive anti-vibration table. The granite is levelled to better than 1 µrad using three piezo motors and a two-axis tilt sensor in a closed feedback loop [10]. Everything is enclosed in a vacuum chamber ($p \approx 5 \times 10^{-5}$ mbar) to eliminate UCN absorption by air. Residual magnetic fields are suppressed by a μ-metal shield, preventing stray coupling to the neutron magnetic moment. Figure 2.7 shows the assembly.

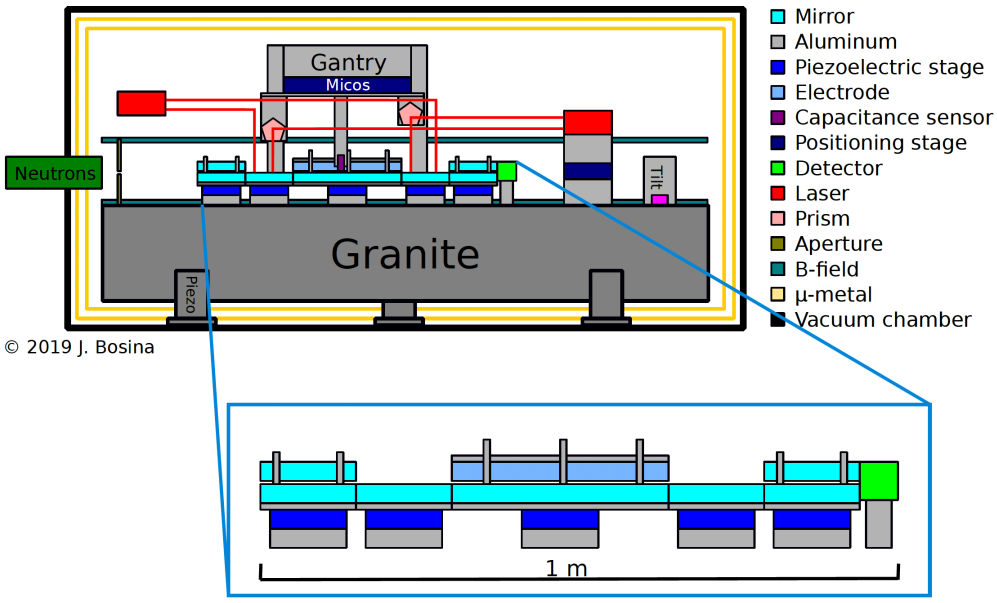


Figure 2.7: The Ramsey-type GRS setup (RamseyTR) installed at PF2, ILL Grenoble. UCNs enter from the left and travel horizontally through five mirror regions mounted on a granite block. A gantry above the mirror lane carries capacitive sensors (for alignment) and laser interferometers (for oscillation monitoring). The central region III acts as an electrode for electric-field measurements. The entire system is enclosed in a vacuum chamber surrounded by μ-metal magnetic shielding.

Source: © 2019 J. Bosina; the credit is carried on the figure itself. It appears as Figure 3 of [10].

2.4.2. The five-region Ramsey setup

A detailed account of Ramsey spectroscopy of gravitationally bound quantum states of ultracold neutrons in this apparatus is given in [16].

The Ramsey spectrometer consists of five physically distinct regions through which neutrons travel sequentially from left to right. Each region consists of a piezoelectric stage, a coarse-alignment motor, and a borosilicate glass mirror. What makes an ultracold neutron the right probe is the ratio of two energies. The mirror presents a Fermi potential barrier of $V_{\mathrm{F}} \approx 100\,\mathrm{neV}$, and the gravitational states have energies of a few peV: five orders of magnitude below it. The barrier is therefore a hard wall in the exact sense the textbook problem assumes, and the level structure is set by gravity alone and not by any detail of the surface. The comparison is with the *vertical* energy alone, which is the only component the mirror sees: a neutron crossing the spectrometer at 8 m/s carries 335 neV in total and still bounces, because its vertical motion stays below the 4.4 m/s that would clear the barrier. All mirror surfaces are ground to sub-nanometre roughness. The step heights between adjacent regions are controlled to $< 0.5\,\mu\mathrm{m}$ by capacitive sensors that directly measure mirror surfaces. Figure 2.8 names the five and what each does.

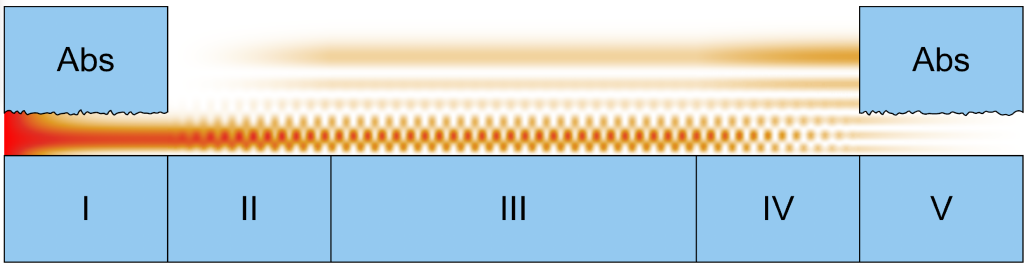


Figure 2.8: Schematic of the five-region Ramsey spectrometer, with the neutron wavefunction propagating above it. UCNs enter from the left. The five mirror regions are the numbered blocks along the bottom: I and V carry rough-mirror absorbers, marked Abs, that transmit only the lowest gravitational states; II and IV are oscillating mirrors applying $\pi/2$-pulses; and III is a stationary mirror where the neutron propagates freely in a coherent superposition of $|p\rangle$ and $|q\rangle$, and which can host an electrode or a magnetic coil. The colour above the mirrors is the probability density, red where it is highest at the entrance and fanning out into the interference pattern that region III produces. The detector sits beyond the right-hand edge of the drawing and is not shown.

Source: from [15].

Region I — State Selector (Polariser). A rough glass plate is clamped $\ell \approx 25\,\mu\mathrm{m}$ above the polished mirror. States with $E_n > m_{\mathrm{n}} g\,\ell$ have a turning point above the rough surface; their wavefunction overlaps with the absorber and they are scattered out of the system within milliseconds. Only the lowest-energy states (principally $|1\rangle$ and $|2\rangle$, with a small admixture of $|3\rangle$) survive. After region I, the state population is approximately $(P_1, P_2, P_3) \approx (0.61, 0.22, 0.04)$ [15]. This is a *scattering absorber*: it does not measure anything, it removes what it touches, and what leaves region I is defined by what did not survive it.

The two velocity components play quite different parts, and it is worth separating them. Only the component *perpendicular* to the mirror decides whether a neutron passes: it is what sets the vertical energy, and therefore which state a neutron occupies and whether its turning point reaches the rough plate. The *horizontal* component decides nothing about selection and everything about time. Neutrons enter with a spread of horizontal velocities, so they arrive at a range of angles, and a faster neutron crosses the spectrometer sooner. Since the whole

measurement is made during that flight, the slow ones are the useful ones: the time available to drive a transition is L/v_x, and the resonance a Ramsey sequence can resolve narrows in proportion. That is why the beam is selected toward small angles rather than simply toward high flux.

Region II — First Oscillating Mirror ($\pi/2$-flip). The bottom mirror is driven by a piezo actuator at a tunable frequency ν and amplitude $a\omega$ (vibration velocity). If $\nu \approx \nu_{pq} = (E_q - E_p)/h$, the oscillating mirror couples states $|p\rangle$ and $|q\rangle$ via the matrix element $V_{pq} = \langle q|\partial_z|p\rangle$. The length of region II and the oscillation amplitude are tuned to apply a $\pi/2$-pulse: a neutron entering in $|p\rangle$ exits in an equal superposition $(|p\rangle + |q\rangle)/\sqrt{2}$. The surface position is monitored in real time by a laser interferometer attached to a gantry above the mirrors.

Region III — Free Propagation. No perturbation is applied. The neutron propagates freely for a time $T = L_{\mathrm{III}}/v_x$, where $L_{\mathrm{III}} = 340\,\mathrm{mm}$ and v_x is the horizontal velocity. During this time, the relative phase between $|p\rangle$ and $|q\rangle$ accumulates as:

$$\phi_{pq} = \frac{(E_q - E_p)}{\hbar}\, T = 2\pi\, \nu_{pq}\, T. \tag{2.4}$$

The sensitivity of the final transmission to the transition frequency scales as $\sim T$: a longer free-propagation region gives sharper Ramsey fringes. This region may also contain an electrode (parallel-plate capacitor) for measurements of the neutron electric charge [10], or a coil for magnetic-field-dependent measurements.

Region IV — Second Oscillating Mirror ($\pi/2$-flip). Identical to region II in frequency and amplitude, but with a controlled phase offset $\Delta\phi = \phi_{\mathrm{IV}} - \phi_{\mathrm{II}}$. If $\Delta\phi = 0$ (in-phase), the second pulse completes the excitation, driving the neutron preferentially into $|q\rangle$. If $\Delta\phi = \pi$ (anti-phase), the second pulse reverses the excitation and the neutron returns to $|p\rangle$. The interference between the two amplitudes produces Ramsey fringes: the transmission oscillates as a function of the detuning $\delta\nu = \nu - \nu_{pq}$.

Region V — State Analyser. Identical to region I: the rough-mirror absorber removes all states with $E_n > m_{\mathrm{n}} g\, \ell$. Since the excited state $|q\rangle$ has higher energy than $|p\rangle$, it has a larger spatial extent and is preferentially absorbed. The neutron count rate at the detector therefore *drops* when a transition $|p\rangle \to |q\rangle$ has occurred.

Detector.

The detector that closes the sequence, and that this thesis is about, is a CMOS image sensor with a ^{10}B converter layer deposited on it. A neutron that survives region V is captured in the boron, and one of the two charged daughters of that capture — a lithium ion or an α particle, emitted back to back so that only one can reach the silicon — ionises the sensor along a path of a few micrometres. The charge collected by the pixels it crossed is read out with the frame, as a compact cluster of order ten to twenty pixels (Figure 2.9).

What the sensor actually records.

A neutron is never seen directly. It is captured by the boron-10 converter layer — the same conversion principle as the UCN detectors described in [5] — and what the CMOS records is one of the two charged daughters:

$$\mathrm{n} + {}^{10}\mathrm{B} \longrightarrow {}^{7}\mathrm{Li}^{*}\,(0.84\,\mathrm{MeV}) + \alpha\,(1.47\,\mathrm{MeV}) \qquad (94\,\%) \tag{2.5}$$

$$\mathrm{n} + {}^{10}\mathrm{B} \longrightarrow {}^{7}\mathrm{Li}\,(1.01\,\mathrm{MeV}) + \alpha\,(1.78\,\mathrm{MeV}) \qquad (6\,\%) \tag{2.6}$$

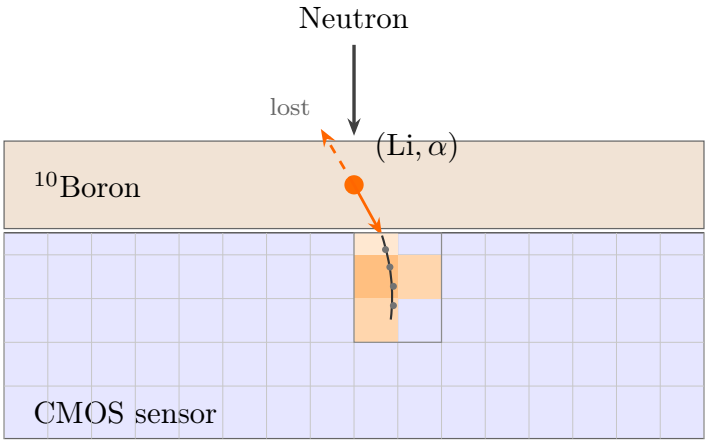


Figure 2.9: How a neutron becomes a signal in the CMOS detector. The neutron is captured in the ^{10}B converter layer; the reaction sends a lithium ion and an α particle back to back, so at most one of the two reaches the sensor. The one that does ionises the silicon along its path, and the charge collected by the pixels it crossed is read out with the frame as a single cluster. Drawn as the direct counterpart of Figure 8.3 of [6], where the same capture leaves structural defects in a CR39 polymer that are etched for five hours and then scanned under a microscope — same conversion, entirely different readout.

Source: drawn for this thesis, `docs/detector_figure.tex`.

The two daughters leave back to back, so only one of them can reach the sensor from a given capture. A working energy axis would therefore show four lines — two strong and two weak satellites about a fifth higher in energy — and separating the lithium population from the alpha population is what turns a cluster count into a physically labelled neutron count.

Why a CMOS sensor, and not the alternatives.

Three detector technologies are in use for this kind of measurement, and they differ less in what they detect than in what they record about it.

A **proportional counter** is the reference instrument for the job: the spectrometer already uses one, a ^{10}B-coated tube filled with an argon–CO_2 mixture, to count transmitted neutrons [5]. Two materials do two different jobs there. The ^{10}B is a solid layer on the inner wall and it is what captures the neutron; the argon is the counting gas and captures nothing, its cross-section for neutrons being negligible. What the gas does is amplify: the charged daughter leaves the boron layer, ionises the argon along its path, and the avalanche in the field of the anode wire turns that ionisation into a pulse. The CO_2 absorbs the ultraviolet photons the avalanche emits, which would otherwise start further avalanches and destroy the proportionality the counter is named for. Its efficiency is high and well characterised, and it timestamps every event, so it answers *how many* and *when* with an accuracy nothing here matches. What it does not answer is *where*: the counter integrates over its whole volume, and a gravitational state is a structure in height, which is exactly the coordinate it throws away.

A **CR39** sheet answers *where* better than anything else. Each capture leaves a physical pit in the polymer, and after chemical etching those pits are read under a microscope at a resolution finer than any pixel grid. The price is time in both senses. The sheet is passive: every neutron of the whole exposure lands on the same surface with no record of when it arrived, so the

measurement is one image summed over the run, and a rate that changes during the exposure is indistinguishable from one that does not. And the sheet cannot be read while it is exposed. It has to come out, be etched — five hours in sodium hydroxide, in the procedure of [6] — and then be scanned under a microscope, so a full turnaround is days rather than minutes.

A **CMOS sensor** gives both coordinates at once. Every frame is a time slice, and within it every cluster carries the pixel positions it covered, so a track records where the neutron landed and when. The spatial resolution is set by the pixel pitch and is coarser than CR39. In exchange the run can be watched as it happens, which is what makes the live path of Section 3.7 possible and what a beam test needs when something goes wrong at three in the morning. The two-stage arrangement of the proportional counter is the one the CMOS sensor inherits, and it inherits only the first stage: the same ^{10}B layer converts the neutron, and silicon replaces the gas as the medium that collects what comes out of it. The two instruments therefore see the same reaction and differ only in how they read it. Section 4.8 compares this sensor against the CR39 pipeline on the numbers.

The sensor this thesis works with. The camera is an industrial machine vision unit driven through the IDS peak SDK, built around the Sony IMX183 back-illuminated CMOS sensor — identified from its format, since 5472×3672 pixels on a 2.4 µm pitch is that sensor's. Its frames are 5472×3672 pixels, 20.1 Mpixel, read out at 8 bits in these acquisitions — a choice of format, not a limit of the sensor, and Section 4.5 returns to what it cost — and the camera sustains 1.8 fps at the 575.6 ms exposure these acquisitions used. At a pixel pitch of 2.4 µm the active area is 13.1 mm by 8.8 mm, which is a little larger than the beam footprint the arrival maps of Section 4.3 show.

That pitch is the number the rest of this argument turns on. The gravitational states of Section 2.2 are structures tens of micrometres tall: the first two classical turning points sit at 13.7 µm and 24.0 µm, so the closest pair of states is separated by 10.3 µm, or about four pixels. A sensor of this pitch can therefore resolve the level structure in principle, and the outlook of Chapter 5 takes that further. What it cannot do at this gain is resolve the *energy* of a single capture, and Section 3.5 says why.

The pitch is not the resolution, though, and the resolution of this instrument has not been characterised. What exists is an estimate read off the signal itself: a track covering about twelve pixels has an RMS width of 1.86 µm, which places its centroid to some 0.5 µm, a fifth of the 2.4 µm pitch, because a wide track is many samples of one position. Nothing in this thesis tests that number against a known geometry. Doing so takes a dedicated acquisition, a mask, a sharp edge or a collimated source imaged on the sensor, and none was recorded during the beam time. The estimate is what the outlook works with, and it is what a measurement of the spatial resolution would replace.

Turning a transmission measurement into an image has a price, and it is paid once per cluster: something has to decide, for every bright object in every frame, whether it is a particle or the sensor's own noise. Section 3 is that decision.

What the sensor does when nothing hits it. A CMOS sensor is a grid of photodiodes, each with its own amplifier, and each one produces a signal whether or not anything arrives. Three effects contribute. *Dark current* is the thermal generation of charge in the depleted silicon, which accumulates over the exposure and roughly doubles for every 6 to 8 K. On this sensor at a 575.6 ms exposure it stays below the first digitisation step over most of the array — Section 4.1 measures 97.9 % of pixels reading exactly zero — but it is the term that grows with exposure and with temperature, so it is the one that sets how long a frame can be. *Read noise* is added by the

amplifier and the analogue-to-digital conversion, and is independent of exposure. *Fixed-pattern noise* is the part that matters most here, because the offset and the gain differ from one pixel to the next by amounts fixed at manufacture. A single threshold applied to the whole sensor therefore cuts at a different physical level on every pixel, which is why the detector of Section 3 learns a mean and a variance *per pixel* rather than one pair for the frame.

Two failure modes sit on top of that noise. A *hot pixel* has an anomalously large dark current, usually from a lattice defect that creates a mid-gap trap state, and it saturates faster than its neighbours; at a fixed threshold it fires on frame after frame and mimics a permanent particle. A *dead pixel* does the opposite and never responds. Some of both are present on any sensor as delivered. What matters for a neutron experiment is that their number is not fixed: fast neutrons and the recoiling daughters of the capture reaction displace atoms in the silicon lattice, each displacement adds a trap, and a pixel that was merely noisy becomes hot. That is a slow degradation of the instrument under its own signal, and Section 4.7 measures it.

2.5. Precision Metrology and Expected Results

What makes this measurement demanding is the scale of the energies involved. The transitions being driven separate states of order 1–5 peV, and the frequencies are measured to about 0.05 Hz out of ∼900 Hz, a relative precision near 10^{-5}.

That number is worth reading correctly, because it is easy to attribute to the wrong cause. It is a *statistical* limit. The width of the resonance is set by the free flight time, as Section 2.3 describes, and does not improve with counting; what improves is how well its centre can be placed, and that goes only as the square root of the neutrons counted. The apparatus is not being held back by an uncontrolled effect that a better model would remove; it is being held back by how many neutrons the beam delivers in a shift. Systematic effects at this level are real but small — the residual magnetic field inside the μ-metal shield, the spread of the velocity spectrum — and they enter the frequency well below the statistical width. They would have to be reconsidered before the precision improves by an order of magnitude; they do not set it today.

Which is precisely where a position-sensitive detector earns its place. A counter returns one number per configuration and can only be used to measure a systematic effect by changing something and counting again, which costs a shift each time. A pixel sensor returns a distribution. The divergence of the beam, for instance, could be measured directly by moving the sensor around the spot and reading the profile, rather than inferred from a series of transmissions. The value in what follows is therefore not only counting faster: it is that counting with position makes some systematic tests cheap enough to be worth doing at all.

The reward is that a frequency measured this well constrains physics beyond the Standard Model. A short-range fifth force is conventionally written as a Yukawa correction to the Newtonian potential between two masses,

$$V(r) \;=\; -\,\frac{G\,m_1 m_2}{r}\left[\,1+\alpha\,e^{-r/\lambda}\,\right], \tag{2.7}$$

where α is the strength of the new interaction relative to gravity and λ its range. Gravitational quantum states probe the plane spanned by these two parameters directly, because the state energies of Eq. (2.2) shift if the potential departs from $m_{\mathrm{n}}gz$ over the tens of micrometres the neutron explores. Dark-energy and dark-matter scenarios are constrained the same way [17]. Chameleon fields strongly coupled to the neutronic quantum bouncer are treated in [18], and

numerical methods for scalar-field dark energy in table-top experiments in [19]. Figure 2.10 shows where those bounds currently stand.

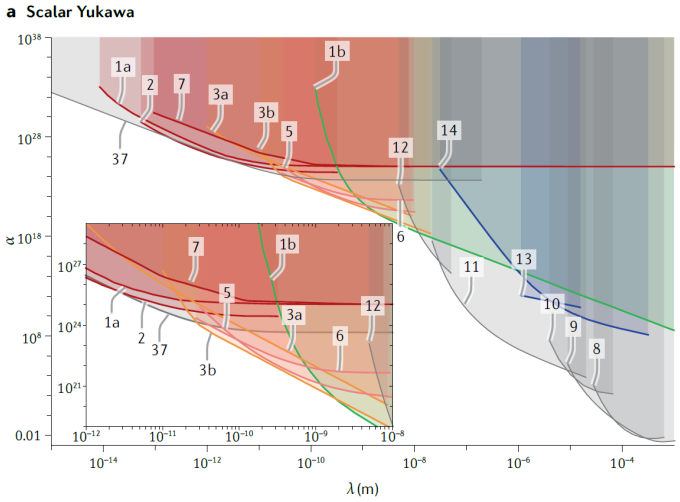


Figure 2.10: Existing bounds on a scalar Yukawa interaction, in the coupling strength α against the range λ. Each shaded region is excluded by one class of experiment, and the inset magnifies the four decades from 10^{-12} to $10^{-8}\,\mathrm{m}$, where the short-range scattering limits crowd together. Gravity resonance spectroscopy with ultracold neutrons works much further to the right: its states extend over tens of micrometres (Section 2.2), so what it constrains is the region around $\lambda \sim 10^{-5}\,\mathrm{m}$. A frequency measured to $0.05\,\mathrm{Hz}$ buys a further bite out of that part of the plane.

Source: panel (a) of Figure 6 of [20].

2.6. Why the Data Must Be Measured and Analysed

The precision targets above are what make the detection software necessary.

The scale of the problem follows from three numbers, and rounding them is enough to see it. Take a beam delivering of order $100/(\mathrm{s\,cm^2})$ onto a sensor of about a square centimetre, read out at roughly two frames per second: that is some fifty captures in every frame. Each is a cluster of a handful of pixels somewhere on twenty million, and a run of a couple of thousand frames is tens of gigapixels and something like a hundred thousand clusters. Nobody inspects that by hand, and the exact figures do not change the conclusion — they are measured in Section 4.

Volume is not the only difficulty. The sensor produces clusters of its own even in the dark, at a rate of the same order as the neutron rate itself, and higher still in the first frames of a run while the background model is still learning. So the noise floor has to be measured and subtracted before any rate means anything. The events that survive then have to be told apart, because a neutron capture, an ambient gamma and a noise excursion of the sensor all appear as bright clusters, and only their shape, size and deposited charge distinguish them.

That is the gap Section 3 fills: a background model built from dark frames, a cluster detector that works against it, and a classifier trained on human-labelled examples, fast enough to run while the beam is on so that a problem shows up during the run.

3. Detection Software Pipeline

This section describes how the pipeline is *built*, from raw image processing in C++ to cluster classification in Python. How it is *operated* — which button to press, in which order, and what each field means — is the subject of the companion user guide [21], which ships with the offline lab kit and is the document an operator has open at the beamline. Section 3.7 summarises the two paths through it.

3.1. Data Flow

Section 3.2 answers the question *which file invokes which*. This section answers the other one: *what the data is at each point, and what it becomes*. The two are kept apart on purpose, because they do not line up — one file appears at several stages of the flow below, and one stage is often served by more than one file.

A run is a chain of eight stages, shown in Figure 3.1. Each discards what the next does not need, which is how a campaign that starts as tens of gigabytes of images ends in a text file and a handful of figures.

1. **Dark frames.** 500 beam-off frames of 5472×3672 8-bit pixels, acquired with the same exposure as the measurement they will be subtracted from.

2. **Background model.** The frames are folded, one at a time, into three per-pixel accumulators — the running mean, the running sum of squared deviations from it that Welford's recurrence carries as M_2, and the sample count — plus a mask of dead and hot pixels. They are never held in memory together and never revisited. What survives the stage is a single 261 MB binary model.

3. **Threshold.** Each beam-on frame is compared against that model pixel by pixel, and the stage emits one binary mask per frame, set where the pixel exceeds $\mu + n_\sigma \max(\sigma, 1\,\mathrm{ADU})$, where one ADU is one analogue-to-digital unit, the least significant bit of the sensor's digitiser. This is the only point in the whole pipeline where a decision is taken about a single isolated pixel.

4. **Clusters.** Connected components turn scattered mask pixels into objects. From here on the pipeline no longer handles images at all.

5. **Measurements.** Each cluster is characterised into the twenty columns of `Cluster::csvHeader()`, which is the single source of truth for every consumer downstream — offline and live alike, which is why a cluster counted at the beamline and a cluster counted three weeks later from archived frames are directly comparable.

6. **Hand labels.** A human verdict is attached to a sample of the clusters, in the `true_label` column, through the labelling interface of Figure 3.8. This is the one stage that cannot be automated, and everything the classifier later claims rests on it.

7. **Classifier.** A random forest is fitted on the labelled sample, exported to ONNX, and applied to every cluster of the full set.

8. **Analysis.** The end products: a classified CSV, the rates, arrival map and deposited-charge distributions of Section 4, and a report scored against the reference set.

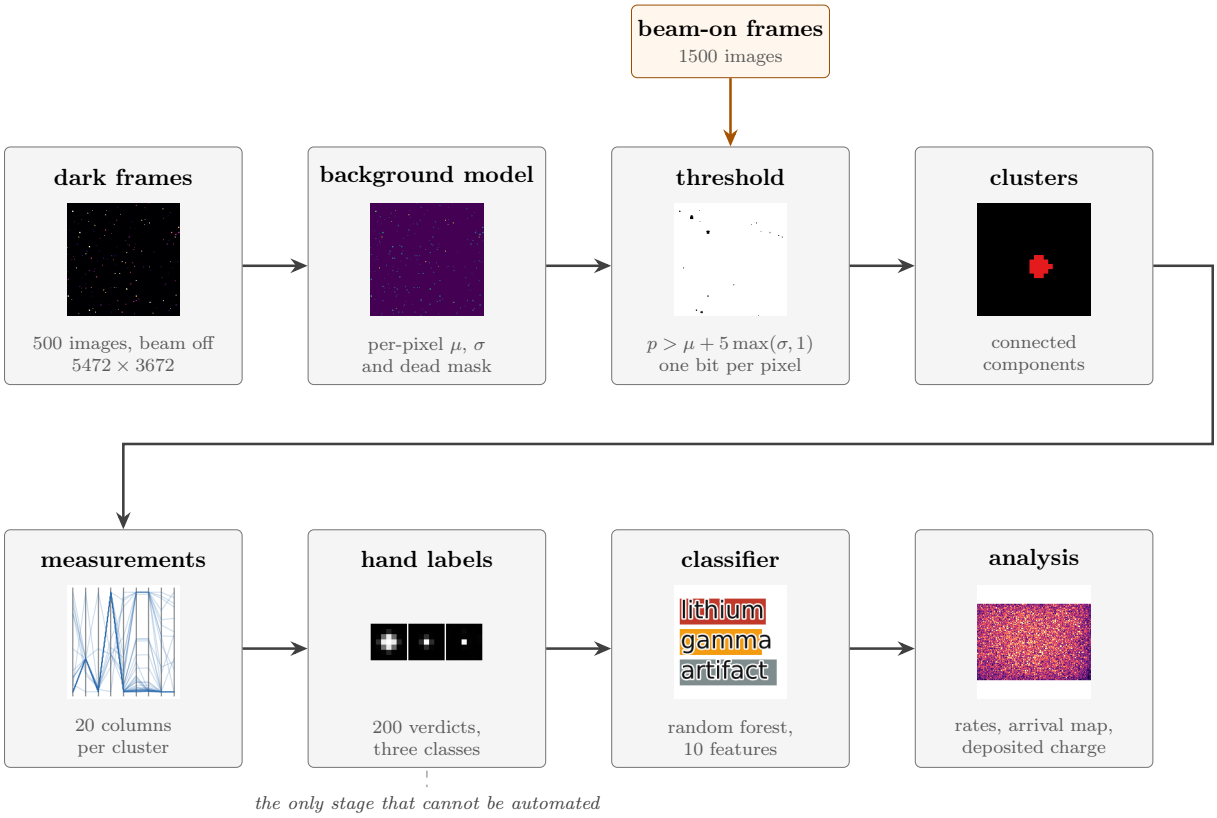


Figure 3.1: The processing chain, stage by stage: what the data is at each point and what it becomes. Every thumbnail is made from the data itself rather than drawn as an icon. Stages 1 to 3 all show the same 128×128 corner of the same frames — the dark frame, the per-pixel σ learnt from 500 of them, and the handful of pixels that survive the threshold on a beam-on frame — and stage 4 zooms to 24×24 around one 17-pixel track, because connected components are about what happens around a single cluster. The background model is initialised on beam-off frames, and it keeps updating on every frame after that, beam-on frames included: `bg.update()` runs once per frame for the whole acquisition. What protects the statistics is not that beam-on frames are held out, but that the pixels which cross the threshold on a frame are excluded from that frame's update, as Section 3.4.1 shows in the source. The beam-on frames are drawn as a side input because that is where they are compared, not because the model stops learning from them. Sections 4.6 and 4.7 both rest on the continued updating. Drawn after Figure 6.13 of [6], which lays out the simulation chain of that thesis the same way. Figure 3.2 answers the other question — which file invokes which — and the two are deliberately kept apart.

Source: drawn for this thesis, `docs/dataflow_figure.tex`; the thumbnails by `make_flow_thumbnails.py`.

Two paths run through this chain, and they differ only in stages 1 to 4. Offline, the detector reads a folder of files, processes them through a three-stage thread pipeline and writes one CSV at the end. Live, the same binary is instead held open as a persistent server and receives frames over a pipe as they leave the camera, streaming the same CSV lines back as it goes; keeping the background model in memory between frames is what makes that path possible at all, since rebuilding it per frame is not. Stages 5 to 8 are shared verbatim between the two.

3.2. Architecture

The twenty-one source files fall into four *poles*, each with one responsibility, plus the scripts that deploy and drive them. Figure 3.2 shows how a run reaches each pole: `PrepareSetupLab.py` builds the offline kit on a machine that has internet, `WindowsLauncher.py` unpacks and compiles it on the lab PC that does not, `LabValidation.py` checks that result without a human present, and `PipelineController.py` — the Tkinter GUI — is the only process that ever launches any of the four poles. Figures 3.3 to 3.6 then open each pole in turn.

The split between the two languages is a division of labour rather than a preference: the C++ side carries everything that has to touch all 20.1 million pixels of every frame, and the Python side carries everything a human is in the loop for — labelling, training, and the offline studies. The boundary between them is the cluster CSV, and nothing crosses it in the other direction.

Every arrow carries the relationship it stands for, naming the function where a single call is what couples two files. The graphs are generated from `docs/architecture_graphs.tex`, which this thesis and the companion document `docs/CodeArchitecture.tex` both `\input`, so the two can never disagree about the code.

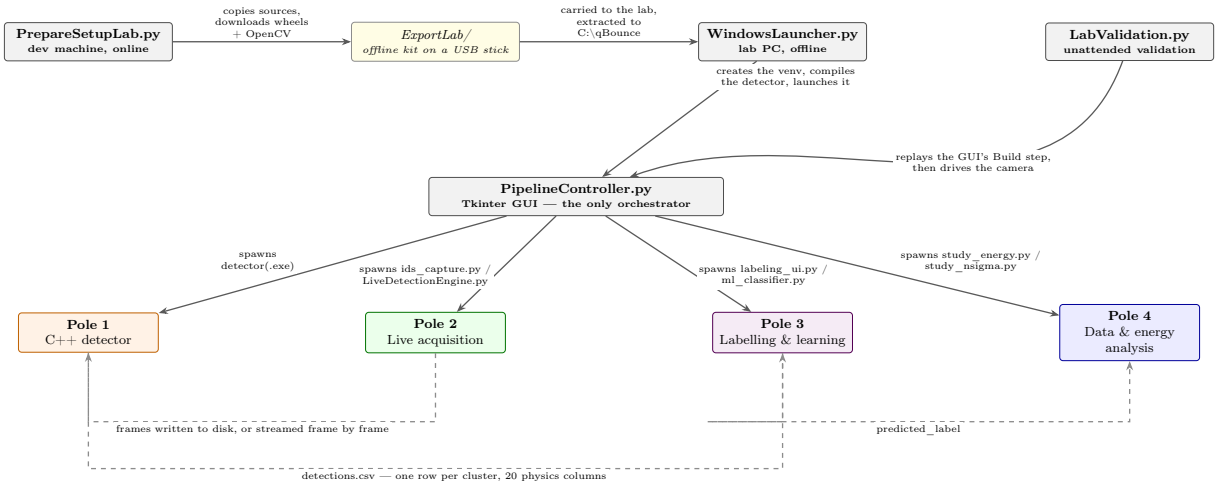


Figure 3.2: The four entry-point scripts, the GUI, and the four poles. Solid arrows are process launches; dashed arrows are the data each pole hands to the next.

Source: drawn for this thesis, `docs/architecture_graphs.tex`.

3.3. Detection Method

Section 3.1 traced what the data becomes; this section states what the detector actually decides, and on what grounds.

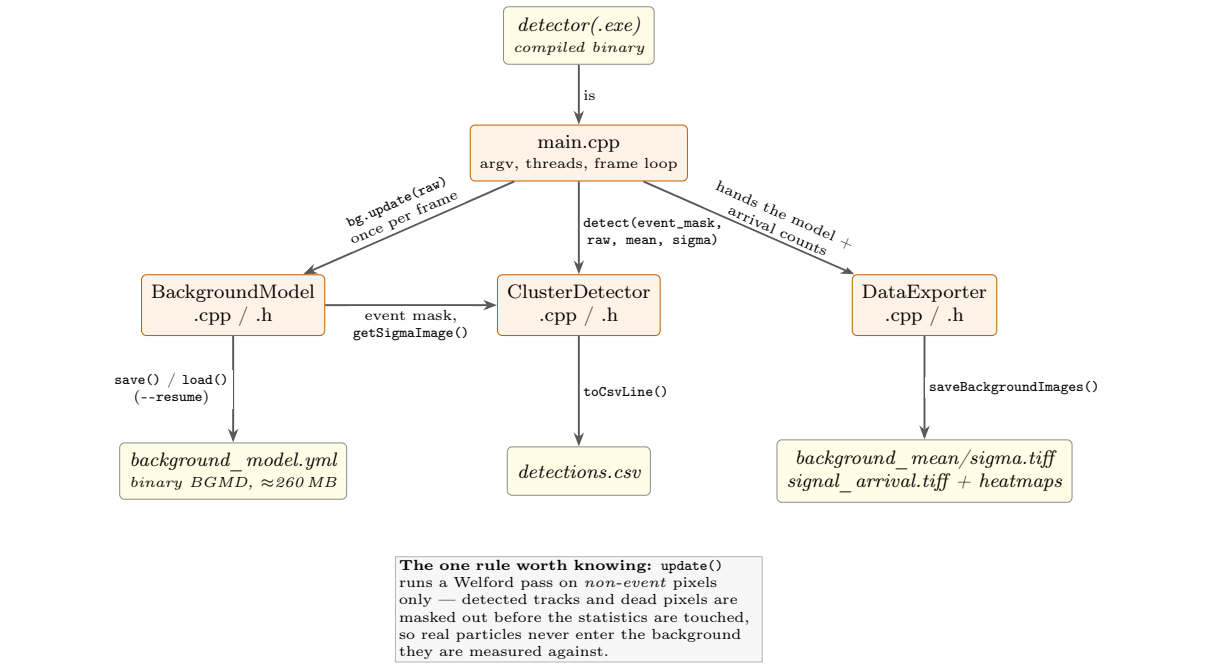


Figure 3.3: Pole 1 — the compiled C++ detector. `main.cpp` owns the frame loop; the three translation units below it each own one step of the measurement.

Source: drawn for this thesis from the source tree, `docs/architecture_graphs.tex`.

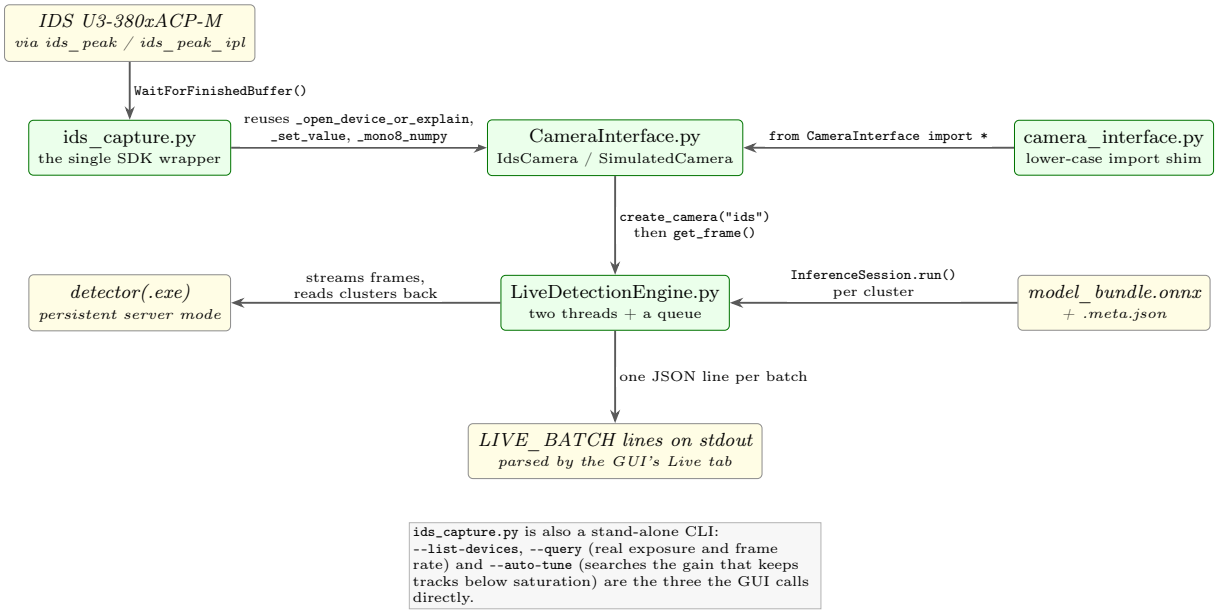


Figure 3.4: Pole 2 — live acquisition. `ids_capture.py` is the single place the IDS SDK is touched; everything above it reuses those helpers rather than re-implementing them.

Source: generated from `docs/architecture_graphs.tex`, which this thesis and the companion architecture document both input.

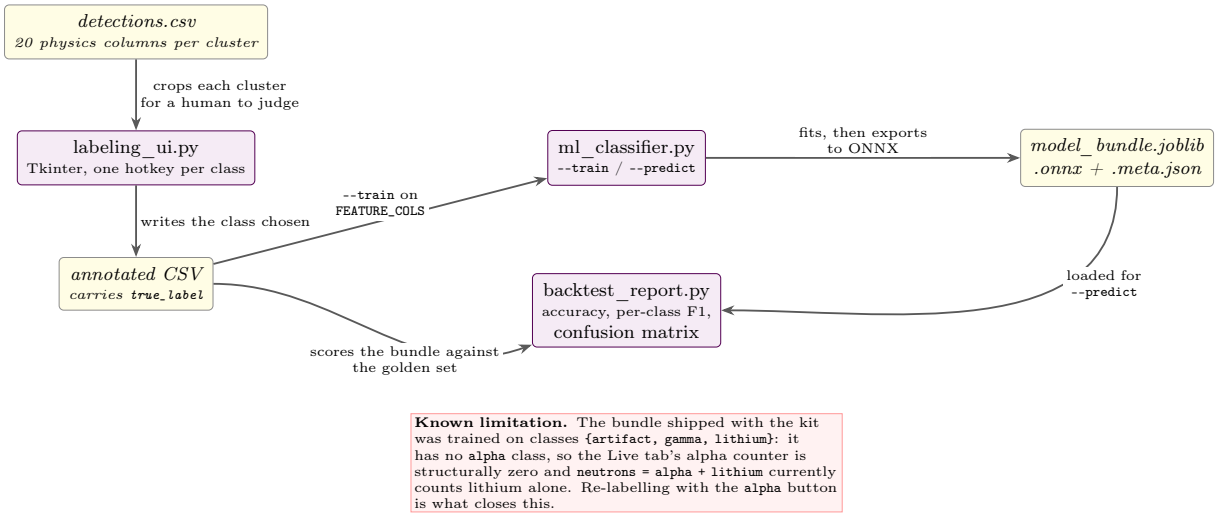


Figure 3.5: Pole 3 — labelling and machine learning, from a human verdict on a cluster crop to an ONNX model the live engine can run per cluster.

Source: generated from `docs/architecture_graphs.tex`, which this thesis and the companion architecture document both input.

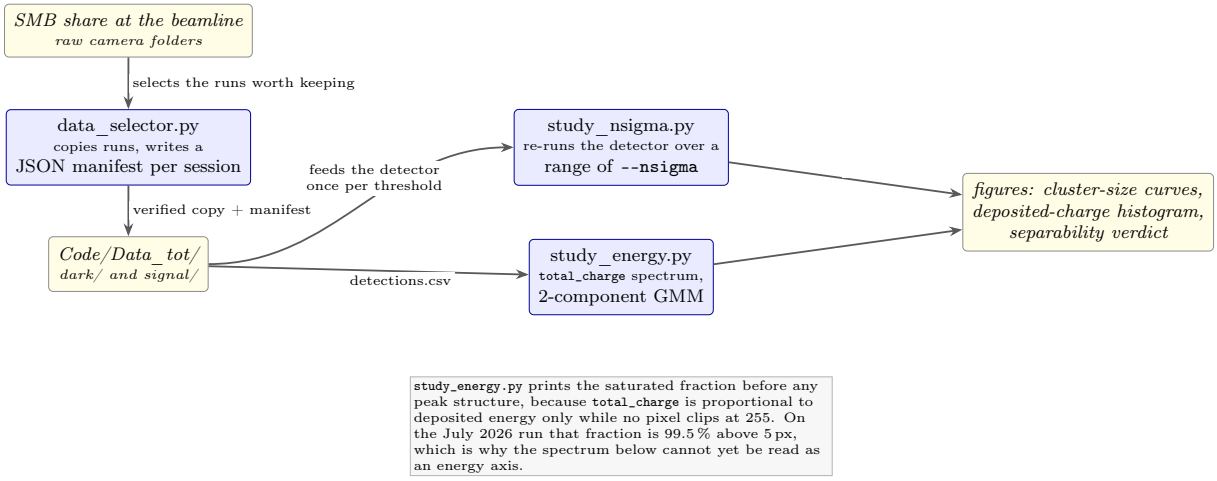


Figure 3.6: Pole 4 — data selection and offline analysis, the two studies that read the detector's CSV rather than the images.

Source: generated from `docs/architecture_graphs.tex`, which this thesis and the companion architecture document both input.

3.3.1. Where the threshold comes from

The detector never compares a pixel against a fixed value. It compares it against the distribution that pixel has shown in the dark. Each pixel carries its own running mean and variance, accumulated with Welford's recurrence so that neither the frames nor their squares have to be held in memory, and a pixel counts as an event on a given frame when

$$p_{ij} \;>\; \mu_{ij} \;+\; n_\sigma \,\max(\sigma_{ij},\, 1\,\mathrm{ADU}), \qquad n_\sigma = 5, \tag{3.1}$$

where μ_{ij} and σ_{ij} are that pixel's own mean and standard deviation. The floor of one ADU on σ is not cosmetic: almost every pixel of this sensor has a measured σ below one ADU, most of them exactly zero, so without the floor the criterion would degenerate and any excess whatsoever would register. Over almost the whole sensor the effective rule is therefore $\mu + 5\,\mathrm{ADU}$ — a fixed cut with a measured offset, which is a more honest description of the method than "adaptive 5σ".

Why five. The threshold was scanned rather than assumed. Running the detector over the same 500 dark frames at n_σ from 3 to 16, and over 500 beam-on frames for the cost side, gives Table 3.1. Two things change in opposite directions. Lowering the threshold finds more noise: 190 clusters per dark frame at 3σ against 78 at 5σ. Raising it loses tracks: the beam-on population above ten pixels falls from 35.9 per frame at 3σ to 27.0 at 16σ.

Between those two, one line does not vary smoothly. At 3σ and 4σ a single dark cluster reaches ten pixels, measuring 13 and 12 pixels; from 5σ upward none does and the largest drops to six. It is worth looking at that one object before drawing a conclusion from it, because the columns are in the CSV: it peaks at 23 ADU where a track saturates at 255, its elongation is 105 where a track sits near 1.2, and its compactness is 0.14 against 0.75. It is a long thin streak, and no cut on size was ever going to be what separated it from a capture.

So the threshold is not what protects the false-neutron floor, and that is the stronger statement. Nothing in a beam-off frame has the *shape* of a track at any threshold between 3σ and 16σ; five is chosen because it is where the largest beam-off object stops reaching into the track band, and above it the only thing bought is a smaller CSV.

What raising the threshold costs is less than the largest column suggests, because part of what leaves that column has not left the run. From 3σ to 16σ the beam-on population above ten pixels falls by a quarter while the population between five and nine pixels *grows* by a sixth. A higher threshold shaves the faint outer pixels off a track, so a track of eleven pixels becomes one of nine instead of disappearing. Counted over both bands, 5σ costs 3 % of the objects against 3σ and 16σ costs 13 %: migration accounts for a quarter of the apparent loss, and the remainder is real.

Both bands hold objects that are not tracks, so the statement is worth making on tracks alone. Saturation identifies them: no beam-off object reaches the clipping level in 500 frames (Section 4.4), so a saturated object is one the beam produced. Counting only those, the upper band loses 8.9 per frame between 3σ and 16σ while the lower band gains 2.6, and 29 % of the loss is migration rather than disappearance.

A track could also shed pixels by breaking into two objects, which would look the same in a histogram of sizes. It does not. Matching clusters between the two thresholds by frame and centroid over sixty frames, 2221 of the 2223 tracks of at least ten pixels at 3σ are still a single object within three pixels of where they were at 16σ, and exactly one is matched by two. The migration is tracks shrinking, not tracks splitting.

	beam off, 500 frames			beam on, per frame	
n_σ	clusters/frame	largest	≥ 10 px	≥ 10 px	5–9 px
3	189.7	13 px	1	36.0	14.2
4	97.4	12 px	1	35.1	14.2
5	**78.1**	**6 px**	**0**	**34.3**	**14.3**
6	66.4	4 px	0	33.5	14.4
8	52.3	4 px	0	32.0	14.8
12	33.2	3 px	0	29.2	15.7
16	19.6	3 px	0	27.0	16.5

Table 3.1: The threshold scan, at a warm-up of 500 frames. The beam-off columns are counted over the full 500 frames of the dark run and the beam-on columns over the 499 the signal scan carries — which is why the rate at 5σ reads 78.1 here and 79.5 in Section 4.3, where the warm-up is ten and the average is taken over the 499 frames that follow it. Between 4σ and 5σ the largest beam-off object falls from twelve pixels to six, and the count of beam-off objects that reach the size of a track falls from one to none. The last two columns move in opposite directions: raising the threshold trims tracks out of the upper band and into the lower one rather than deleting them.

Source: `study_nsigma.py` driving the C++ detector, ten values of n_σ on each dataset; the CSVs are in `Code/scan_linux/`.

Two consequences follow, and both are properties of the design rather than accidents. Detected pixels are excluded from the statistics they are measured against, so a track cannot raise the threshold at the place it landed (Section 3.4); and because the exclusion is itself a threshold, the faint edge of a track escapes it, and an excess survives into the measurements. Section 4.6 measures where that excess sits, and finds it on the track pixels themselves rather than around them.

3.3.2. From pixels to objects

The event mask is reduced to objects by connected-component labelling, and each component is then reduced to the twenty columns of `Cluster::csvHeader()`: two identifiers, the charge-weighted centroid, the bounding box, four shape descriptors (size, aspect ratio, elongation from the eigenvalues of the inertia tensor, compactness), three photometric ones (total charge above background, peak value, peak signal-to-noise), the two radial spreads, and the classifier's verdict with its confidence and the filter status.

A cheap size cut is available before characterisation rather than after, because the component area is already known at that point and characterisation is the expensive step; the beam-on data is dominated by exactly what such a cut would remove. Every measurement reported in this thesis nonetheless runs with it set to zero, so nothing is discarded before characterisation and every count is of every connected component the threshold found. The signal-to-noise cut sits elsewhere and for a reason Section 3.4 gives with the source in front of it.

3.3.3. From objects to classes

Ten of the twenty columns are passed to a random forest: the four shape descriptors, the three photometric ones, the two radial spreads, and the derived ratio σ_x/σ_y. The centroid is deliberately *not* among them. Position on the sensor carries no physics about what a cluster is, and including it would let the model learn where on the sensor neutrons happened to land during the training run — an association that would then be reported as a classification.

The trained model distinguishes three classes.

`lithium` a nuclear track: the charged daughter of a capture in the converter, a compact object of order 10 to 20 pixels. Lithium and α are not separated, and are not meant to be: they are the two products of the same reaction, so either one signals a neutron. The energy analysis of Section 3.5 is where the two would be told apart, and that is not yet possible.

`gamma` a small, faint event of a few pixels, ambient rather than from the beam. The name records what such an event is usually taken to be and not what was identified: nothing in this thesis distinguishes an ambient gamma from any other small deposit, and the class would be better read as *small ambient event*.

`artifact` an excursion of the sensor itself rather than a particle. Beam-off these are overwhelmingly single pixels; the hand-labelled examples the classifier learnt from are larger, from 3 to 15 pixels, so on a single-pixel object the model is extrapolating outside what it was shown.

Figure 3.7 shows what each of the three looks like on the data itself.

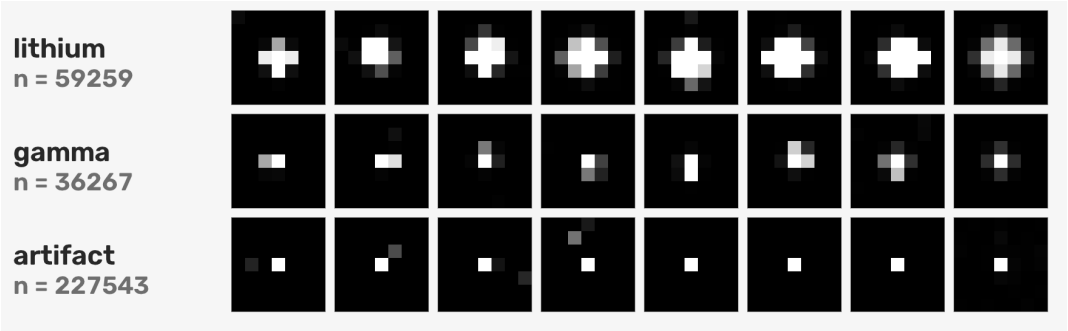


Figure 3.7: The three classes as the classifier assigns them on the 1500 beam-on frames. Each row shows seven members, one from each seventh of that class's size distribution, then the mean image of the class over 400. Every tile is the same 7×7-pixel window at the same magnification, so the black inside a tile is real sensor rather than padding and the rows compare directly. Compare Figure 8.14 of [6, p. 154], which does the same for CR39 with eleven classes.

Source: `make_class_plate.py`.

The mean images in the last column are very nearly round. Averaged over the 59 259 clusters called `lithium`, the second moments are $\sigma_x = 0.7891$ and $\sigma_y = 0.7686\,\mathrm{px}$: the mean track is 2.7 % wider than it is tall. With that many objects the asymmetry is far from chance, but it is not a property of the beam either, because the 36 267 `gamma` objects carry the same excess in

the same direction, 3.9 %, and a gamma has no reason to share an axis with the products of a neutron capture. What the two classes share is the sensor, so the residual anisotropy belongs to the pixel geometry or to the readout axis. Individual tracks are not round at all — half of them have an elongation above 1.2 — and the mean image is round because their orientations are distributed evenly, which is what an isotropic emission of the reaction products should produce.

The plate makes the separation visible: a lithium track covers a neighbourhood, a gamma a few pixels, an `artifact` one. It also makes the limitation visible — `gamma` and `artifact` differ by a pixel or two, which is where most of the classifier's error falls, and why it does not matter for a neutron count.

It also explains where the hand labelling is least reliable, and the plate is the honest place to say so. A `lithium` track in the labelled sample has a median of twelve pixels and a shape to judge; a `gamma` has five, and an `artifact` four. Below about half a dozen pixels there is very little for a human to go on: the object is a handful of bright squares, its elongation and compactness are quantised by the pixel grid rather than by anything physical, and two of them that differ by one pixel can be told apart only by guessing. The `gamma` and `artifact` labels are therefore softer than the `lithium` ones, which is consistent with where the confusion matrix of Section 4.4 puts nineteen of its twenty-eight errors. It is also why that boundary is left where it is rather than refined: no quantity of further labelling sharpens a distinction the images do not carry.

3.3.4. Known limitations

Saturation destroys the energy axis. At the gain used for these acquisitions essentially every heavy track clips, so `total_charge` is a lower bound and the deposited-energy spectrum cannot be read; Section 3.5 works that out in full.

Two things put the ceiling at 255, and only one of them is the gain. The other is the output format: `ids_capture.py` requests `Mono8` and converts to it, so a sensor that digitises more finely is written to disk with 256 levels. Both are choices made at acquisition and neither can be undone afterwards, which is why this is a limitation of the acquisition and not of the analysis, and why it bites hardest in live operation. It also means the remedy is not only a lower gain: requesting a deeper format would widen the same axis at the cost of twice the bytes per frame, against the throughput budget of Section 3.6.

The classifier has no `alpha` class. No labelled sample containing identified α particles exists, so the shipped model cannot report one and the alpha counter reads zero by construction. Any interface that displays a class the model was never trained on displays a zero that means "never trained", not "never seen".

The labelled set is small, and it is not a random sample. The labelled set holds two hundred hand-labelled clusters, against roughly four thousand in [6, Table 8.5]. What that size costs is measured later, and the ceiling turns out to lie in the class definitions and not in the number of labels.

We drew the two hundred from frames 539 to 638, the fortieth to the hundred and thirty-ninth of a run of fifteen hundred, so they sample the first tenth of it and nothing after. That is the stretch in which the background model is still settling, and the tracks labelled there were therefore found against a slightly different background from the ones classified later. Nothing in the results depends on the labels being representative of the whole run — the beam-off floor

rests on saturation and shape rather than on the classifier, and the counted rate treats every frame the same way — but a labelled set drawn evenly across the run would be a better one, and drawing it is cheap.

3.4. Commented Code Excerpts

Three excerpts are reproduced below, chosen because each carries a decision rather than a mechanism. Each is followed by the reasoning it encodes; the code is there to be checked against that reasoning, not read on its own. All three are pulled from the real source files at compile time with `\lstinputlisting`, so this thesis cannot drift away from the code it describes. Each caption names the file and the function rather than relying on the line numbers, which move whenever the source is edited; the excerpts themselves are numbered from 1.

3.4.1. Why a detected track never enters the background statistics

```
1    cv::Mat normal_mask;
2    cv::bitwise_not(event_mask_f, normal_mask);
3    if (!dead_mask_.empty()) {
4        normal_mask.setTo(0, dead_mask_);
5    }
6    welfordUpdate(pixels, normal_mask);
```

Code excerpt 1: `BackgroundModel::update()` — building the mask of pixels the Welford pass is allowed to learn from (`Code/src/BackgroundModel.cpp`).

The entire decision fits in two masks. `event_mask_f` holds every pixel that exceeded the threshold on this frame; `dead_mask_` holds every pixel already judged dead or hot. The complement of the first, minus the second, is the set of pixels the update is allowed to touch, and `welfordUpdate()` skips everything outside it.

Letting the detected pixels through instead would be self-defeating. A track would raise both the mean and the variance of exactly the pixels it landed on; that raises their threshold; and that desensitises the detector precisely where the signal arrives, so the background model would progressively absorb the signal it exists to separate. The price of excluding them is that every pixel accumulates its own number of samples — which is why `count_` is a full matrix and not a scalar — so a frequently hit pixel carries a less precise estimate than a quiet neighbour. Welford's recurrence is exact for any sample count, so this costs precision, never correctness.

This is where the escape of Section 3.3 happens in the source: `normal_mask` is the complement of the event mask, so a pixel carrying the faint edge of a track, below mean $+ 5\sigma$, is not in the event mask and does enter the update.

3.4.2. Where the size cut is applied, and why it belongs there

```
    // Connected-components labeling (8-connectivity)
    cv::Mat labels, stats, centroids;
    int n = cv::connectedComponentsWithStats(closed_mask, labels, stats, centroids,
                                             8, CV_32S);

    std::vector<Cluster> clusters;
    clusters.reserve(n - 1);   // label 0 = background

    // Runtime floor takes precedence over the compile-time one when higher ---
    // filtering on the cheap connected-components area BEFORE the expensive
    // characterise() call is what makes this a throughput lever, not just a
    // CSV-size lever.
    const int min_size = std::max(MIN_CLUSTER_SIZE, min_size_px_);

    for (int i = 1; i < n; ++i) {
        int sz = stats.at<int>(i, cv::CC_STAT_AREA);

        // Quick size pre-filter before full characterisation
        if (sz < min_size || sz > MAX_CLUSTER_SIZE) continue;
```

Code excerpt 2: `ClusterDetector::detect()` — connected-component labelling followed by the size pre-filter (`Code/src/ClusterDetector.cpp`).

The cut itself is unremarkable; where it sits is the point. `connectedComponentsWithStats()` already returns each component's area, so the filter reads a number that has been computed anyway, and it runs *before* `characterise()` — the expensive call that walks every member pixel to accumulate charge, centroid and inertia. Moving the same cut downstream would still shrink the CSV, but it would no longer save any work. Placed here it is a throughput lever, and that matters because the beam-on data is dominated by exactly the clusters it removes: some 224 000 single-pixel noise excursions in 1500 frames, against a track population some four times rarer.

The runtime floor takes the maximum of the compile-time constant and the command-line value, so a lab operator can raise it but never silently lower it below what the detector considers physical. The signal-to-noise cut is deliberately *not* here: it needs `peak_snr`, which only exists after characterisation, and it marks the cluster `is_valid = false` instead of dropping it. Rejected clusters are still written to the CSV, with `filter_status` recording the verdict, so a cut can be audited or reversed after the fact without re-running the detector over the frames.

3.4.3. The single-cluster call the live path uses

```python
def predict_single(features: dict[str, float],
                   bundle_path: str = BUNDLE_PATH) -> dict[str, Any]:
    """Classify ONE cluster's feature vector.

    'features' must contain every raw column FEATURE_COLS needs except the
    derived 'axis_ratio' (computed here): size_px, aspect_ratio, elongation,
    compactness, total_charge, peak_value, peak_snr, sigma_x, sigma_y.

    Returns {"label": str, "confidence": float, "proba": {class: prob, ...}}.
    """
    bundle = load_bundle(bundle_path)
    row = add_derived_features(pd.DataFrame([features]))
    X = row[bundle["feature_cols"]].values

    proba = bundle["model"].predict_proba(X)[0]
    idx = int(np.argmax(proba))
    label = bundle["label_encoder"].inverse_transform([idx])[0]
    return {
        "label": str(label),
        "confidence": float(proba[idx]),
        "proba": {cls: float(p) for cls, p in zip(bundle["classes"], proba)},
    }
```

Code excerpt 3: `predict_single()` — classifying one cluster without going through a CSV (`Code/src/ml_classifier.py`).

Offline, classification is a batch operation: a CSV of every cluster in the run goes in, a classified CSV comes out. On a live beam there is no such file, and waiting for one would defeat the purpose. `predict_single()` is the same computation reduced to one cluster — the derived `axis_ratio` column is recomputed here rather than assumed, and the feature order is taken from `bundle["feature_cols"]` rather than from a local constant, so a model trained with a different column order cannot be silently misread.

It returns the full probability vector alongside the winning label, not just the label. That is what lets the live interface show how confident the classifier actually is on a given cluster, instead of presenting an arbitrary verdict as a measurement.

3.5. Deposited Charge as a Proxy for Energy

The pipeline's proxy for deposited energy is the `total_charge` column: the sum over a cluster's pixels of (pixel value − background mean), which is exactly the excess charge the track deposited. It is proportional to the deposited energy *only while no pixel clips at 255*. A saturated cluster loses whatever charge sat above the clip, so its `total_charge` is a lower bound, and the four lines compress towards each other and merge.

This is why `study_energy.py` prints the saturated fraction before it prints any peak structure, and why the acquisition offers a reduced-gain "energy" preset alongside the full-gain "sensitivity" one: on this sensor the gain, not the exposure, sets whether a prompt track clips, because the charge is deposited essentially instantaneously.

At the gain used for the 2026-07-11 acquisitions essentially every cluster above five pixels is saturated, and above ten pixels it is all but every one. So `total_charge` is a lower bound

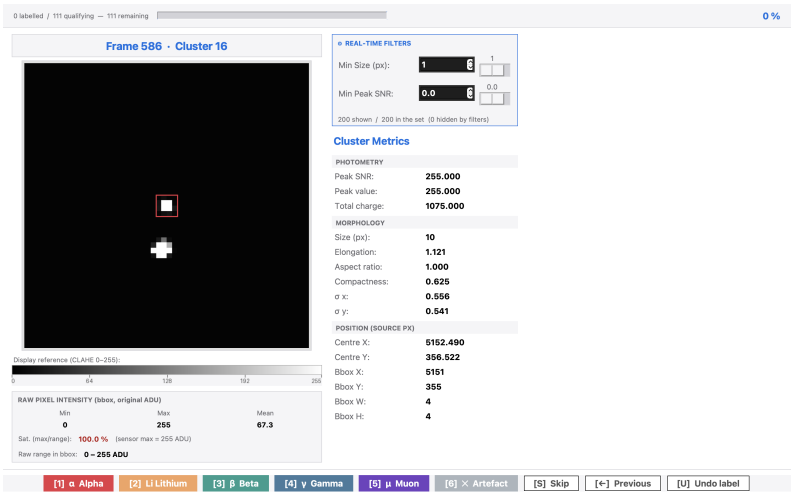


Figure 3.8: The labelling interface, `labeling_ui.py`. Each cluster is shown at the same magnification as the plate of Figure 3.7, with its measured features beside it, and the verdict is written into the `true_label` column of the CSV. The two hundred clusters of Section 4.4 were labelled here; the counter at the top counts the current sitting, not the file, and eighty-nine of the two hundred were already done when this was taken.

Source: screenshot I took of the interface running on the annotation sample shipped with the kit.

for every heavy track, and the histogram cannot yet be read as an energy axis. What a two-component fit returns for the ratio of the two peaks is not quoted here. Deriving it needs the fit of `study_energy.py` run on the present dataset, and that has not been done.

Closing this requires one acquisition with the gain tuned down until the saturated fraction falls to a few percent. The `--auto-tune` search in `ids_capture.py` performs exactly that scan automatically, with the beam on, and writes the resulting value into the energy preset. A single short run of that kind would let this section carry a spectrum with the lithium and alpha populations resolved, and re-labelling a sample from it would also give the classifier the `alpha` class it currently lacks.

3.6. What the Pipeline Costs

The detector has to keep up with the camera if it is to be used during a run, so we measured it. We ran the same binary on the same 500 dark frames ten times on each platform, with the frames already in the operating system's cache, and Table 3.2 reports the mean and the sample standard deviation over those ten repeats.

The sequential part is essentially the whole cost on all three, 98.9 % on the M1 and 99.5 % and 99.7 % on the Ryzen, so adding loader or detector threads cannot help; Section 5 returns to that. Against an acquisition period of 557 ms, which the benchmark derives from the run's own frame rate, the Windows figure leaves no margin at all.

The third column is the reason the second one is there. Windows and Linux ran on the same laptop, the same 500 frames and the same flags, so the only variables between them are the operating system and the compiler — and the same work takes 371.7 ms under GCC and 561.4 ms under MSVC, a factor of 1.5. That is enough to change the verdict rather than just

	Apple M1	Ryzen 7 5700U	
per frame, ms	macOS, clang	Linux, GCC	Windows, MSVC
wall clock	169.3 ± 5.3	371.7 ± 58.3	561.4 ± 13.7
total sequential	167.5 ± 5.3	369.9 ± 58.3	559.9 ± 13.5
background update	159.6 ± 5.0	343.1 ± 39.7	442.3 ± 10.0
snapshot	7.8 ± 0.3	26.5 ± 19.1	117.3 ± 3.8
mean clone	3.8 ± 0.1	11.4 ± 8.9	61.1 ± 1.9
σ derive	4.0 ± 0.1	15.2 ± 12.0	56.2 ± 2.1
waiting for frames	0.1 ± 0.0	0.2 ± 0.0	0.2 ± 0.0

Table 3.2: Cost of one 20.1-Mpixel frame, ten repeats per column, mean and sample standard deviation. The two right-hand columns are the *same laptop*, a 15 W mobile part, booted into two operating systems and built with two compilers; the M1 is a different machine, which is most of the factor of 3.3 between the outer columns. All three returned the same 39 071 clusters on every repeat.

Source: `benchmark_detector.py`; the reports are `Code/benchmark_mac.txt`, `Code/benchmark_linux_2026-09-02.txt` and `Code/benchmark_windows_2026-08-26.txt`.

the number: the Linux build processes frames faster than the camera produces them, with a third of the period to spare, and every one of its ten repeats, including the slowest at 517 ms, came in under the Windows mean. The margin the conclusion says the pipeline does not have on the laboratory machine is available on that machine today, by booting it differently — for the processing. Nothing was acquired under Linux, and Section 5 says what would be needed to move the live path across. The median interval measured between frames is 555 ms (Section 5). The two are medians of two different runs, the dark one and the signal one, and they differ by 0.4 %; the conclusion below does not turn on which is taken.

The 561.4 ms above were measured with the frames already cached. A first pass over cold files, on a machine whose antivirus reads each one as it is opened, took 785 ± 276 ms and dropped 26.9 % of frames, against 0.5 % for the cached run. The detector is close enough to the camera's rate that whatever else touches the disk decides whether it keeps up.

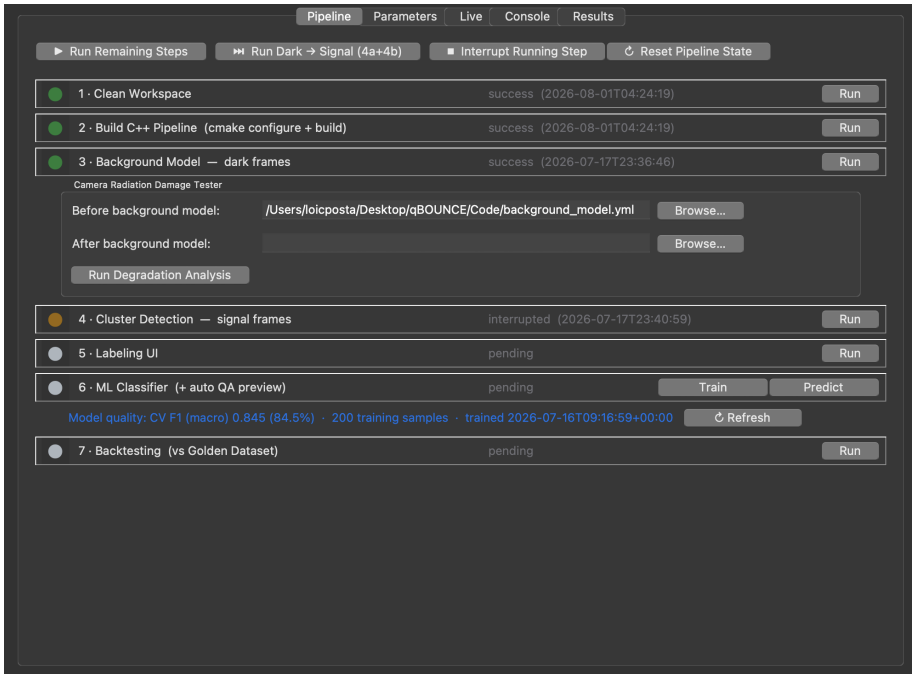
3.7. Operating the Pipeline: Two Paths

The graphical front end serves two quite different situations, and the order of operations is not the same in each. Table 3.3 sets them side by side; the user guide [21] documents every field.

The one ordering constraint worth stating in the text: step ② refuses to start without a background model, because the model is built on the machine and never shipped. Pressing it first on a fresh installation is the normal first-run mistake, and the interface answers it by offering to run step ① there and then.

Figures 3.9 and 3.10 show both paths as the interface presents them.

(a)



(b)

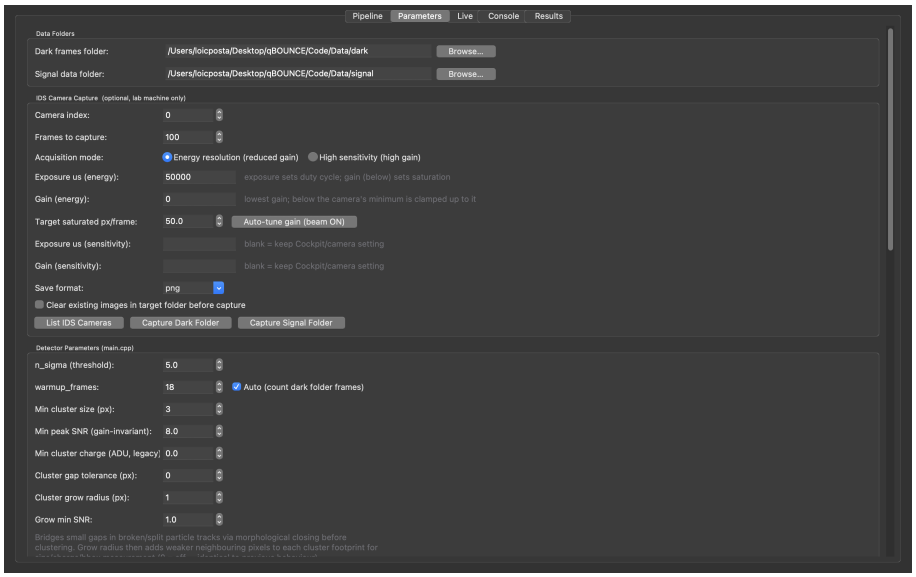
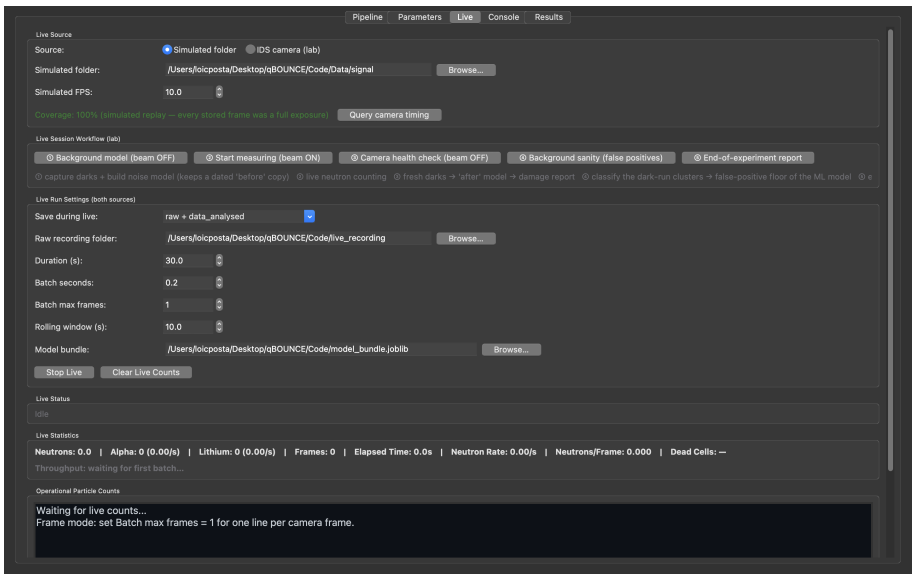


Figure 3.9: The two tabs of the office path. (a) Pipeline: the steps of Table 3.3, each with its own state and timestamp. The *Camera Radiation Damage Tester* under step 3 compares background models taken before and after a beam session, the measurement Section 4.7 rests on; the note under step 6 reports the classifier's cross-validated quality. (b) Parameters: the acquisition presets of Section 3.5 above, every detector threshold quoted in this thesis below, their values listed in Section 3.3.

Source: screenshots I took with the replay source selected; they are not photographs of the beamline installation.

(a)



(b)

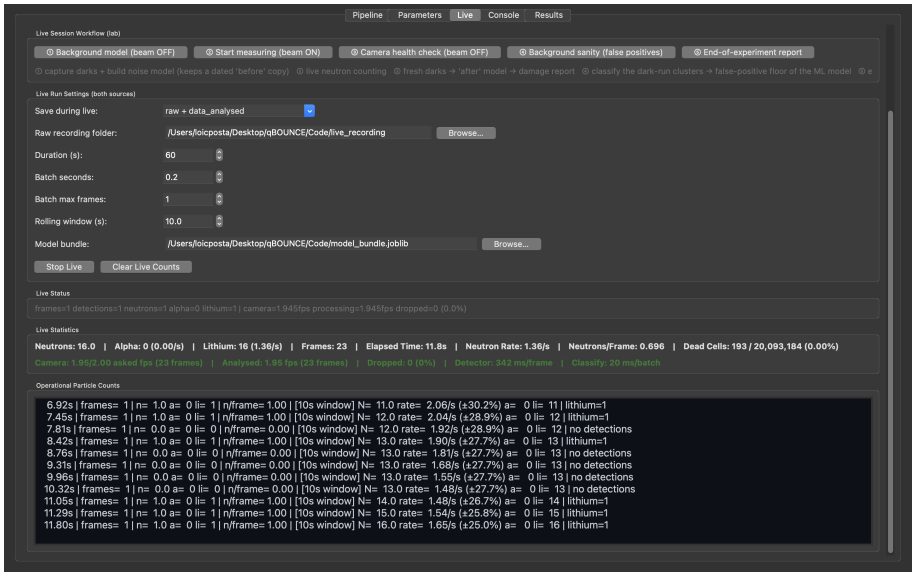


Figure 3.10: The Live tab, at rest and during a run. (a) at rest: the source selector is set to *simulated folder*, which replays recorded frames through the same engine the camera would feed, and the coverage label states the duty cycle any rate must be divided by. The five numbered buttons are the lab path of Table 3.3; step ④ is the beam-off false-positive measurement reported in Section 4.4. (b) during a run — here a replay of recorded frames rather than a live beam. The statistics line counts particles by class and reports the dead-cell total; the throughput line below compares the requested frame rate against the rate actually analysed and turns red as soon as frames are dropped, which is the interface's answer to the acquisition shortfall described in Section 5.

Source: as above.

	Office path — analysing recorded frames	**Lab path** — live acquisition with the beam
1	Parameters: point the dark and beam-on folders at the recorded runs, set n_σ and the cluster filters	Parameters: camera index, and the exposure/gain preset (*energy* for a readable spectrum, *sensitivity* otherwise)
2	Pipeline: Build C++ Pipeline	Live: Query camera timing — confirms the real exposure and frame rate, and the resulting duty cycle
3	Pipeline: Background Model (dark)	Live ①: Background model, **beam off** — records fresh darks and builds the model
4	Pipeline: Cluster Detection (signal)	Live ②: Start measuring, **beam on** — the measurement itself
5	Pipeline: Labeling, then Classify	Live ③: Camera health check, **beam OFF** — fresh darks, "after" model, damage report
6	Parameters: Energy analysis — the deposited-charge spectrum of Section 4.5	Live ④ and ⑤: background sanity, then the end-of-experiment report

Table 3.3: The two ways through the graphical front end. The office path never touches the camera; the lab path alternates beam-off and beam-on steps, which is why it cannot simply be automated end to end.

4. Results and Discussion

Every figure in this section was produced by the pipeline of Section 3, run on the July 2026 datasets in `Code/Data_tot/`: the 500 dark frames of `2026-07-11_3-001_background_575-6ms_2fps` and the 1500 beam-on frames of `2026-07-11_3-002_575-6ms_2fps_first_UCN`, both 5472×3672 pixels at a 575.6 ms exposure, with $n_\sigma = 5$ and no minimum cluster size, and a warm-up of ten frames.

One property of that run matters for everything read out of it. The tracks span 0.5–8.2 mm in height, 87 % of the sensor, so the sensor was looking at a beam that had not been sorted by height: a state-selecting absorber leaves a band a few tens of micrometres tall, three parts in a thousand of the same sensor. Every rate quoted below is therefore the rate of this configuration and not of a state-selected beam, and the outlook of Section 5 carries its budgets in events for that reason, converting to seconds only at this rate. Each figure names the script that draws it. All of them read that run except the beam-off floor of Section 4.4, which is measured on an earlier pass over the same frames, made with a build that retired no hot pixels, and says so where it is quoted. The detector is deterministic, so these counts are exact rather than estimated: the whole analysis was regenerated from the raw frames while this section was being written and returned the same numbers, cluster for cluster. Appendix A states the rule behind every ± quoted below.

4.1. What the Sensor Does in the Dark

The first thing the background model reveals is that this sensor is limited by *quantisation* rather than by read noise. Over 500 dark frames, 97.9 % of the 20.1 million pixels read exactly zero, and 99.88 % have a per-pixel standard deviation below one ADU. The discrete peaks visible at 1, 2, 3 and 4 ADU in Figure 4.1(a) are populations of pixels carrying a *fixed* offset: they return

the same integer in almost every frame, so their measured variance is essentially zero.

This has a direct consequence for the detection threshold. A pixel with $\sigma = 0$ would make the criterion pixel $>$ mean $+ n_\sigma\sigma$ degenerate, since any excess whatsoever would count as a detection. The detector therefore floors σ at one ADU before applying the threshold. Because almost the whole sensor sits below that floor, the effective criterion over 99.88 % of the pixels is mean + 5 ADU — a fixed cut, not an adaptive one. It is more accurate to describe the method as "5σ with a one-ADU noise floor" than as a purely adaptive 5σ threshold, and the distinction matters when comparing this detector against one operating on a sensor with genuine read noise.

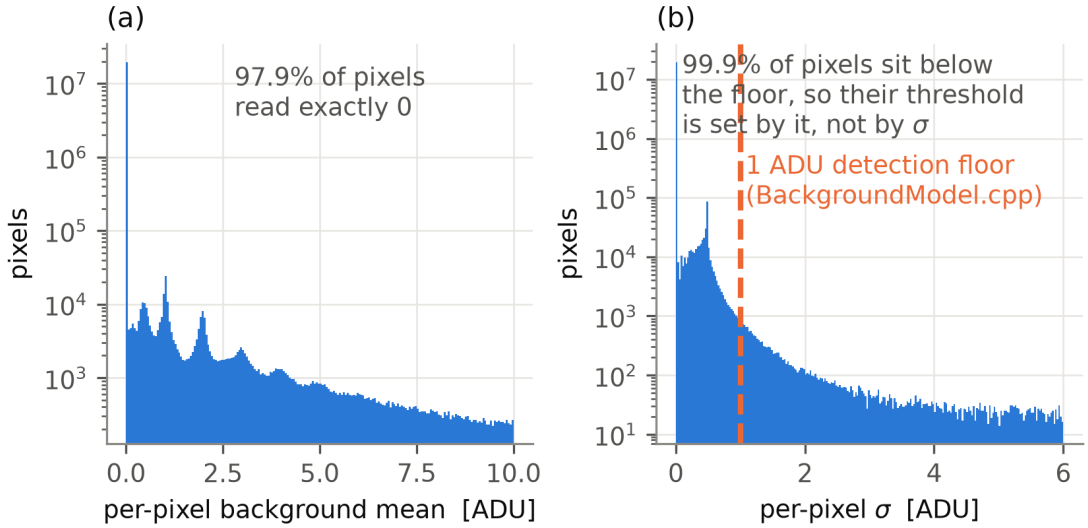


Figure 4.1: What the sensor does in the dark, over 20.1 Mpixel at a 575.6 ms exposure. (a) the distribution of the per-pixel background mean, the pedestal, for the model built from 500 dark frames; (b) the distribution of the per-pixel σ, the read noise, for the same model.

Source: `make_results_figures.py`.

4.2. How Many Dark Frames the Background Model Needs

How many dark frames does the background model actually need? The answer is not visible where one would first look for it. Panels (a) and (b) of Figure 4.2 barely move: the 99th percentile of σ goes from 0.416 to 0.456 ADU while the frame count grows by a factor of fifty. That is not evidence of fast convergence — it is the 98 % of pixels that read zero and require no statistics at all to estimate.

The quantity that does converge, and the one that matters operationally, is the rate of false detections. Panel (d) follows it frame by frame through the 500-frame run: it falls from roughly 250 clusters per frame at the start to about 25 at the end, as the accumulated variance replaces the flat starting value the model uses before it has seen enough frames. This order-of-magnitude improvement is the floor against which any beam-on rate must be compared, so under-running the dark acquisition directly inflates the apparent signal.

Figure 4.3 shows the same maturation from the other side. At small N the histogram of per-pixel σ is a comb rather than a smooth distribution: an estimate built from a handful of

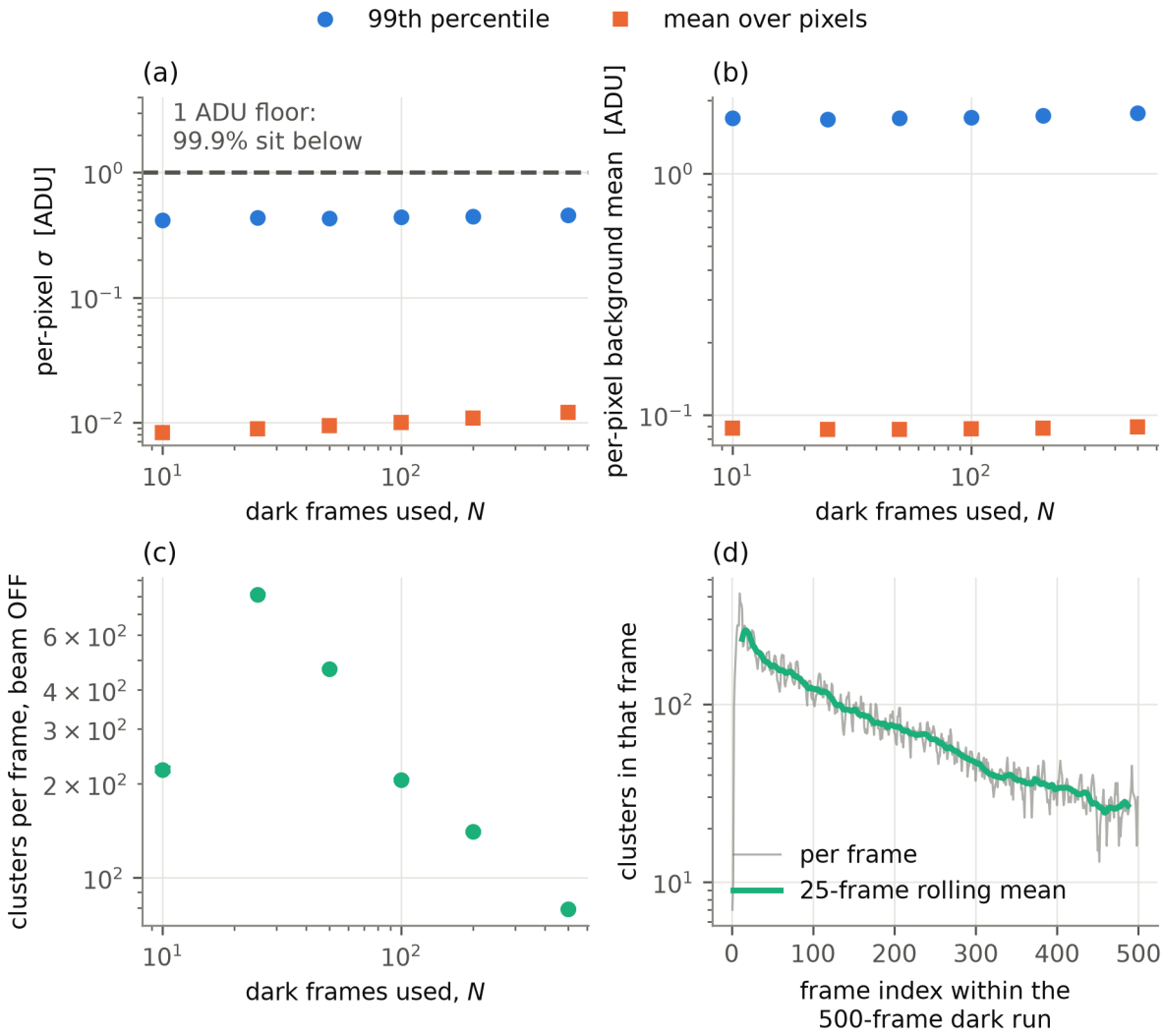


Figure 4.2: Background-model convergence, built from the first N frames of the same 500-frame dark run so that frame count is the only variable. (a) the per-pixel noise estimate against frame count, as the 99th percentile and the mean over pixels; (b) the same for the pedestal, the per-pixel background mean; (c) the false-positive floor at 5σ, averaged over the run; (d) the same floor followed frame by frame through the 500-frame run, with a 25-frame rolling mean.

Source: `make_results_figures.py`.

integer samples can only take certain discrete values. The comb fills in as frames accumulate — a textbook picture of estimator quantisation, and a useful visual check that the model is being fed enough data.

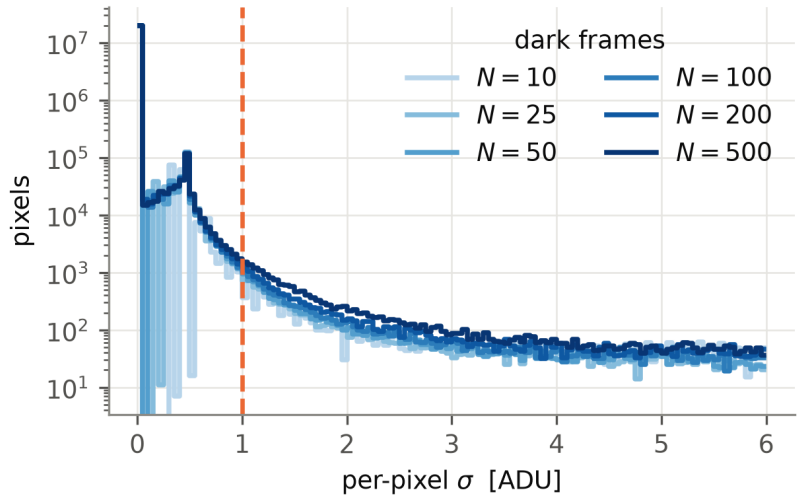


Figure 4.3: The per-pixel σ histogram as dark frames accumulate.

Source: `make_results_figures.py`.

4.3. Beam-On Data: Separating Tracks from Noise

With the background model in place, the separation between sensor noise and particle tracks is unambiguous. Figure 4.4(a) shows two populations far apart in size: roughly 224 000 single-pixel clusters, which are noise excursions, and a track population whose median is 13 pixels and whose bulk lies between 10 and 20, which corresponds to the charged capture products. The two are not disjoint — an eighth of the tracks reach only five to nine pixels, for the geometric reason Section 4.4 gives — but they are separated by more than an order of magnitude in the middle, which is what makes the subsequent classification tractable.

Panel (c) separates the large clusters from noise: the collected charge grows monotonically with the cluster footprint, and a noise excursion has no such relation. It does not show that the charge measures deposited energy. At this gain almost every heavy track is clipped, so a cluster's charge is close to its pixel count times a ceiling, and the monotonic relation would hold whether or not the charge followed the energy. Section 4.5 takes that apart.

The rates follow: 215.4 ± 0.4 clusters per frame with the beam on, against 79.5 ± 0.4 per frame averaged over the dark run. That average is not the floor to subtract, and the distinction matters. It is inflated by the early frames, where the model was still converging and fired on almost anything; Section 4.2 shows the rate decaying through the run to 26.2 ± 0.7 per frame over the last fifty. The beam-on figure was measured against a model that had already seen all 500 dark frames, so the like-for-like floor is the converged one, 26.2 per frame, and quoting the run average instead would over-subtract by a factor of three.

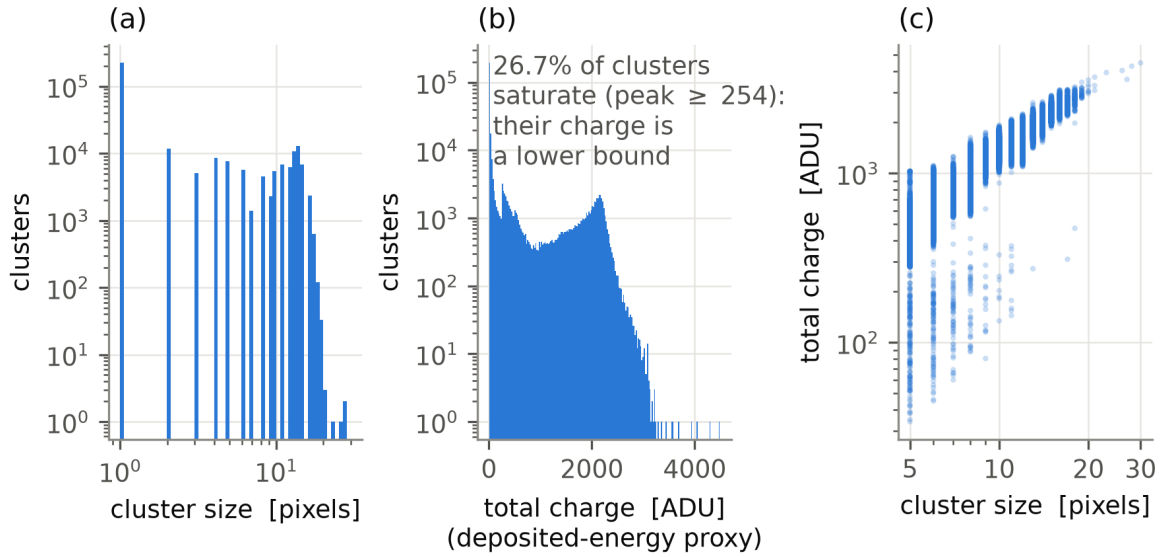


Figure 4.4: Clusters found in 1500 beam-on frames, against the background model built from the 500 dark frames. (a) the distribution of cluster size in pixels; (b) the distribution of `total_charge`, the proxy for deposited energy; (c) charge against size for clusters of at least five pixels.

Source: `make_results_figures.py`.

Is the rate constant? A beam test asks not only how many neutrons arrive but whether they keep arriving at the same rate, and the run is long enough to say something. Fitting a straight line to the per-frame `lithium` count over all 1500 frames gives a fall of $3.1 \pm 1.4\,\%$ across the 833 s of the run, which is $2.2\,\sigma$ from flat, and the three thirds of the run are compatible with each other (Section 4.7). Taken at two standard deviations the measurement bounds any monotonic drift at 6 % over the run.

That bound applies to the *product* of two things, and this setup cannot separate them. The counted rate is the beam intensity multiplied by the efficiency of a detector that is losing pixels while it counts (Section 4.7), so a flat count is consistent with a flat beam and a flat detector, and also with the two drifting in opposite directions by the same amount. Nothing here monitors the incident flux independently: the mirrors carry laser interferometers, the beam does not carry a counter of its own. Adding one — a small monitor detector out of the main path — would turn this bound into a measurement of the beam alone, and it is the cheapest addition on the list of Section 5.

Figure 4.5 shows where the particles actually landed. The illumination is broad and centred, falling away towards the edges and corners of the sensor — the profile of the beam as delivered through the aperture in front of the detector, rather than a property of the sensor itself.

The binning is worth explaining, because the pipeline also exports a per-pixel version of this map that is unusable as it stands. At 20.1 Mpixel, a handful of pixels accumulating up to 135 hits set the linear colour scale against a median of one, so the entire sensor renders as a single flat colour. Summing over 48×48-pixel blocks converts isolated hits into a density that survives display, and clipping the scale at the 99th percentile stops the remaining outliers from taking it

over again. In total 1.11×10^6 hits were recorded, touching 4.55 % of all pixels.

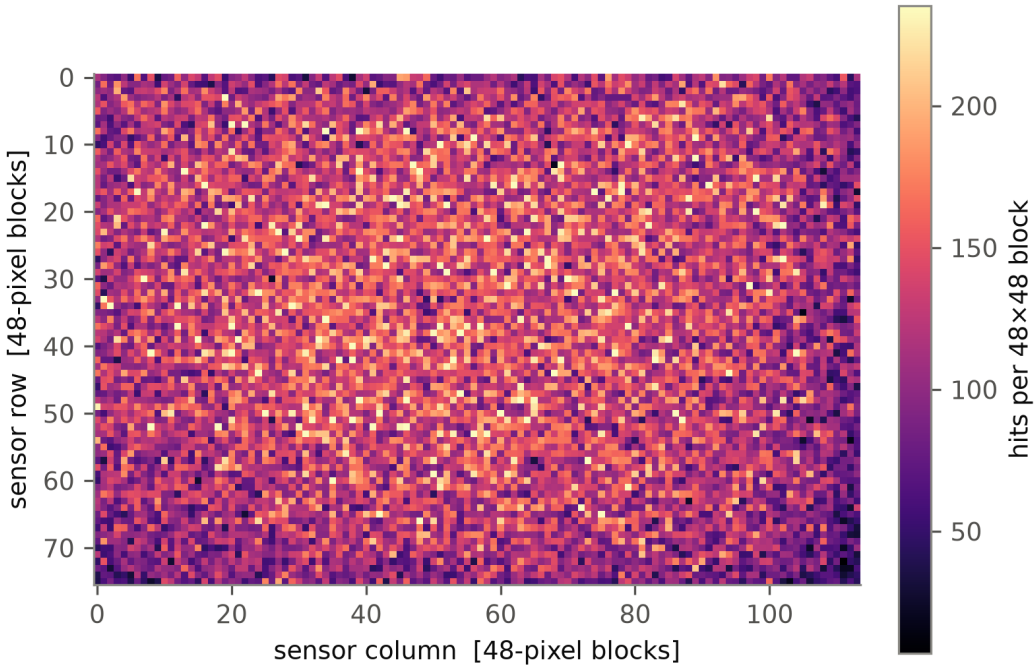


Figure 4.5: Arrival map of every detected particle over the 1500 beam-on frames, summed into 48×48-pixel blocks. The colour scale is clipped at the 99th percentile.

Source: `make_results_figures.py`.

4.4. The False-Positive Floor, Measured

Three different quantities are routinely called "the false-positive rate". They are not interchangeable, and this section measures the second of them.

1. **Classifier error.** Does the model reproduce the human verdict? This is measured against hand-applied labels on clusters the model never trained on. Whether real particles were present is irrelevant to it.

2. **The beam-off floor.** What does the whole chain report with the beam closed? This mixes sensor noise *with* genuine ambient radiation. The two cannot be separated by analysis, and for the purpose of a rate measurement they need not be: both are present with the beam open as well, so both subtract.

3. **The instrumental false-positive rate.** How much of the beam-off floor is the instrument rather than the room? Separating that requires a shielded — cadmium-covered — acquisition, which is a measurement and not a re-analysis. It is listed in Section 5 as outstanding.

The beam-off floor. Two passes over the dark run are available and neither yields a neutron. With the hot-pixel filter active the run leaves five objects in 500 frames, all classified `artifact`; the pass discussed below switches the filter off, which is the harder test and the one worth reporting. That pass is made over the dark run in which no hot pixel was ever retired, and the number it produces has to be read with that in mind. The detector retires a pixel that fires on five consecutive frames; the July build that made this pass did not, and its run log records not one retirement over the whole 500 frames. The current detector retires 7081 of them on those same frames at a warm-up of 500 and 7226 at a warm-up of ten, so the retirement is not something the warm-up switches on. We keep the unretired pass here on purpose, because a floor measured with the noisiest objects left in is the harder test.

Running the full chain — detection and classification — over the 499 beam-off frames that way gives 1 447 466 clusters, 2900.7 ± 2.4 per frame. The current detector on the same frames gives 39 695 at the warm-up of ten that Section 4 declares and 39 071 at a warm-up of 500, and the rate of 79.5 per frame quoted in Section 4.3 is that retired pass. A factor of 36 separates a pass that retires recurrent hot pixels from one that does not. Nothing about the sensor differs between them.

Those 1.45 million clusters come from 8966 distinct pixel positions, and 5857 of them fire on fifty frames or more and account for 97.4 % of the total. The count is large because a few thousand pixels are counted many times over, so it should not be read as 1.45 million independent chances to be wrong. The trained model assigns them to `artifact` (1 444 806) and `gamma` (2660, that is, 5.33 ± 0.10 per frame). It assigns *none* of them to `lithium`. The highest neutron probability given to any of the 1.45 million objects is 0.0196, and not one exceeds 0.2; the decision boundary at 0.5 is never approached.

That verdict has to be read for what it is. Of those 1.45 million objects, 1 446 121 are a single pixel, and the 200 hand-labelled clusters the classifier learnt from contain no cluster smaller than three pixels: their sizes run from 3 to 17, with a median of 5. On single-pixel objects the model is therefore extrapolating outside anything it was shown, and its confidence there is not evidence of anything.

The floor does not rest on that verdict. It rests on what a beam-off frame contains. Of those 1.45 million beam-off clusters, nine reach 3 pixels, one reaches 5, and none reaches 10; an independent re-run of the dark pass under different detector settings reproduces the same counts. Beam-on frames are not made of tracks alone — they also carry 223 821 single-pixel objects and 47 089 between 2 and 9 pixels — which is why the comparison that matters is between the beam-off objects and the tracks, and not between the two runs as a whole.

Size alone does not quite close it, and the honest version is the stronger one. The smallest tracks reach five pixels and one beam-off object in 500 frames reaches six, so the two distributions touch at their edges. What separates them there is not size but saturation: of the 39 071 objects the beam-off run yields at 5σ, not one reaches the clipping level, the highest peaking at 244 ADU, while all 59 259 clusters called neutrons clip at 255 without exception. The beam-on run does hold 379 unsaturated objects of at least five pixels, and the classifier calls none of them a neutron. The single six-pixel object is not a borderline track either — its elongation is 105 where a track sits near 1.2, its peak 23 ADU, its compactness 0.14 against 0.75 (Section 3.3). It is a thin streak, and no cut on size was ever what excluded it. No false neutron classification was observed, and that is an upper bound measured under these conditions rather than a demonstration that the true rate is zero.

Ambient cold neutrons in the experimental hall should produce genuine captures during a dark

run. If they did, they would appear as tracks of that size. At most one candidate does, in 499 frames — an ambient capture rate below 0.002 per frame against the 39.5 neutrons per frame counted with the beam open, a ratio of some 5×10^{-5}.

Classifier error and its stability. Every tree of a random forest is grown on a random subsample, so fitting twice on the same labels yields two different classifiers, and one quoted accuracy does not say whether it is typical. Following the procedure of [6], we therefore repeated the whole fit 1000 times, each with a fresh seed and a fresh stratified split, and scored each time on the held-out third. Figure 4.6 histograms the two quantities that result.

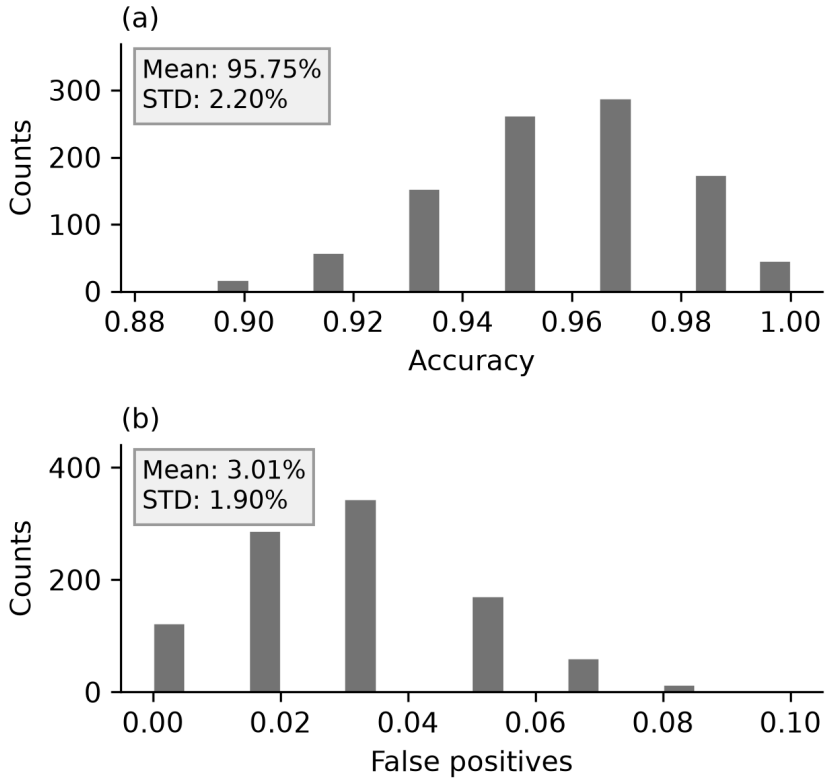


Figure 4.6: Classifier stability over 1000 independent retrainings on the 200 hand-labelled clusters. (a) accuracy on the neutron category, $(\mathrm{TP} + \mathrm{TN})/N$; (b) the fraction of test clusters wrongly called a neutron. The histograms are combed because the held-out set holds 60 clusters, so both quantities can only take multiples of $1/60$ — the same estimator quantisation seen in Figure 4.3. Mean and standard deviation are indicative only for distributions this skewed. The spread measures the randomness of the split and of the forest, and nothing else: it does not carry the uncertainty of the hand labels themselves, the finite size of the labelled set, or any systematic bias in how it was drawn.

Source: `make_classifier_figures.py`.

The accuracy is $95.75 \pm 2.20\,\%$ and the wrongly-called-neutron fraction is $3.01 \pm 1.90\,\%$. That accuracy is the binary one of the lithium-against-rest decision, $(\mathrm{TP} + \mathrm{TN})/N$ on the held-out

third, and not a per-class score: the precision, recall and F_1 of each class are given separately below and are the numbers to compare against a per-class figure elsewhere. Table 4.1 shows where that performance is lost. Its three right-hand columns are the standard per-class scores: precision is the fraction of the objects given a class that belong to it, recall the fraction of the objects belonging to a class that are given it, and F_1 their harmonic mean.

	predicted			**per class**		
truth	`artifact`	`gamma`	`lithium`	prec.	rec.	F_1
`artifact`	**62**	6	3	0.827	0.873	0.849
`gamma`	13	**35**	3	0.795	0.686	0.737
`lithium`	0	3	**75**	0.926	0.962	0.943

Table 4.1: Confusion matrix over the 200 hand-labelled clusters, five-fold cross-validated so that every verdict is made on a cluster the model did not train on. Nineteen of the twenty-eight errors sit on the `gamma`/`artifact` boundary; no lithium is ever taken for an artifact, and three artifacts and three gammas are taken for lithium.

Nineteen of the 28 errors are on the `gamma`/`artifact` boundary, while `lithium` reaches a recall of 0.962 and a precision of 0.926, and no lithium is ever taken for an artifact. For a measurement that counts neutrons, the relevant figure is therefore the lithium F_1 of 0.943, not the macro-averaged 0.843: the averaged number is dragged down by a distinction between two single-pixel classes that does not enter the neutron count.

What the run then contains. With the classifier's behaviour established, its verdict on the run can be read. Applied to all 323 069 clusters of the 1500 beam-on frames it returns 227 543 `artifact`, 36 267 `gamma` and 59 259 `lithium`. The last is the quantity the experiment is after: 39.5 ± 0.2 neutron captures per frame, or 71.1 ± 0.3 per second at the 1.80 frames per second the camera sustained.

That count is not confined to the band of large tracks the threshold scan used as its proxy. 7287 of the lithium clusters, 12.3 %, measure between five and nine pixels against a median of thirteen. They are not a marginal population let in by a loose cut: every one of them clips at 255 ADU, as every large track does and as nothing in a beam-off frame ever does. Their total charge cannot be offered as a second argument, because once pixels clip that charge is a lower bound that largely follows the size (Section 4.5).

What separates them from a long track is the projected length, and the shape says so independently. The daughter leaves the boron layer in a direction nobody chooses: a track emitted steeply into the silicon covers few pixels, a grazing one covers many — and the steep track should be the rounder of the two. It is. The median elongation climbs monotonically with size, from 1.14 in the five-to-nine-pixel band to 1.15, 1.23 and 1.27 in the bands above, which is what one line projected at a varying angle produces and what a population of small *different* objects would not.

The same signature would follow just as well from the two daughters having different ranges, the α travelling further in the silicon than the ^{7}Li. Nothing here separates those two readings, and Section 4.5 says why: telling the daughters apart needs the energy axis this acquisition cannot provide.

This band is also where least is known about the classifier. Of the seventy-eight hand-labelled neutrons, seventy-one lie above ten pixels and *seven* lie here, against twenty-seven gammas and fourteen artifacts in the same range. The F_1 of 0.943 quoted above is a score for the band above ten pixels and it does not transfer to this one. That cost is not in the ± 0.2 on the rate, which is counting statistics alone: an eighth of the neutron count is assigned in a regime constrained by seven examples, and small clusters are where more hand-labelling would now do the most good (Section 5). The classification systematic of Appendix A carries this reservation as well as the one on class balance.

The classes are hard to separate here, and the labels the pipeline attaches in this band should be read for what they are. The 21 555 objects between five and nine pixels are split 7287 `lithium`, 13 942 `gamma` and 326 `artifact`, and the first two groups are not obviously different objects: both saturate — 100 % against 99.8 % — and they are told apart mainly by charge and shape, 1054 ADU and an elongation of 1.14 against 518 ADU and 1.51. Every verdict in this thesis is a *candidate* of its class rather than a certain member of it, and nowhere is that truer than here: a cluster called `gamma` in this band is a cluster the forest found more gamma-like than lithium-like on ten geometric features, trained on seven neutrons and twenty-seven gammas of this size. Where the boundary really lies is not settled by anything in this thesis, and it is the improvement the classifier most needs — a labelled sample drawn from the small clusters specifically, rather than from the run as a whole.

More labels. We repeated the fit with training sets from 6 to 160 clusters, in Figure 4.7, to see whether hand-labelling more would raise the score. It would not. The score rises steeply to about 60 labels and is flat afterwards — the last hundred labels are worth 0.035 in F_1, inside the error bars. The ceiling comes from the class definition. In a beam-off frame every `gamma` found is a single pixel, which is also what electronic noise looks like, so no quantity of examples can separate them. Ref. [6] meets the same wall and answers it with a buffer class (Candidate) between the certain and the ambiguous, together with priors tuned per detector. We adopted neither. Nothing in a beam-off frame has the shape of a track, so no false neutron is there to be classified before any verdict is asked for and a buffer class has nothing to catch; it would sharpen a distinction that does not affect the result.

4.5. Deposited Energy: Why the Spectrum Is Not Yet Readable

The deposited-charge spectrum of Figure 4.8 is where this dataset reaches its limit.

Applying the size cut that Section 4.3 justified, 99.49 ± 0.03 % of the 73 714 clusters above 5 pixels are saturated, and 99.96 ± 0.01 % of the 52 159 above 10 pixels: essentially every heavy track has at least one pixel clipped at 255.

Since `total_charge` sums the excess over the background, a clipped pixel contributes less than it should, and the value becomes a lower bound. The kinematics of Eqs. (2.5)–(2.6) put the α and the lithium of the dominant branch at 1.47 and 0.84 MeV, a ratio of 1.75, and a readable charge axis would show the two lines at that separation. Whether the measured distribution does is not settled here: the two-component fit that would answer it has to be run on this dataset with the model `study_energy.py` implements, and it has not been.

One structure the pooled histogram does show is not energy. It carries a second population at low charge, and that population does not survive being looked at at *fixed* cluster size: the charge distribution of the 6249 clusters of exactly 12 pixels is single-peaked and spans 1094 to

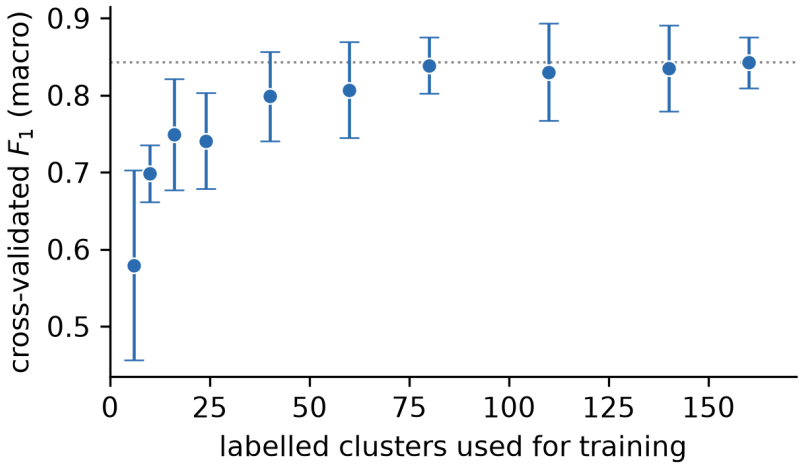


Figure 4.7: Cross-validated macro F_1 against the number of hand-labelled clusters used for training, five-fold, error bars the spread over folds. Points are not joined: nothing was measured between two training-set sizes. The dotted line marks the score reached with the full set.

Source: `make_classifier_figures.py`.

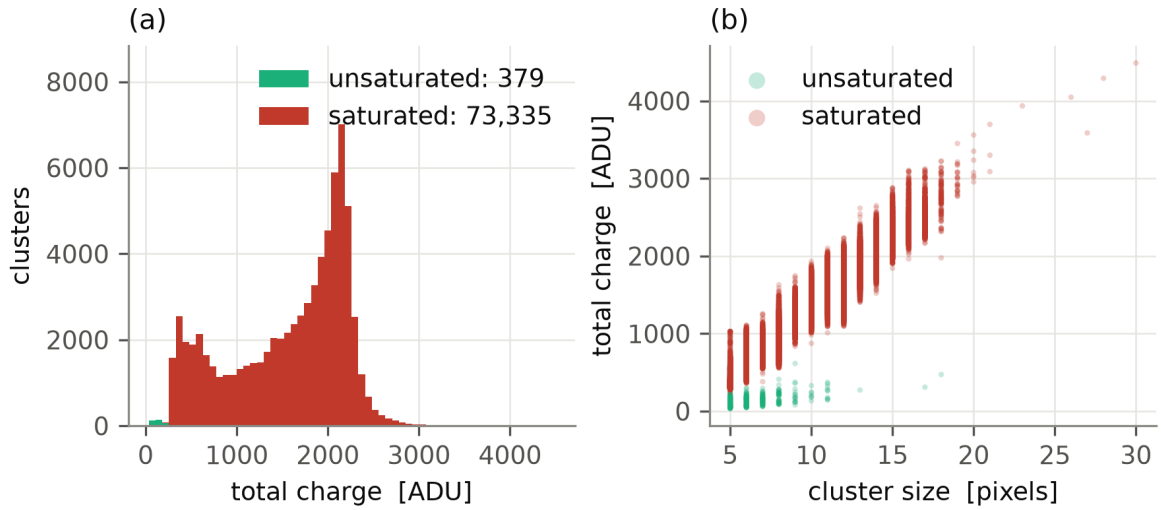


Figure 4.8: Deposited-charge histogram for clusters of at least $5\,\mathrm{px}$ (left) and charge against track size (right); red marks saturated clusters.

Source: `make_results_figures.py`.

2233 ADU, as is that of the 4506 of exactly 8 pixels, and of the 2375 of exactly 16. What looks like two components in the pooled distribution is the cluster-size distribution seen through the charge–size relation of Figure 4.4(c). With every heavy track clipped, the charge axis is currently a size axis in disguise, which is the same conclusion arrived at from the other direction.

What is certain is the direction of the bias — a clipped cluster is recorded lighter than it was, so any measured ratio is compressed towards unity — and that no re-analysis of these frames can undo it: the information was lost in the sensor, before the software saw it. Separating the lithium and alpha populations therefore requires a new acquisition at reduced gain, as argued in Section 3.5.

4.6. A Systematic Effect Found by Measuring

A last effect is small, and we report it for how it was established. Figure 4.9 is the control that established it.

`BackgroundModel::update()` performs its Welford pass on *non-event* pixels only: detected clusters and pixels already marked dead are masked out before the running mean and variance are touched. A real track therefore never enters the background statistics it is measured against. That is the design, and it is what the source does.

Nevertheless, the upper tail of the σ distribution is higher after a beam-on run than after the dark run alone, and the first reading of that observation — that the beam was contaminating the model — was wrong. The control is in Figure 4.9(a): the same tail grows with frame count in *pure dark* as well, from 0.74 to 1.13 ADU between $N = 10$ and $N = 500$, simply because more frames sample more rare excursions.

How much it would have grown by 2000 frames is an extrapolation, and it should be given as one. The six dark points do not lie on a power law: their local slope rises steadily from 0.012 to 0.232, so a line fitted to all six is flatter than the data where it matters and under-predicts. That line gives 1.19 ADU at 2000 frames and a line through the last three gives 1.48, against a measured 2.28 — an excess between $\times 1.5$ and $\times 1.9$. A dark run of 2000 frames would settle it and none was taken, so the width of that band is a limit of the data and not of the analysis.

The residual excess was first read as the faint halo surrounding a track: the edge of a cluster falls below mean $+ 5\sigma$, is therefore *not* excluded by the event mask, and would enter the statistics of the pixels around the track. That reading is testable on the frames already recorded, because the detector writes down where every cluster landed, and `Code/src/study_sigma_excess.py` tests it.

Call a pixel *loud* when its own σ exceeds 1.19 ADU, the value the six-point fit predicts for the 99.9th percentile. Taking the lower end of the band as the reference only sets where the line is drawn; what follows compares one population of pixels against another through the same line, and those ratios move very little with it. A halo would make the pixels next to a track loud. It barely does: one pixel out from a hit the rate is 13 % above what it is far from any track — a real difference over two million pixels, but nothing like an explanation — and by three pixels there is nothing left to measure. What is loud is the hit pixel itself, ten times more often than the far field, and the pixels retired during the run seventy times more often. Every one of those 34 850 retired pixels was hit at least once, so they are the population of Section 4.7 seen through a different quantity.

Removing the 4.5 % of pixels the beam touched takes the percentile from 2.28 to 1.76 ADU. Against the flatter extrapolation that is half the excess, against the steeper one two thirds:

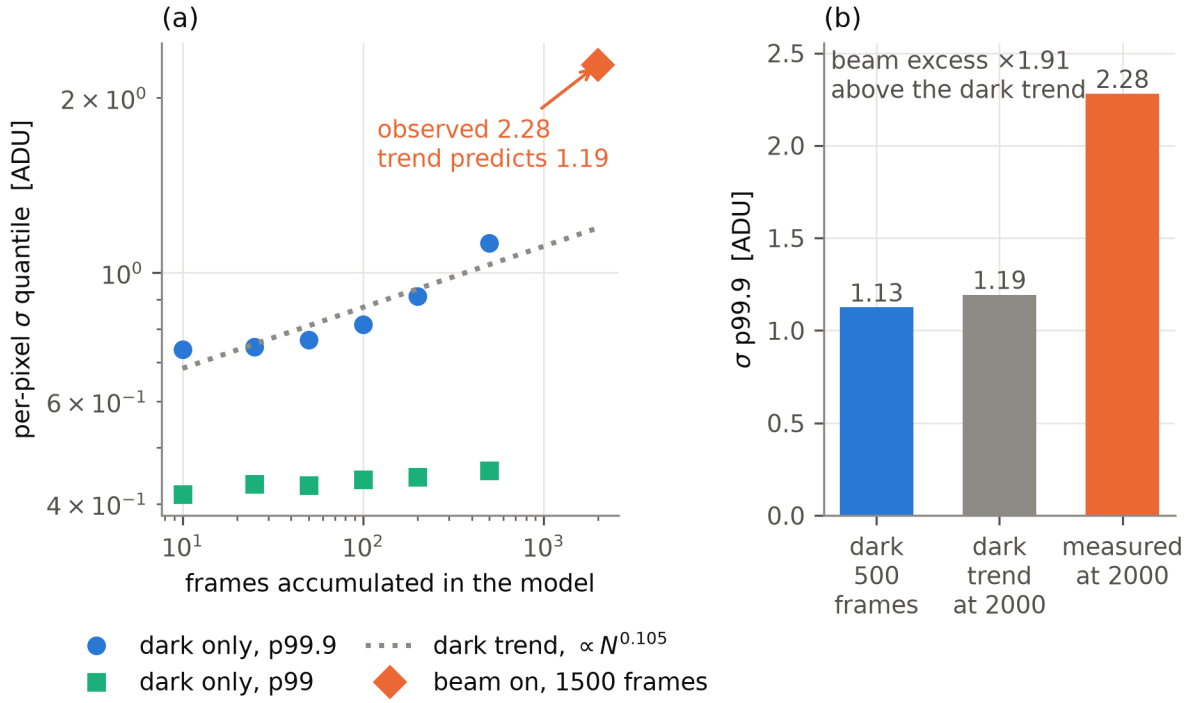


Figure 4.9: Growth of the upper tail of the per-pixel σ distribution with the number of frames accumulated. (a) the tail measured in the dark alone, at the 99th and 99.9th percentiles, against the number of frames; (b) the same quantity after a beam-on run, against what pure accumulation predicts. The dotted line is a power law $\sigma_{99.9} \propto N^{0.105}$, fitted by least squares on the logarithms of the six dark points alone. A power law is what pure accumulation would look like — the 99.9th percentile is set by how far into the tail N draws reach — but the six points are not one: their local slope climbs from 0.012 to 0.232, so the fitted line is flatter than the data at the end of the range and its extrapolation is a lower bound on what accumulation alone gives. At the 2000 frames the beam-on model has seen it predicts $1.19\,\mathrm{ADU}$ against a measured 2.28; a line through the last three points predicts 1.48. The excess is therefore between $\times 1.5$ and $\times 1.9$, and it is that band, not the single figure, that has to be explained by something other than accumulation.

Source: `make_results_figures.py`.

between a half and two thirds of it sits on one twentieth of the sensor. What sits there is not a halo *around* tracks but the track pixels themselves, hit again and again through the run and the same pixels that go on to be retired.

The remainder sits on pixels no cluster ever occupied, where no halo can reach. A sub-threshold contribution spread over the whole sensor would produce it, and nothing here measures one; it is also the part most sensitive to the extrapolation, and it would shrink to nothing if the dark trend were steeper still than the last three points suggest. What the test settles is the part that does not depend on that: the excess is concentrated where the beam landed, and the halo of a track is not what puts it there. Establishing even that much required plotting the dark trend first, without which the monotonic rise would have been read as a beam effect in full.

4.7. What Kills a Pixel

CMOS sensors are new to this experiment and their behaviour under a neutron beam is not documented in the group. The pipeline turns out to record what is needed to answer the first question about it — do pixels die, and does the beam kill them — without any dedicated instrumentation.

What "dead" means here. The word has no canonical definition, so we fixed one. It is a *choice*, and Appendix B states it in full. Two mechanisms mark a pixel as unusable in `BackgroundModel::update()`:

1. **stuck-at.** During the warm-up frames, pixels that read saturated or zero in nearly every frame are collected into a mask and never consulted again.
2. **persistent event.** From then on, any pixel that crosses the detection threshold on *five consecutive frames* is declared hot and masked, on the argument that a real particle does not strike the same pixel every frame.

The five is arbitrary. It was picked as a value large enough that no plausible sequence of real events would reach it and small enough to react within a few seconds of acquisition, and no scan over that parameter was performed. The measurements below make it clear that the choice is doing real work: what the rule catches is not a pixel that has failed outright but one that fires *intermittently*, and where the line falls between "noisy" and "dead" is set by that number. A definition built on the firing *rate* over a window — say, more than k events in m frames — would describe an intermittent pixel better than a run-length rule does, and would not need a pixel to misbehave on consecutive frames to be caught at all. Section 5 lists this among the things worth revisiting.

The second rule has one useful side effect: it *dates* each loss. The detector prints the frame index at which a pixel is retired, so the death time is recorded for free.

Comparing the two background models settles the question. After the 500 dark frames the mask holds 7075 pixels; after the 1500 beam-on frames it holds 41 925. The difference, 34 850 pixels, matches the running total in the detector log exactly, and Table 4.2 breaks it down by phase of the run.

The contrast in Table 4.2 is the result, and it is a contrast in shape rather than in magnitude alone. In the dark the loss rate decays throughout, from 41.9 to 3.9 pixels per frame, and has not levelled off when the 500 frames run out: most of those early losses were never damage but an immature threshold firing repeatedly on the same pixels, and what remains is still falling. With the beam on the rate does the opposite. It drops for the first two hundred frames, as the model adapts to a regime it was not built in, and then settles at roughly 15 pixels per frame and stays there for the remaining 1300 — flat, not decaying.

A rate that decays means the model is still converging, and a flat one means a process that does not stop. The fit of Figure 4.10(c) predicts a beam-independent part of 6.48 pixels per block, which over 2166 blocks and 1500 frames is 9.4 per frame. That is the same order as the mature dark rate of 3.9 per frame but a factor of 2.4 above it. Both count retired pixels, so what separates them is not the quantity but how each was obtained: one is a direct count at the end of the dark run, the other an extrapolation of a regression back to zero tracks per block, where

Frames	**Pixels lost per frame**	**Regime**
dark, 1–50	41.9 ± 0.9	model still converging
dark, 51–100	29.0 ± 0.8	
dark, 101–200	17.7 ± 0.4	
dark, 201–350	8.9 ± 0.2	
dark, 351–500	$\mathbf{3.9 \pm 0.2}$	still falling at the end of the run
beam-on, 501–700	75.5 ± 0.6	model adapting to the new regime
beam-on, 701–1000	15.5 ± 0.2	
beam-on, 1001–1300	15.1 ± 0.2	
beam-on, 1301–1600	14.7 ± 0.2	
beam-on, 1601–2000	$\mathbf{15.4 \pm 0.2}$	steady, to the last frame

Table 4.2: Rate at which pixels are retired into the dead mask, by phase of the run, over the full 500 dark frames and the full 1500 beam-on frames. The two behave differently in kind and not only in size: in the dark the rate decays monotonically by a factor of eleven and is still falling when the run ends, whereas with the beam it settles at some 15 pixels per frame by frame 700 and is unchanged 1300 frames later.

nothing was measured, and a single straight line over the whole sensor is too crude to carry it there. The two are not identified here. The agreement is in the order of magnitude only.

Against the 39.5 neutrons counted per frame, that is, 0.59 pixels retired per detected neutron. The figure is an upper bound in one direction and a lower bound in the other: far more neutrons reach the converter than are detected, which lowers the per-neutron cost, while a pixel is only retired after firing on five consecutive frames, which means slow degradation goes unrecorded. It is an order of magnitude, not a cross-section.

Where they fall. The losses are not scattered at random over the sensor. Figure 4.10 puts the 34 850 pixels retired during the beam-on run beside the arrival map of the tracks classified as neutrons over the same frames, binned identically. The two carry the same broad, centred profile, and their block-by-block correlation is $r = 0.493 \pm 0.016$. Pixels are lost where the beam illuminates the sensor.

Panel (c) makes the association quantitative rather than visual. A straight line fitted block by block has a slope of 0.35 pixels lost per detected neutron track and an intercept of 6.5 pixels per block — that is, a block that records no track at all still loses six or seven pixels over the run. Against a mean loss of 16.1 pixels per block, two fifths of the damage is independent of the beam and three fifths scale with it.

A test on the raw positions, rather than on smoothed maps, agrees. Rasterising the bounding box of every cluster classified as a neutron over the 1500 frames covers 4.87 % of the sensor; 33.07 ± 0.25 % of the pixels retired during the run fall inside that footprint. A pixel that dies is therefore 6.79 ± 0.05 times more likely than chance to have sat under a recorded neutron track.

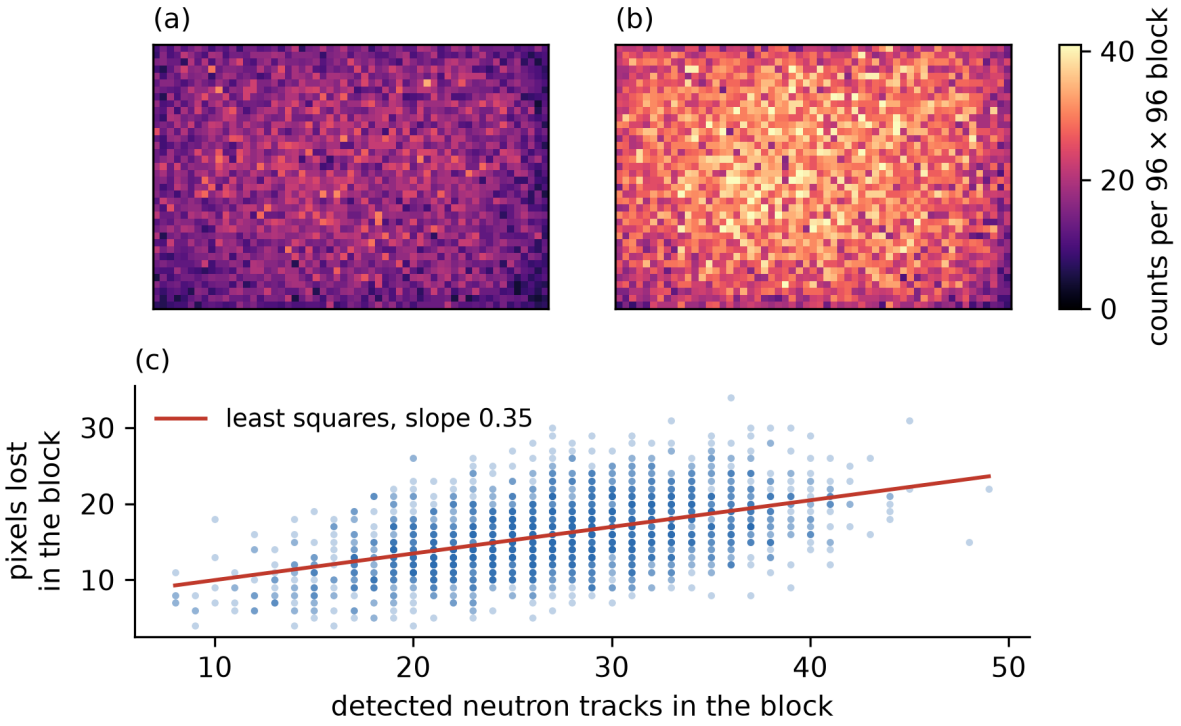


Figure 4.10: (a) the pixels retired into the dead mask during the 1500 beam-on frames, summed into 96×96-pixel blocks; (b) the neutron tracks detected over the same frames, binned identically. Both panels share one colour scale, set from panel (b) and clipped at its 99th percentile, so the two can be compared directly — fewer pixels are lost than tracks are recorded, and the losses carry the same illumination profile. (c) the same data block by block: each point is one of the 2166 whole blocks the sensor divides into, the line is a least-squares fit of slope 0.35 and intercept 6.5, and the Pearson correlation is 0.49.

Source: `make_deadpixel_figures.py`.

What they were doing beforehand. Reading the stored frames back for a sample of those pixels gives Figure 4.11, and it does not show what one might expect. The pixels that end up retired are not ordinary pixels struck once and destroyed: on the *first* beam-on frame they already read some 13.4 ADU, against 0.06 ADU for a control sample of 250 pixels that survive the run, and over the run their mean drifts up to 21.2 ADU while the control stays where it started.

The control is drawn uniformly over the sensor, not matched to where the retired pixels sit, and the retired ones sit in the beam spot. Part of the gap between 13.4 and 0.06 ADU is therefore position rather than history: a pixel inside the illuminated area reads higher than one at the edge whatever its condition. A control drawn from the same blocks, and only from pixels that survive, would separate the two and has not been run.

That first frame already has the beam on it, so it cannot say whether these pixels were anomalous beforehand. Reading the *dark* run back for the same two samples answers that, and the answer is yes: with the beam closed, the pixels that will later be retired average 8.85 ADU against 0.037 ADU for the survivors, a factor of 237, and they read non-zero on 27.6 % of dark

frames against 1.2 %. Both medians are zero. These are not permanently bright pixels — the dark mask would already have caught those — but intermittently firing ones, flickering often enough to stand out and not often enough to trip a rule that asks for five consecutive frames.

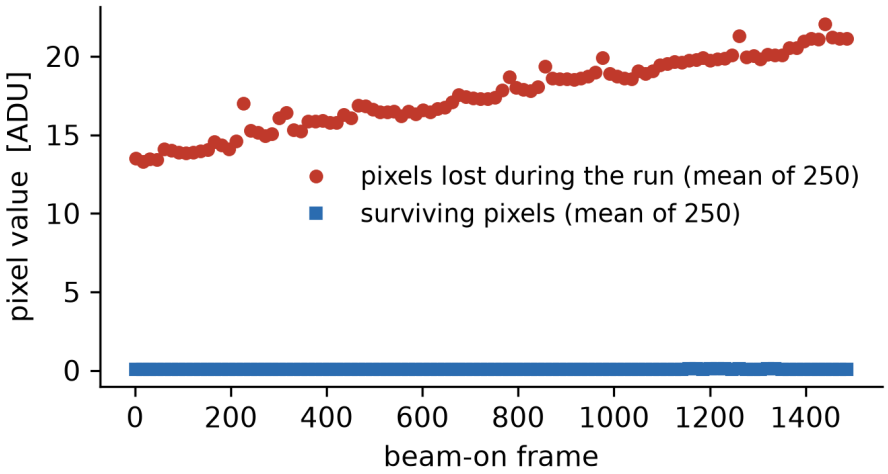


Figure 4.11: Mean pixel value against beam-on frame, for pixels retired during the run and for a control sample that survives it, sampled every fifteenth frame. Points are not joined: the frames between two samples were not read.

Source: `make_deadpixel_figures.py`.

Putting the three measurements together gives a mechanism rather than a correlation. There is a pre-existing population of intermittently firing pixels, distributed over the whole sensor, which is retired slowly and at a decaying rate even with the beam closed; the intercept of 6.5 pixels per block, worth 9.4 per frame, is that population, of the same order as the 3.9 per frame the dark run gives directly. The beam raises their firing rate where it illuminates, until each in turn trips the rule — which is the slope of 0.35, the enrichment of $\times 6.8$ inside track footprints, and the drift from 13.4 to 21.2 ADU over the run.

What this is *not* is one neutron destroying one pixel. A pixel that dies was already marginal before the beam existed, and the measurements are consistent with the beam pushing it over the threshold; no controlled irradiation was performed, so that remains a reading of the correlation rather than a demonstrated cause. The per-neutron figure quoted above should therefore be read as a bookkeeping rate for planning a run, not as a probability of destruction.

Charge sharing. A track might deposit enough charge in one place that the eight pixels around the impact are damaged with it. The geometry of the losses answers this directly. Of the 34 850 pixels retired during the run, 99.1 % are isolated single pixels, and the largest connected clump anywhere in the sensor is four pixels — against the 10 to 20 pixels a track actually covers. If charge spreading were killing neighbourhoods, the losses would arrive in clumps the size of a track, and they do not.

There is nonetheless a small, real excess of adjacent pairs. 1.78 ± 0.07 % of the lost pixels have another lost pixel among their eight neighbours, against 1.35 ± 0.06 % for a null model that reshuffles the same losses inside each 96×96 block — a null that preserves the beam profile exactly

and destroys only pixel-to-pixel structure. The excess of $\times 1.32$ is therefore not the illumination pattern in disguise, but it accounts for 149 pixels out of thirty-four thousand. Neighbour effects exist, and at a hundred and fifty pixels they are too small to be the mechanism.

What this still does not settle. The retirement rule is a detection criterion, not a physical one: a pixel that degrades slowly without ever firing five frames running is never counted, so these numbers are a lower bound on damage. And nothing here separates damage to the silicon from damage to the boron converter above it. All three are analyses of data already recorded, or short dedicated runs, rather than new physics.

What it does to the rate. Losing 34 850 pixels inside the beam spot during a run raises the question the experiment actually cares about: whether the counted rate drifts while the sensor degrades. It is answered by splitting the run in three. The lithium rate is 40.12 ± 0.26 per frame over the first five hundred frames, 38.99 ± 0.29 over the middle five hundred and 39.40 ± 0.28 over the last. The middle third is the lowest of the three, so there is no monotonic decline to attribute to the damage, and the difference between the first and last thirds is 0.72 ± 0.38, or $1.9\,\sigma$. The spreads in this paragraph are standard errors of the mean over the frames of each third rather than the Poisson error of Eq. (A.1), because the frame-to-frame scatter carries the beam as well as the counting; on the Poisson rule the same difference is $1.8\,\sigma$, and neither is significant. A straight line fitted to all 1500 frames falls by 3.1 % across the run, which is the largest effect the data will support and is still not significant. For a Ramsey scan, whose observable is the ratio of counts between settings rather than their absolute value, a drift of this size and this significance is not a limitation yet. It would become one over a longer run, and measuring it again after a full beam time is the way to find out.

4.8. Comparison with CR39-based Detection

The closest published counterpart to this work is the CR39-based track detection of [6], developed in the same group for the same neutron-capture reaction. Both convert the neutron in a ^{10}B layer and both classify the resulting objects with a random forest, which makes the two directly comparable; what differs is everything between the capture and the classifier.

Where the systematics live. In the CR39 the systematics are *static and in the medium*: a crack in the boron coating or a grain of dust is etched along with the tracks, is re-scanned identically every time, and cannot be subtracted. Figure 8.13 of [6] shows this directly — the candidate set is full of cracks — and it is why a buffer class (Candidate) and a human pass over the neutron-classified tracks are both necessary there.

A crack is long and a track is round, so a cut on elongation might be expected to replace that human pass. It does not, and [6] says why in a footnote: a large textured object is *broken up into smaller pieces by the binarising threshold.* What reaches the classifier is therefore not one long crack but a chain of fragments, each about the size of a track and none of them elongated. A cut on size fails for the same reason, and so does a cut on compactness. The information that would separate them — that the fragments lie along a line, and that the line runs off the edge of the field of view — is contextual rather than per-object, and no per-object cut can see it. That is the structural reason the CR39 route needs a classifier trained on appearance and then a human check, where the CMOS route subtracts the problem before it arises.

None of this arises here, and the reason lies in the measurement rather than in the analysis. A CR39 scan is a *photograph of a medium* that has accumulated damage: a track and a crack are both dark features in the same greyscale image, distinguishable only by shape, and the only way to turn that image into objects is to threshold it — which is what shatters the crack. A CMOS frame is not a photograph of a medium. It is a measurement of the charge collected during one exposure, and a crack in the boron coating deposits no charge. It is not a faint feature that has to be separated from the tracks; it is simply not in the measurement. There is therefore nothing extended to fragment, and no contextual information is needed to recognise something that never became an object.

The second half of the reason is that a CMOS frame is one of many. Anything static — a hot pixel, a scratch on the window, a defect in the coating — contributes the same value in every frame, so it lands in the per-pixel mean and is subtracted with it. A CR39 scan is a single image; there is no ensemble to average over, and nothing static can be removed by averaging.

On the CMOS the systematics are *dynamic and modelled*: the background is measured per pixel from dark frames and subtracted from every frame, so anything that repeats is either absorbed into the model or retired into the dead mask before it reaches the classifier. The pixels that matter here fire intermittently rather than permanently, which is why the retirement rule counts five consecutive frames and not one. Section 4.4 measures what that difference costs and buys: across the 1 447 466 clusters that the unfiltered pass of Section 4.4 finds in 499 beam-off frames, not one was classified as a neutron, and the highest neutron probability assigned to any of them was 0.0196. There was therefore no neutron-classified track to check by hand, whereas [6] checked its own one by one. The credit for that belongs to what a beam-off frame contains rather than to the classifier (Section 4.4): on single-pixel objects the classifier is extrapolating outside anything it was trained on. Its Figure 8.17(b) (p. 160) puts that rate at $1.1 \pm 0.5\,\%$ of the objects called neutrons, and its Figure 8.18 (p. 162) reports 63 such cases on a single detector.

What it costs. Stability runs the other way. A CR39 detector integrates without electronics and has no pixels to lose, while this sensor degrades as it measures. Between the model built on the 500 dark frames and the model after the 1500 beam-on frames, the dead-pixel mask grew from 7075 to 41 925 entries: 34 850 pixels were lost during a single run (Section 4.7). The CR39 method costs human time and gives stability back. This one is the other way round.

On the numbers themselves. Ref. [6] reports a neutron-category accuracy of $99.32 \pm 0.32\,\%$ against the $95.75 \pm 2.20\,\%$ of Figure 4.6, and the two are not comparable as they stand.

Both are the same quantity, $(\mathrm{TP} + \mathrm{TN})/N$ over the neutron category. That quantity depends on how common neutrons are in the set it is measured on, because the true negatives sit in its numerator. The set used here is deliberately balanced, with neutrons at 39 %, so a classifier that answered "not a neutron" to everything would already score 61.0 %. On that set the score of 95.75 % closes $89.1 \pm 5.6\,\%$ of the room between the floor and a perfect answer.

The same normalisation cannot be applied to [6] with any precision, because that thesis does not state how common neutrons are in the set its accuracy is measured on. Its detector was not directly exposed, which puts the fraction low, but no number is given. What can be said is how little the answer depends on it. Taking the neutron fraction anywhere between 5 and 30 %, the room closed there runs from 86.4 ± 6.4 to $97.7 \pm 1.1\,\%$, and the difference from the 89.1 % measured here runs from -0.3 to $+1.5$ standard deviations. At no point in that range are the

two distinguishable. The comparison is inconclusive whichever fraction is assumed, and saying so does not depend on recovering a number the source never printed.

Neither figure is a single number. Both are the mean of 1000 retrainings, which makes a comparison possible at all: $95.75 \pm 2.20\,\%$ here and $99.32 \pm 0.32\,\%$ in [6]. Both spreads are narrower than the quantity a reader will take them for. Each of the 1000 runs here re-splits the same 200 hand-labelled clusters into a fresh training and test set, so the spread measures how much the answer depends on which two-thirds are used to train, and on the randomness of the forest. It does not contain the sampling error of the 200 labels themselves, nor any error in the labels. A second set of 200 clusters, labelled by the same hand, would move the mean by more than ± 2.20. Propagating them through the normalisation above (Appendix A) gives the range quoted there. Over that whole range the largest term that *is* quantified is the ± 2.20 measured here, the spread of one classifier retrained on one training set, and it alone is larger than any difference between the two methods. The label sampling error above it is not quantified and would only widen the interval.

What *is* measurable is where the residual error sits, and Table 4.1 places it on the `gamma`/`artifact` boundary rather than on anything involving neutrons. Two things would move that boundary. One is the buffer class of [6], which is not adopted here for the reason given above. The other is available to this pipeline and not to his: a `gamma` and an `artifact` are both single pixels and are therefore nearly degenerate in the ten geometric features, but they differ in *time* — an artifact is a property of the sensor and recurs, a gamma happens once. The detector already carries the per-pixel history that would express this, in the same accumulators that drive the dead-pixel mask, and it is not currently offered to the classifier as a feature. A CR39 scan is a single static image and has no such quantity to offer. On the test that *is* like for like — a beam-off dataset — the false-positive count here is zero out of 1.45 million objects. What is genuinely weaker is the training set: roughly 4000 hand-classified tracks there against 200 here.

One quantity is missing from the comparison altogether. Neither method's *detection* efficiency is measured here: what fraction of the neutrons that reach the converter ends up as a counted track. That depends on the boron layer and on the collection of the reaction products, not on the classifier, and comparing the two detectors on it would take a source of known activity in front of each. Everything above compares how well the two sort what they already found.

No attempt was made to close that gap by changing the classifier. The comparison is reported as it stands, with the difference in test populations named, because the number this experiment depends on — the false-neutron count on beam-off data — is an observed zero, and no accuracy on the labelled set can lower a count that is already at the floor of what can be counted. Raising the cross-validated accuracy on a balanced labelled set would sharpen a number the beam-off floor does not depend on. It would not leave the thesis untouched: Section 5 uses the purity and efficiency measured on that same labelled set to put a $-4\,\%$ systematic on the counted rate, and those two would move with it.

The feature composition. Section 9.4.1 of [6] proposes, as future work, investigating whether a different feature composition would classify more robustly than the raw 30×30 pixel patches it uses. That is what this pipeline does: it classifies on ten measured physical descriptors — shape, photometry and radial profile — rather than on 900 raw pixel values, which is also why its decisions can be read and audited one column at a time.

5. Conclusion and Outlook

Two problems came out of checking numbers against the files that carried them, and neither would have shown up in the physics. They are operational rather than systematic; the two systematic effects this thesis measures are the sigma-tail excess of Section 4.6 and the pixel losses of Section 4.7.

The acquisition ran slower than requested. The 2026-07-11 folders are named `..._2fps`, which asks for a 500 ms frame period. The file timestamps give a median interval of 555 ms, that is 1.80 fps: a 10 % shortfall that nothing reported at the time. The cause is that the camera cannot begin a new exposure before the previous one has been read out, so once the requested period falls below exposure + readout the sensor silently caps the rate at what it can sustain. The folder name also records a 575.6 ms exposure, which is longer than the interval actually measured between frames; the two cannot both be literally true, and which of them is wrong is not established here. Nothing measured is wrong because of it, since every rate in the pipeline divides by real elapsed time, but a run planned to last a given time came back 10 % short of the frames that rate implies. The frames themselves are all there. The interface now displays the requested and achieved rates side by side and warns beyond a 5 % shortfall.

The pipeline has no margin on Windows. Section 3.6 measures the cost of a frame on all three platforms. Of that cost 99.7 % sits on a single sequential thread — the background model update plus the snapshot handed to the detector — so no number of loader or detector threads can take the total below that floor. They can raise it, and they do. Twenty combinations of loader and detector threads, from one of each up to eight and twelve, span 322–535 ms per frame, a factor of 1.7 that is entirely contention: the fastest is a single thread of each, at 322 ± 2 ms, and the two-and-four of Table 3.2 is the configuration the laboratory runs and is not the best one. Adding threads to a sequential problem buys nothing and costs something.

That scan was run on the Linux machine and its numbers do not transfer to the Windows one, where the sequential section is half again as long. Whether retuning the thread counts under Windows would recover enough to clear the frame period is not measured here, and it is the cheapest of the things left to try. All twenty returned the same 39 071 clusters, which is a stronger statement of determinism than the three-platform comparison: it holds across the scheduling as well as across the compiler. That is Amdahl's law, not a mis-tuning: the pipeline waits only 0.2 ms per frame for the loaders once the frames are in the operating system's cache, so decoding is already hidden entirely behind the sequential work, which is negligible. The sequential section divides into the background update, 442.3 ± 10.0 ms, and the snapshot handed to the detector, 117.3 ± 3.8 ms, itself the mean clone at 61.1 ± 1.9 ms and the derivation of σ at 56.2 ± 2.1 ms. Removing a redundant 80 MB allocate-and-copy from that section was worth a measured gain at the time it was made, on a build no longer on disk; the figures quoted here are from the current one.

What matters for a live run is the margin, and there is none. Ten repeats on 2026-08-26 give 559.9 ± 13.5 ms of sequential work against the 557 ms the camera leaves between frames, so the detector is $0.2\,\sigma$ over the period and drops 0.5 % of frames, and only while the frames are already cached. Section 3.6 gives the cold figure and what it costs.

That conclusion is about the operating system as much as the hardware. The same laptop, the same frames and the same flags, built with GCC under Linux instead of MSVC under Windows,

does the work in 371.7 ± 58.3 ms: it keeps up with a third of the period to spare, and its slowest repeat still came in under the Windows mean. The recommendation follows without any purchase. Why the same source is half again as fast under one toolchain as the other is not something these measurements answer, and it would be worth knowing.

The recommendation has a boundary, and it should be stated. Every live acquisition in this thesis was taken under Windows; neither the camera nor the IDS peak SDK has ever been run under Linux. What the Linux machine measured is the processing of frames already on disk, which is the whole of the offline path and the whole of the timing argument, but not the part that talks to the camera. Moving the live path across needs the vendor's Linux SDK and one beam-off session to confirm it, and until that is done the margin quoted here is a margin on the analysis rather than on the acquisition.

How it compares. The closest published counterpart is the CR39 pipeline of [6], built in the same group for the same capture reaction. On the like-for-like test — a dataset taken with the beam off — it put its false-neutron rate at 1.1 ± 0.5 % of the tracks it called neutrons; the pipeline here classified none of its beam-off objects as a neutron at all, on either the filtered pass or the unfiltered one. The two rates are not measured over the same population and should not be divided into one another. On classifier accuracy the two are not distinguishable. Normalising each score against the do-nothing baseline of its own test set closes 89.1 ± 5.6 % of the available room here, and between 86 and 98 % there depending on a neutron fraction that [6] does not state. Across that whole range the two differ by less than two standard deviations, so the measurement is compatible with no difference at all, and calling either method the better one would go beyond what was measured. Sharpening it is a matter of labelling more clusters, not of changing the classifier (Section 4.8).

What was built. A detection pipeline that takes raw CMOS frames and returns classified particle candidates: a C++ core that models the sensor noise per pixel and finds clusters against it, a Python layer for labelling and machine-learning classification, a graphical front end that drives both, and an offline deployment kit for a lab machine with no internet access.

Measured rate. The 1500 beam-on frames of 2026-07-11 yield 59 259 clusters classified as neutron captures over 833.1 s of elapsed acquisition, that is 39.5 per frame and

$$R = 71.1 \pm 0.3\,\mathrm{s}^{-1}, \tag{5.1}$$

the uncertainty being the Poisson error on the number of counts, $\sqrt{59\,259} = 243$, or 0.4 %. Poisson is the right statistic here because each capture is an independent event in a fixed observation window, which is what the counting statistics of a detector are. The frame timing contributes nothing because the rate divides by measured elapsed time rather than by a nominal frame rate: the timestamps give a median interval of 555 ms with a spread of 11 ms and no gap longer than a second anywhere in the run.

Three systematics are larger than the statistics, and none of them is measured here. The classifier's purity on the neutron class is 0.926 and its efficiency 0.962 on the labelled set, which if carried over to the run would move the count by some -4 %; that transfer is itself an assumption, since the labelled set is balanced and the run is not. The temporal coverage is not established: the exposure is known only from the folder name, 575.6 ms, which is longer than the 555 ms

interval measured between frames, so the two cannot both be right and any neutron arriving outside the integrating window is never counted. And the figure counts *detected* captures — the conversion efficiency of the boron layer and the collection efficiency of the sensor are not part of it, so the arriving flux is higher by a factor this thesis does not determine. The rate should therefore be read as a well-measured detector count rate, not as a beam flux.

Measured performance. On Apple Silicon the background pass runs at 169.3 ± 5.3 ms per 20.1 Mpixel image, about 119 Mpixel/s, over ten repeats. On the Windows laptop used for comparison the same binary and the same 500 frames give 561.4 ± 13.7 ms per image over ten repeats, roughly 3.3× slower and consistent with memory-bandwidth-bound work on a 15 W mobile part. Both platforms returned 39 071 clusters, bit-identical despite six worker threads, which is the check that most directly shows the pipeline to be deterministic.

Outlook. Everything reported above was extracted from data already recorded. What follows needs beam time. Each item states what would be measured and what it would settle.

An acquisition at reduced gain, and the four lines it would resolve. At the gain used, 99.5 % of tracks above five pixels have at least one clipped pixel, so `total_charge` is a lower bound and the deposited-energy spectrum cannot be read (Section 4.5). The information was lost in the sensor, before the software saw it, and no re-analysis recovers it. A single short run with the gain tuned down until the saturated fraction falls to a few percent would put the spectrum within reach — and what is within reach is specific: Eqs. (2.5)–(2.6) predict *four* lines, the lithium and α of each branch, the second branch about a fifth higher and some sixteen times rarer. Resolving them is the difference between counting captures and identifying which daughter was recorded.

A sensor where the absorber is, and the state structure it would show. The scattering absorbers of Section 2.4 prepare and analyse the states by destroying what they touch: a rough surface held at a given height removes the neutrons that reach it, so what passes is a known mixture of states and what is counted downstream is a transmission. The height information is used and then thrown away, because nothing at that plane records *where* a neutron was when it was removed. A pixel sensor put at the same plane would record exactly that. Instead of a transmission it would return the vertical density $|\psi(z)|^2$ directly, and a transition between states would appear as a change in the shape of that profile, not as a change in one number.

Whether the sensor of this thesis could resolve it is a question of scale, and the scales are measured. The states are structures tens of micrometres tall, with the first two classical turning points at 13.7 µm and 24.0 µm, so the closest pair is 10.3 µm apart. A capture deposits its charge along the path of the daughter ion, and the clusters are around 12 pixels, an equivalent diameter near 9.4 µm. Comparing those two numbers would suggest the states are unresolvable, and it would be the wrong comparison: what has to be located is not the extent of a track but its centre, which the detector already computes for every cluster. The second moments of Section 3.3 give a track an RMS width of 1.86 µm over about 12 pixels, so its centroid is fixed to some 0.5 µm, twenty times finer than the spacing to be resolved. A wide track is not a blurred position; it is many samples of one.

What does limit the position is the physics of the conversion. The neutron is captured in the boron layer and the daughter then travels a few micrometres in a direction nobody records, so the centre of charge sits a micrometre or two from where the neutron arrived. That displacement is not noise to be averaged away per event, but across many events it is a fixed blur of a couple of micrometres against a structure of ten, and a convolution is not a limit.

Reducing the gain would help here too, and for the same reason it helps the energy axis. A smaller cluster is a tighter estimate of its own centre, and one acquisition would improve the position and open the spectrum at once.

Figure 5.1 shows what that sensor would return. Panel (a) gives the density of the first three states; panel (b) is the same after each capture has been displaced by the flight of its daughter, taken at 2 µm. The nodes survive. That is the result: a structure whose finest feature is a few micrometres is not erased by a blur of the same order, it is only softened, and the profile stays recognisably that of one state rather than another.

The same statement can be made scale by scale, since a Gaussian displacement is a low-pass filter. It passes about four fifths of the contrast at 20 µm, close to half of it at the 10.3 µm that separate the first two states, a fifth at 7 µm, and almost nothing below 5 µm. The level structure therefore arrives with roughly half its contrast intact, while anything finer than the daughter's own range is gone, and gone for good: no processing recreates what was never recorded.

The blur also settles how the profile should be read. The densities are known functions, so nothing has to be inverted: the recorded profile is a mixture of them and the only unknowns are the weights. Fitting those weights returns the population of every state at once, each with its own error bar — which is what a transmission measurement cannot do, one absorber height giving one number. Panels (c) and (d) of Figure 5.1 carry that out on simulated data. Six hundred and sixty neutrons are drawn from a mixture of the first five states and binned at the pixel pitch; the fit returns 371 ± 20 neutrons in $n = 1$ against the 396 injected, 188 ± 26 against 165 in $n = 2$ and 60 ± 22 against 66 in $n = 3$, while the two thinly populated states above come back with errors as large as themselves. The low states are read out and the high ones are not, and the figure says which is which instead of leaving it to be assumed. Those intervals are the whole claim, so the script measures them rather than asserting them: over two hundred independent draws they cover the injected value between 64 % and 71 % of the time against the 68 % they claim, which two hundred draws themselves pin down only to about three points either way.

What it would cost is a smaller number than the transmission measurement it replaces. Between $n = 1$ and $n = 2$ the mean height moves from 9.2 µm to 16.0 µm, a shift of 6.9 µm on a profile about 5.9 µm wide. Detecting a transfer of a fraction f of the population at three standard deviations needs $N = (3\sigma / f\,\Delta z)^2$ events: seventy for a transfer of 30 %, some six hundred and sixty for 10 %, twenty-six hundred for 5 %. At the 71.1/s this detector counts, those are one second, ten seconds and forty seconds of beam.

Put beside the transmission method at the same significance, that is a gain of under three. Detecting the same fractional change in a rate needs $2(3/f)^2$ counts, the factor of two because a change is the difference of two measurements: eighteen hundred against the six hundred and sixty above. The advantage is real and it is modest, and the larger figures that follow come from how a scan is scheduled rather than from the profile itself.

One systematic deserves naming, because it is the one that would manufacture the effect being looked for. The fit needs to know where the mirror surface is: both densities are anchored at $z = 0$, and the estimate above holds that anchor fixed. Displacing it by a micrometre moves the fitted fraction by 0.06, twice the statistical error, and by two micrometres it moves it by 0.13. A mirror position known only to a micrometre would therefore be indistinguishable from a population transfer of six percent. The remedy is cheap — fitting the anchor alongside the fraction costs about 0.002 of precision — and it has to be done, which is the sort of thing worth knowing before the beam time and not after.

None of this is settled here. It is a calculation of what the geometry allows, on one assumption

— the displacement — that a dedicated measurement would replace. It says the idea is worth a shift with an absorber removed, and roughly what that shift would buy.

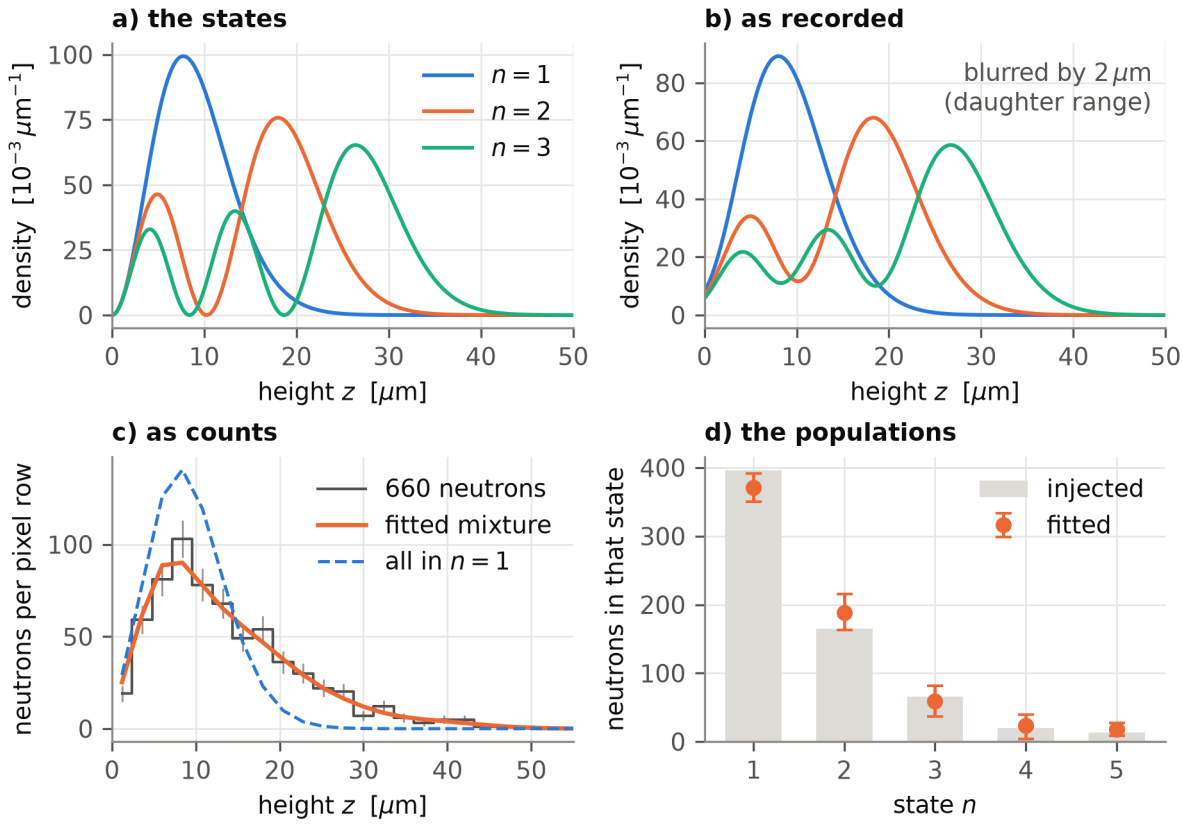


Figure 5.1: What a pixel sensor at the analysing plane would record. (a) the vertical density of the first three gravitational states. (b) the same after every capture has been displaced by the flight of its daughter in the silicon, taken at $2\,\mu\mathrm{m}$: the nodes soften but survive. (c) 660 neutrons drawn from a mixture of the first five states, binned at the $2.4\,\mu\mathrm{m}$ pitch with their counting errors, and the fitted mixture through them; the dashed curve is what the same neutrons would give with all of them in $n = 1$. (d) the populations that fit returns, against the ones injected. The error bars are 68 % intervals from four hundred resampled fits, and $n = 1$ falls just outside its own, as about one point in three should. The displacement is the only assumption in the figure; the densities are Airy functions.

Source: `make_absorber_cmos.py`, seed 3.

A feedback loop on the transition frequency. The reason to want that image live, and not afterwards, is arithmetic. A resonance scan needs the transmission at each of some thirty vibration frequencies. One percent of statistics at a point is 10^4 counts, and at the 71.1/s this detector counts that is 140.6 s per point and 4219 s for the scan — an hour and a quarter.

Neither the one percent nor the thirty frequencies is quoted from anywhere; both are working conventions, and it is worth saying what they cost. What fixes the per-point precision in practice is the precision wanted on ν_{12} at the end. At one percent per point a single scan of this geometry places a fringe to about 0.4 Hz, so the 0.05 Hz that Section 2.5 quotes for the method takes

some sixty scans averaged, which is where the seconds below turn into days of beam. A tighter per-point target buys a proportionally better scan and costs the square of it, and the ladder that follows is not indifferent to the choice: at one percent it ends at 9.2, at three percent at 6.4, because the survey is priced by the abandon rule and not by the fine grid. Today the shape of the curve is known when the shift is over. The pipeline processes a frame in 169 ms against the 555 ms between frames, so every cluster of a frame is classified before the next one arrives and the curve can be drawn as it accumulates. That margin is the Apple Silicon figure; the laboratory's own Windows machine has none, and the Linux build that would give it one has never been connected to the camera (Section 3.6). Closing that gap is the prerequisite for everything in this paragraph, and it is a matter of installation rather than of design. An operator would see a resonance form, and could stop a point that has converged or abandon a working point that is not producing contrast, instead of discovering either the following morning. That is the argument for the live path of Section 3.7 being real time and not merely fast, and it is the one thing in this list that needs no new hardware at all.

`Code/src/ramsey_feedback.py` puts numbers on it. The program simulates a scan around ν_{12} with the sensor in place of the absorber, fits the population fraction after every frame, and compares four ways of spending the same beam. Every one of them is made to reach the same error on the fraction, one percent per point, which is the precision the laboratory already works to. That choice does two things: it stops the comparison counting a choice of precision as though it were a property of a detector, and it makes the first rung the scan as it is run today. All four count neutrons at the 71.1/s measured in Section 4.3, this detector's own rate on this beam, so the ladder compares detectors at equal beam. A fixed dwell is one duration for every frequency, so on both sides it is sized for the frequency that needs the most counts: off resonance, where the absorber transmits everything and its error on the fraction is exactly $1/\sqrt{N}$.

Averaged over three hundred scans at a 30 % transfer, an absorber at fixed dwell needs 4219 s: the same scan, and the same number, as page 62. It is not an independent check — both sides size the dwell from $1/\sigma^2$ counts at the measured rate — but it does fix the first rung of the ladder to the shift the laboratory already runs. The sensor at the same fixed dwell needs 3098 s; stopping each point on its own error bar brings that to 1270 s; and surveying coarsely before zooming brings it to 459 s, eight minutes. End to end that is a factor of 9.2, of which only 1.4 is the profile against the rate. That 1.4 and the 2.7 of the previous section are the same comparison in two different experiments, and the factor of two between them is the reference: an isolated detection has to measure the unperturbed rate as well, while a thirty-point scan reads it off its own wings for nothing. Figure 5.2 shows one such scan and the ladder beside it.

The live stop is nonetheless something only the profile can do, and the reason is worth spelling out. A counter measures the transferred fraction to $1/\sqrt{N}$ and to nothing else: its precision depends on how many neutrons have arrived, and on nothing about the frequency it is sitting at. An operator watching a counter in real time therefore sees only what could have been written down before the shift — the error bar will reach the target at a number of counts that was known in advance — and there is no decision to take. A profile fit is not like that. Its precision depends on the shape it is fitting, so how long a frequency takes is not known until it is measured, and some frequencies reach the target well before others. Watching then tells the operator something they could not have known, and stopping early becomes a decision rather than bookkeeping.

The target is what makes the ladder honest, and it is worth saying what happens without it. An earlier version of this comparison ran the three sensor rungs at an error of 0.03 on the fraction while leaving the absorber at the one percent the laboratory actually uses, and the ratio came

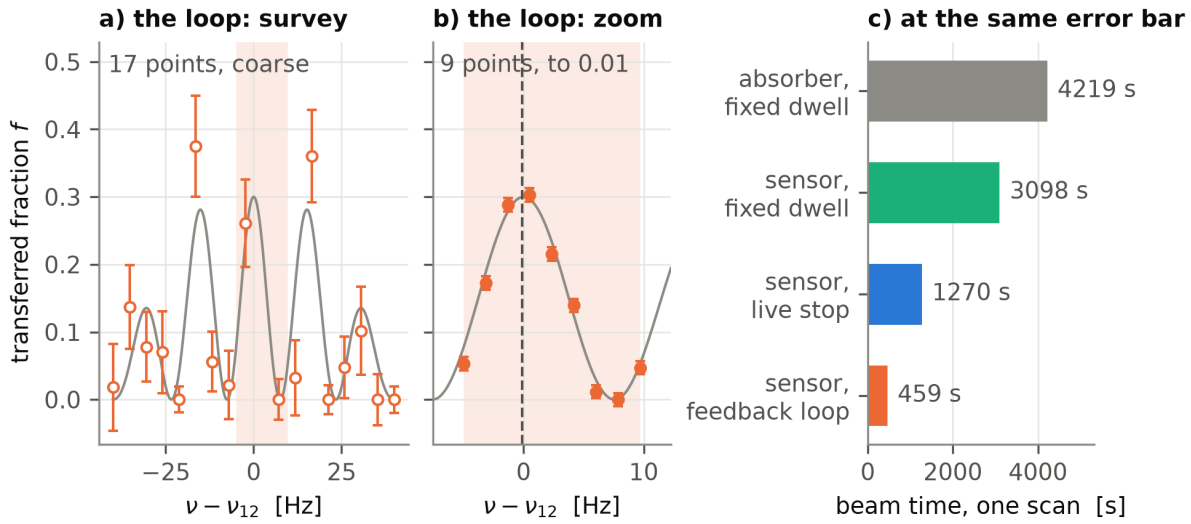


Figure 5.2: What a feedback loop does with a shift, on one simulated scan around ν_{12} at a $30\,\%$ transfer. Panels (a) and (b) are the two stages of a single method — the bottom rung of (c) — which is why they share its colour and differ only in the fill of their markers. The other three rungs are not drawn. (a) the survey: seventeen frequencies across the whole range, each measured only well enough to answer whether there is contrast and roughly where, which is why the error bars are large — they are the point of the stage, not a defect of it. The shaded band is the window it selects — not simply the tallest sample, but the crest inside the group of fringes picked out by a running mean one fringe wide, which is why the single highest point of the survey need not be in it. (b) the zoom, the same window at four times the horizontal scale: nine frequencies inside one fringe, each measured to the target error of one percent, and the dashed line is the fringe centre they return, here $0.14\,\mathrm{Hz}$ from the true one. The two stages spend comparable beam; what differs is that the second spends it where the first found something. The scan drawn is one of those that locked onto the central fringe, since the text takes the fringe order as known, and among those the one whose residual is closest to their median. (c) the beam time one scan costs on each rung, averaged over three hundred scans, every rung reaching the same error on the fraction.

Source: `make_ramsey_loop.py`, which imports `Code/src/ramsey_feedback.py`.

out at fifty-eight. Nine of that fifty-eight was nothing but the coarser target. Setting every rung to one percent removes the temptation, and it costs the argument nothing: the loop is better off at the finer target, not worse, because the survey is priced by what it takes to reject a dead working point and not by the precision of the fine grid. Its cost is fixed while everything around it scales as $1/\sigma^2$.

What that buys at the end of a campaign is the number worth carrying. One scan places a fringe to $0.42\,\mathrm{Hz}$ with the absorber and $0.39\,\mathrm{Hz}$ with the loop, so reaching the $0.05\,\mathrm{Hz}$ of Section 2.5 takes some seventy scans one way and sixty the other: $83\,\mathrm{h}$ of beam against $8\,\mathrm{h}$. Those are hours in a simulation of this geometry and not a prediction for a campaign, but the ratio is the same 9.2, and it is the form in which it would be felt.

What the saving is worth depends on what is done with it. It is beam time and not precision: the four rungs all aim at the same error bar, so they all place the fringe to about the same 0.4 Hz, and the loop's 459 s buys no better a single scan than the absorber's 4219 s. At fixed shift length it converts, because the same hours then hold nine times as many scans and their centres average: a factor of 9.2 in time is a factor of 3.0 on the statistical error of ν_{12}. That conversion carries a condition, and it is the one named below — centres average only if they are centres of the same fringe, so ν_{12} has to be known beforehand well enough to fix the order. That is not a formality: left to itself the loop settles on the central fringe in 110 scans out of three hundred and the fixed grid in 107, so without prior knowledge of ν_{12} neither set of centres averages to anything. Given that knowledge, the gain is real, and it is a gain of three.

The same program is where the limits of the idea show, and they are milder at this target than at a coarser one. Chasing a 10 % transfer instead of 30 %, the loop still halves the beam time against the fixed grid, 319 s against 743 s, where at an error of 0.03 it saved nothing at all. Ten seconds of dead time to retune the piezo at each frequency costs it a third, 722 s instead of 459 s, and leaves the ladder at 6.3 rather than 9.2.

What does not improve is the fringe order. The first side fringe transfers 0.279 against 0.300 at the centre, a gap of 0.021 against error bars of 0.01 that the scan cannot resolve into a decision, and the loop lands on the central fringe in 110 scans of three hundred against the fixed grid's 107 — neither strategy settles it, and the difference between them is noise. The order has to come from knowing ν_{12} beforehand; what a scan measures is where a fringe sits. The loop also walks away from a real resonance twice in three hundred, which is the price of being allowed to walk away at all.

Tuning the gain from the live path. The lever for that already exists and has never been used against a beam. `ids_capture.py` carries an auto-tune search that raises and lowers the gain while measuring the saturated pixel count per frame, and writes the value it converges on into the energy preset. Running the pipeline live at the beamline, with that search closing the loop, would set the working point in minutes instead of by trial between campaigns.

Little of that is missing. The offline kit is already deployed on the Windows 10 machine that serves the detector at the ILL: the environment, the Python dependencies and the IDS drivers are installed, the C++ detector compiles there, the camera is recognised and delivers frames, and the graphical front end drives the sequence end to end. What remains are defects rather than missing pieces, and two were open at the last session.

The warm-up frame count is derived from the number of image files present in the dark folder rather than from the number of frames the run itself captured, so a folder holding images from an earlier session yields a warm-up longer than the acquisition that follows it. The software should count what it acquired, not what it finds.

The background-model step also terminated once with a Windows access violation, exit code `0xC0000005`, and that one is not yet diagnosed. It is a memory fault rather than a logic error, so it will be found by running the same step under a debugger on that machine rather than by reading the source.

Neither is architectural. What stands between the software as it is and a beam-steering acquisition is debugging on that machine, not development.

The part of the noise excess that is not accounted for. Removing the pixels the beam hit leaves the tail above what accumulation predicts on pixels no cluster ever occupied — by 47 % against the flatter extrapolation and 19 % against the steeper one, so how much is left to explain depends on an extrapolation the data do not pin down (Section 4.6). A sub-threshold contribution spread

over the sensor would do it, and the test is cheap: dilate the event mask by a pixel or two and re-run the same frames, which is a re-analysis rather than a new acquisition. It would settle whether what is left is a wider halo than this one measured or something that has nothing to do with tracks at all.

Separating α from lithium. The classifier has no `alpha` class because no labelled sample containing identified α particles exists, and the two cannot be told apart by eye in a cluster crop — they differ in range and in deposited energy, not in appearance at this pixel scale. A readable energy axis is therefore the prerequisite: with the four lines resolved, the labels can be assigned from the charge rather than by judgement, and the classifier can be retrained on a distinction that is physical instead of visual. That is one continuous piece of work — reduced gain, then energy calibration, then an α class — and it is the largest single thing still missing from the detector's physics.

A shielded measurement of the instrumental floor. The beam-off false-positive floor reported in Section 4.4 is zero, but it mixes sensor noise with genuine ambient radiation and cannot separate them by analysis. A run with the detector under cadmium would, and would turn an upper bound into a measurement.

More labels, for precision rather than for score. The spread quoted on the classifier, ± 2.20 points, is dominated by the size of the held-out set: sixty clusters, so accuracy can only take multiples of 1/60 and the retraining histogram comes out combed. That spread makes the comparison with [6] inconclusive, and it is the one thing more hand-labelled clusters would fix. The learning curve says they would not raise the score; they would sharpen the measurement of it. Since that spread falls as $1/\sqrt{N}$, halving the bar takes four times the labelled set.

A temporal feature for the classifier. The residual classification error sits almost entirely on the `gamma`/`artifact` boundary (Section 4.4), where both classes are single pixels and the ten geometric features can barely tell them apart. What separates them is not shape but recurrence: a sensor artifact is a property of that pixel and comes back, a gamma does not. The detector already maintains the per-pixel history that measures this — it is what drives the dead-pixel mask — and the classifier is simply never shown it. Adding one column, the number of times that pixel has fired in the preceding frames, is the single change most likely to move the number. It is also a quantity a static detector cannot have, which makes it worth reporting either way.

The counter costs little, and the obstacle is elsewhere. It already exists as a full-frame accumulator updated on every pass, so reading it for a cluster is a handful of lookups per cluster, against the 442 ms the background update itself spends on that frame — nothing that would threaten the live path. What does not work live is the *start* of a run: the feature is undefined until enough frames have gone by, so a live session would classify its first frames on a column that has no meaning yet. The sensible order is therefore to add it on the office path, where the whole run is available before anything is classified, measure what it is worth there, and only then decide whether it earns the special case the live path would need.

A better definition of a dead pixel. The rule used here retires a pixel after five consecutive threshold crossings, and the five was chosen rather than measured (Section 4.7). The measurements show why that matters: what the rule catches is an intermittently firing pixel, not one that has failed outright, and a run-length criterion describes intermittency badly — a pixel firing on nine frames out of ten but never twice in a row is never caught at all. A rate-based criterion over a sliding window would fit the observed behaviour better, and a scan over both the window and the threshold, scored against how many genuine tracks each choice discards, would turn the definition from a convention into a measurement. The neutron count barely moves with the

choice, the losses being 0.2 % of the sensor, but two things do: the number quoted as “pixels lost”, and the beam-off object count, which falls by a factor of nearly forty when the rule is active (Section 4.4). That is why the floor is reported on the pass with the rule switched off.

Throughput. The sequential stage bounds the run, and the persistent server mode that keeps the background model in memory across frames is the documented extension point for it. This one is software, and needs no beam.

A. Uncertainties

Almost every quantity in this thesis is a count, so almost every uncertainty is the counting statistics of the detector rather than a propagated instrument error. This appendix states which rule produced each ± in the text.

Counts. A number of independent events observed in a fixed window is Poisson distributed, so a count N carries

$$\sigma_N = \sqrt{N}, \qquad \sigma_{N/T} = \frac{\sqrt{N}}{T} \tag{A.1}$$

for a rate over an exposure T that is itself known exactly. We checked that assumption on the data: over the 1500 beam-on frames the per-frame lithium count has mean 39.51 and variance 38.62, a Fano factor of 0.977, and the standard error of the mean measured over the frames is 0.406 % against the 0.411 % that $\sqrt{N}$ predicts, an agreement of one percent. The Fano factor carries an uncertainty of $\sqrt{2/(N-1)} = 0.037$ on 1500 frames, so 0.977 sits $0.6\,\sigma$ from unity: the counting is consistent with Poisson, which is what this check can establish, and not Poisson to a fraction of a percent. This gives, for example, $R = 71.1 \pm 0.3\,\mathrm{s}^{-1}$ from 59 259 captures over 833.1 s, and every per-frame rate quoted in Section 4.

Fractions. A fraction $p = k/N$ of a counted population is binomial,

$$\sigma_p = \sqrt{\frac{p\,(1-p)}{N}}, \tag{A.2}$$

which is what puts ±0.03 % on the 99.49 % of clusters above five pixels that saturate, and ±0.25 % on the 33.07 % of lost pixels that fall inside a track footprint.

Correlations. A Pearson coefficient is not a count and does not follow either of the rules above. The one quoted in Section 4.7, $r = 0.493 \pm 0.016$ between the dead-pixel density and the track density, is computed over the 2166 blocks of 96×96 pixels the sensor divides into whole, and its uncertainty is the standard error of Fisher's z-transform,

$$z = \operatorname{artanh} r, \qquad \sigma_z = \frac{1}{\sqrt{N-3}}, \tag{A.3}$$

transformed back, which for $N = 2\,166$ gives the interval $[0.477, 0.509]$.

Derived quantities. Where a number is built from others, the general first-order propagation applies:

$$\sigma_f^2 = \sum_i \left(\frac{\partial f}{\partial x_i}\right)^2 \sigma_{x_i}^2. \tag{A.4}$$

Three instances occur. The enrichment of lost pixels inside the track footprint is a ratio of a measured fraction to an exact geometric one, so it inherits the relative error of the numerator alone, giving 6.79 ± 0.05. The fraction of the available room closed by a classifier is a linear rescaling of its accuracy, $(a - b)/(100 - b)$ with the baseline b fixed by the test set, so the uncertainty scales by the same factor: 95.75 ± 2.20 becomes 89.1 ± 5.6 %. And the difference between two such numbers adds in quadrature, which is how Section 4.8 compares this work with [6] across the range of neutron fractions that thesis leaves open.

Spreads over repetitions. Four quantities in this thesis carry a plus-or-minus that is neither a count nor a propagation, but the sample standard deviation over a set of repetitions, and each one measures only what its repetitions vary. The ± 2.20 and ± 1.90 of Section 4.4 are taken over 1000 refits of one labelled set, so they carry the choice of training split and the randomness of the forest and not the sampling of the labels. The spreads in Table 3.2 are taken over ten runs of one binary on one folder of frames, so they carry the state of the machine and not any variation in the detector, which returns the same counts every time. The $\pm 0.11\,\%$ of the null model in Section 4.7 is the spread of five draws and should be read as an indication of scale rather than an error bar. And the per-third rates of that section are standard errors of the mean over the frames of each third, which is wider than Eq. (A.1) would give because the frame-to-frame scatter carries the beam as well as the counting.

What carries no uncertainty, and why. Three kinds of number in this thesis are quoted bare on purpose. The per-pixel statistics of the background model are computed over 20.1 million pixels, so their sampling error is of order 10^{-4} and showing it would be misleading rather than informative. The fraction of the sensor covered by track footprints is a geometric property of a known set of rectangles, not a sample. And the cluster counts of Section 4 are exact for the run they describe — the detector is deterministic, and regenerating the whole analysis returns the same numbers cluster for cluster — so a $\pm$ on them would describe a spread that does not exist. Where those counts are used as an estimate of a *rate*, the Poisson error of Eq. (A.1) is what applies and is quoted.

B. The dead-pixel rule

Section 4.7 rests on a definition rather than a measurement, and this is it: a pixel that crosses the detection threshold on five consecutive frames is retired. The counter is reset by any frame in which the pixel is quiet, so the five must be consecutive, and a retired pixel is removed from the event mask of the same frame that retires it.

```
110     // -- Persistent-event pixels become dead pixels ------------------------
111     // Increment the streak where the pixel is an event, reset it elsewhere;
112     // pixels reaching DEAD_EVENT_STREAK are hot (real particles never hit
113     // the same pixel every frame) and are masked from now on --- including
114     // from THIS frame's mask, so they stop polluting clusters and the
115     // arrival heatmap immediately.
116     {
117         if (event_streak_.empty())
118             event_streak_ = cv::Mat::zeros(pixels.size(), CV_32FC1);  // lazy: resume path
119         cv::Mat event01;
120         event_mask.convertTo(event01, CV_32FC1, 1.0 / 255.0);
121         event_streak_ = (event_streak_ + event01).mul(event01);
122         cv::Mat stuck = event_streak_ >= static_cast<float>(DEAD_EVENT_STREAK);
123         int n_stuck = cv::countNonZero(stuck);
124         if (n_stuck > 0) {
125             if (dead_mask_.empty())
126                 dead_mask_ = cv::Mat::zeros(pixels.size(), CV_8UC1);
127             dead_mask_.setTo(255, stuck);
128             event_mask.setTo(0, stuck);
129             event_streak_.setTo(0.f, stuck);
130             // stderr: stdout is a machine-parsed protocol in server mode.
131             fprintf(stderr,
132                     "[BackgroundModel] %d pixel(s) event for %d consecutive "
133                     "frames -> added to dead mask (frame %d)\n",
```

Code excerpt 4: `BackgroundModel::update()` — retiring a pixel that fires on `DEAD_EVENT_STREAK` consecutive frames (`Code/src/BackgroundModel.cpp`).

The five is a choice, not a measurement, and Section 5 lists what a better definition would look like.

Acknowledgements

I thank the *q*B*ounce* group at the Atominstitut for taking me in and for letting a bachelor student near a real experiment, and the team at the Institut Laue-Langevin for how they received me during the ten days of beam time in Grenoble. The CMOS detector this thesis is about was mounted and tested there, and most of what the software had to be able to survive I learnt in that week rather than at a desk.

References

[1] V. V. Nesvizhevsky et al. "Quantum states of neutrons in the Earth's gravitational field". In: *Nature* 415 (2002), p. 297.

[2] T. Jenke, P. Geltenbort, H. Lemmel, and H. Abele. "Realization of a gravity-resonance-spectroscopy technique". In: *Nature Physics* 7 (2011), p. 468.

[3] G. Cronenberg et al. "Acoustic Rabi oscillations between gravitational quantum states and impact on symmetron dark energy". In: *Nature Physics* 14 (2018), p. 1022.

[4] H. Abele. *Das Neutron im freien Fall*. Deutsches Museum. 0.

[5] T. Jenke, G. Cronenberg, H. Filter, P. Geltenbort, M. Klein, T. Lauer, K. Mitsch, H. Saul, D. Seiler, D. Stadler, M. Thalhammer, and H. Abele. "Ultracold neutron detectors based on ^{10}B converters used in the qBounce experiments". In: *Nuclear Instruments and Methods in Physics Research Section A* 732 (2013), pp. 1–8. DOI: 10.1016/j.nima.2013.06.024.

[6] H. M. Filter. "Interference Experiment with Slow Neutrons: A Feasibility Study of Lloyd's Mirror at the Institut Laue-Langevin". Dissertation, supervised by H. Abele, Atominstitut (E141). PhD thesis. Technische Universität Wien, 2018.

[7] S. Navas et al. "Review of Particle Physics". In: *Physical Review D* 110 (2024). Particle Data Group, p. 030001. DOI: 10.1103/PhysRevD.110.030001.

[8] G. Cronenberg, K. Durstberger-Rennhofer, and P. Geltenbort. "Methods and applications of gravity resonance spectroscopy". In: *Journal of Physics: Conference Series* 340 (2012), p. 012045. DOI: 10.1088/1742-6596/340/1/012045.

[9] G. Pignol, S. Baeßler, V. V. Nesvizhevsky, K. Protasov, D. Rebreyend, and A. Voronin. "Gravitational Resonance Spectroscopy with an Oscillating Magnetic Field Gradient in the GRANIT Flow through Arrangement". In: *Advances in High Energy Physics* 2014 (2014), p. 628125. DOI: 10.1155/2014/628125.

[10] J. Bosina et al. *qBounce: First Measurement of the Neutron Electric Charge with a Ramsey-type GRS Experiment*. arXiv:2301.05984. 2023.

[11] E. J. Sung, B. Koch, T. Jenke, H. Abele, and D. I. Bondar. *Generalized Boundary Conditions for the qBounce Experiment*. arXiv:2510.15341. 2025.

[12] H. Abele, T. Jenke, H. Leeb, and J. Schmiedmayer. "Ramsey's method of separated oscillating fields and its application to gravitationally induced quantum phase shifts". In: *Phys. Rev. D* 81 (2010), p. 065019.

[13] T. Jenke. "qBounce – vom Quantum Bouncer zur Gravitationsresonanzspektroskopie". PhD thesis. Technische Universität Wien, 2011.

[14] F. J. Rodríguez-Fortuño. "An optical double-slit experiment in time". In: *Nature Physics* 19 (2023), pp. 929–930. DOI: 10.1038/s41567-023-02026-2.

[15] J. Micko, J. Bosina, S. S. Cranganore, T. Jenke, M. Pitschmann, S. Roccia, R. I. P. Sedmik, and H. Abele. *qBounce: Systematic shifts of transition frequencies of gravitational states of ultra-cold neutrons using Ramsey gravity resonance spectroscopy*. arXiv:2301.08583. 2023.

[16] T. Rechberger. "Ramsey spectroscopy of gravitationally bound quantum states of ultracold neutrons". MA thesis. Wien: Technische Universität Wien, 2018.

[17] T. Jenke et al. "Gravity Resonance Spectroscopy Constrains Dark Energy and Dark Matter Scenarios". In: *Phys. Rev. Lett.* 112 (2014), p. 151105.

[18] P. Brax and G. Pignol. "Strongly Coupled Chameleons and the Neutronic Quantum Bouncer". In: *Phys. Rev. Lett.* 107 (2011), p. 111301.

[19] H. Fischer and R. I. P. Sedmik. *Numerical Methods for Scalar Field Dark Energy in Table-top Experiments and Lunar Laser Ranging*. arXiv:2401.16179. 2024.

[20] S. Sponar, R. I. P. Sedmik, M. Pitschmann, H. Abele, and Y. Hasegawa. "Tests of fundamental quantum mechanics and dark interactions with low-energy neutrons". In: *Nature Reviews Physics* 3 (2021), pp. 309–327. DOI: 10.1038/s42254-021-00298-2.

[21] L. Posta. *qBOUNCE CMOS Detection Pipeline — User Guide*. Shipped with the offline lab kit as `docs/UserGuide.tex`. Technische Universität Wien, Atominstitut. 2026.